

OOT Syllabus

Object orientation, Encapsulation, information hiding, polymorphism, generosity, Object Oriented modelling, UML, Structural Modelling, Behavioural. Modelling and Architectural Modelling. Object Oriented Analysis, Object oriented design, Object design. Structured analysis and structured design (SA/SD), Jackson Structured Development (JSD). Object oriented programming style Introduction to Java, Java Beans, Enterprise Java beans (EJB), Java Swing; **Java as internet programming language**. The connectivity model, JDBC/ODBC, Bridge, Introduction to servlets.

Chapter -1

Object-Oriented Technology

How Software is Constructed

- Wanted:
 - robust large-scale applications that evolve with the corporation
- It isn't easy!
- Modular Programming
 - Break large-scale problems into smaller components that are constructed independently
 - Programs were viewed as a collection of procedures, each containing a sequence of instructions

Modular Programming

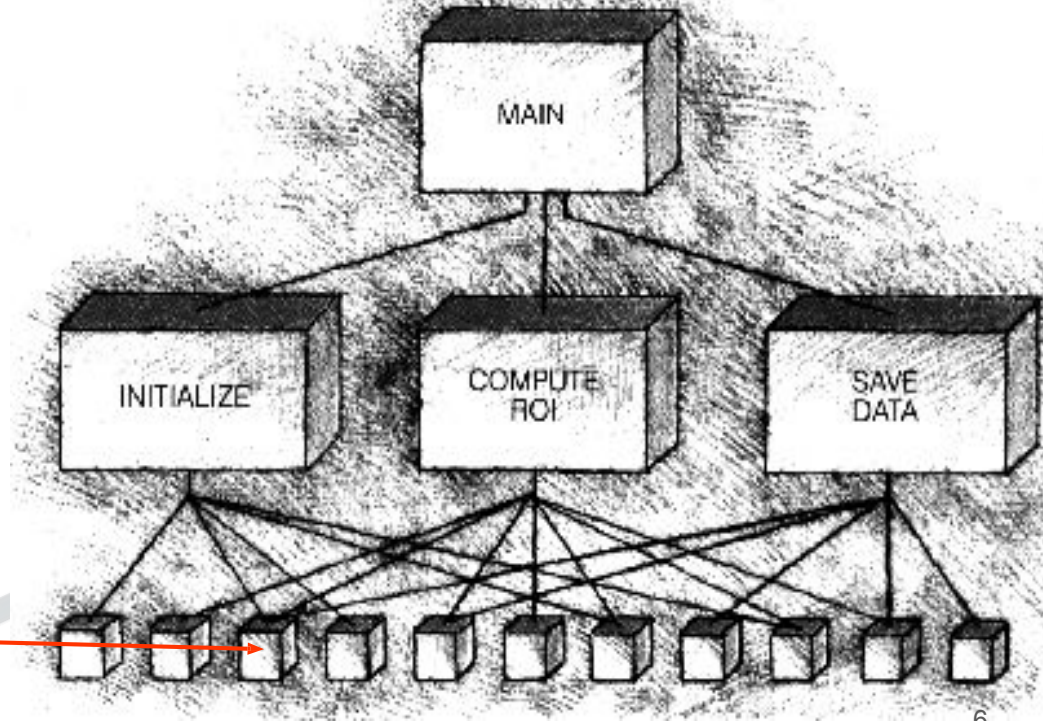
- Subroutine (1960s)
 - Provided a natural division of labor
 - Could be reused in other programs
- Structured Programming and Design (1970s)
 - It was considered a good idea to program with a limited set of control structures (no go to statements, single returns from functions)
 - sequence, selection, repetition, recursion
 - Program design was at the level of subroutines
 - functional decomposition

What About the Data?

- Software development had focused on the modularization of code,
 - the data was either moved around between functions via argument/parameter associations
 - or the data was global
 - works okay for smaller programs or for big programs when there aren't too many global variables
 - Not good when variables number in the hundreds

Don't use Global Variables

- Sharing data (global variables) is a violation of modular programming
- This makes all modules dependent on one another
 - this is dangerous



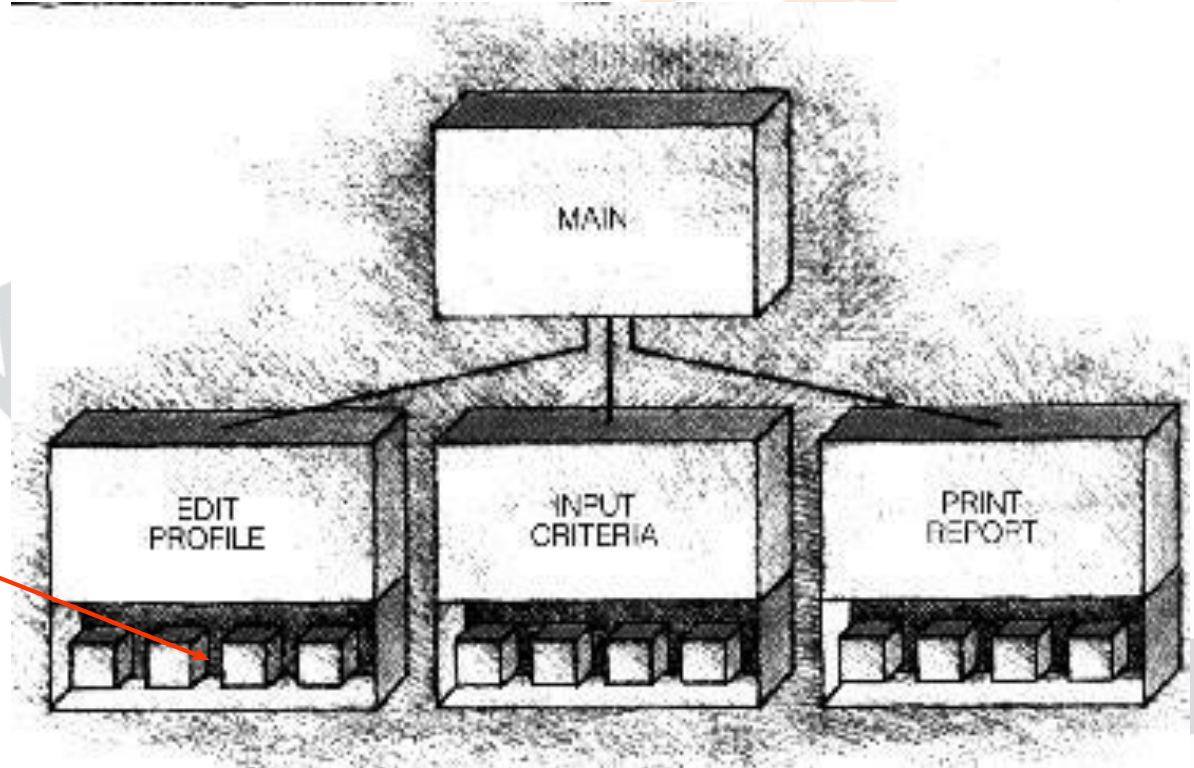
Information (Data) Hiding

- An improvement:
 - Give each subroutine it's own local data
 - This data can only be “touched” by that single subroutine
 - Subroutines can be designed, implemented, and maintained independently
- Other necessary data is passed amongst the procedures via argument/parameter associations.

Use functions

- Localize data inside the functions
- This makes functions more independent of one another

- Local Data



The Structured Design Approach

- The procedural style of programming, using structured design builds systems one subroutine at a time
 - This approach doesn't work well in large systems
 - The result is defective software; difficult to maintain
- There is a better way, the OO way:
 - A 10,000 statement program (in 1960) becomes structured with 1,000 functions (in 1980) where there are 10 statements in each function, which then becomes (in 1990), a program with 100 classes, with 10 functions each, 10 statements each

“The one thing we missed was putting the data and functions together”

Larry Constantine, author of Structured Design

Object-Oriented Technology

- OOT began with Simula 67
 - developed in Norway
 - acronym for simulation language
- Why this “new” language?
 - to build accurate models of complex working systems
- The modularization occurs at the physical object level (not at a procedural level)

1. Object-Oriented Technology (OOT)

- A **broad concept / methodology** for building software systems based on the idea of **objects**.

Encompasses:

Object-Oriented Analysis (OOA) → studying the problem domain in terms of objects.

- **Object-Oriented Design (OOD)** → designing system architecture with objects, classes, relationships, generalization, etc.
- **Object-Oriented Programming (OOP)** → implementation step in a language like C++, Java, Python.

- OOT = **philosophy + methods + tools** for the full software lifecycle.

OO Analysis and Design

- New methods and technologies for analysis, design, and testing have been created to deal with larger systems
 - Use cases help analyze and capture requirements
 - Responsibility-Driven Design helps determine which objects are needed and what they must do
 - Test Driven Development aids in implementation, design, and maintenance of software
 - Unified Modeling Language (UML) is used to design large systems

Q. Object-Oriented Technology (OOT) primarily focuses on:

- A) Data flow and process decomposition
- B)  Objects that combine data and behavior
- C) Procedural abstraction and modular programming
- D) Sequential execution of functions

- Q. Object-Oriented Technology (OOT)** primarily focuses on:
- A) Data flow and process decomposition
 - B) Objects that combine data and behavior
 - C) Procedural abstraction and modular programming
 - D) Sequential execution of functions

Answer: B) Objects that combine data and behavior

Explanation:

OOT integrates data and functions that operate on that data into a single unit called an object. This promotes encapsulation, reusability, and better modeling of real-world systems.

Q. Which of the following correctly matches the OOT stages?

1. OOA → Problem domain study
2. OOD → System architecture
3. OOP → Implementation

- A) 1 and 3 only
- B) 2 and 3 only
- C) 1, 2, and 3
- D) 1 and 2 only

Which of the following correctly matches the OOT stages?

1. OOA → Problem domain study
2. OOD → System architecture
3. OOP → Implementation

C) 1, 2, and 3

Explanation:

- **OOA (Object-Oriented Analysis):**
Understands real-world objects and interactions.
- **OOD (Object-Oriented Design):** Structures these objects into a software design.
- **OOP (Object-Oriented Programming):**
Implements them using languages like C++ or Java.

- A) 1 and 3 only
- B) 2 and 3 only
- C) 1, 2, and 3
- D) 1 and 2 only

Q. Which of the following statements about **Object-Oriented Technology (OOT) is **FALSE**?**

- A) It includes both analysis and design methodologies.
- B) It is limited to programming languages only.
- C) UML is a key tool for system design.
- D) It supports the entire software development lifecycle.

Q. Which of the following statements about Object-Oriented Technology (OOT) is FALSE?

- A) It includes both analysis and design methodologies.
- B) It is limited to programming languages only.
- C) UML is a key tool for system design.
- D) It supports the entire software development lifecycle.

Answer: B) It is limited to programming languages only.

Explanation:

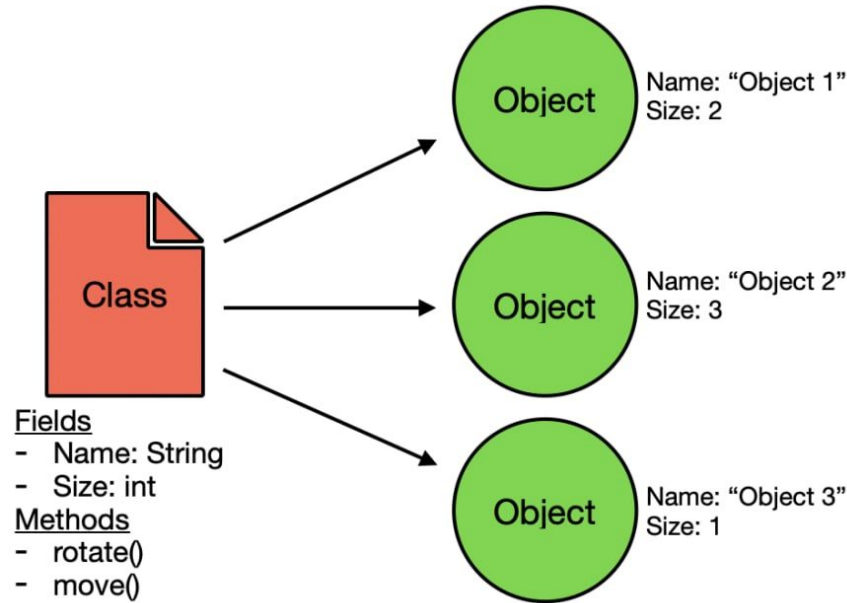
OOT is **not limited to programming** — it's a complete methodology involving analysis, design, implementation, and testing using object concepts.

Object

- An object is:
 - a software “package” that contains a collection of related methods and data
 - an excellent software module
 - an instance of a class
- We understand an object through:
 - the values the object stores (state)
 - the services the object provides (behavior)

Class

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.



**Each object has its own values for fields and can call the methods found in the class*

Objects

An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. An object has an identity, state, and behavior. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

For example "Dog" is a real-life Object, which has some characteristics like color, Breed, Bark, Sleep, and Eats.

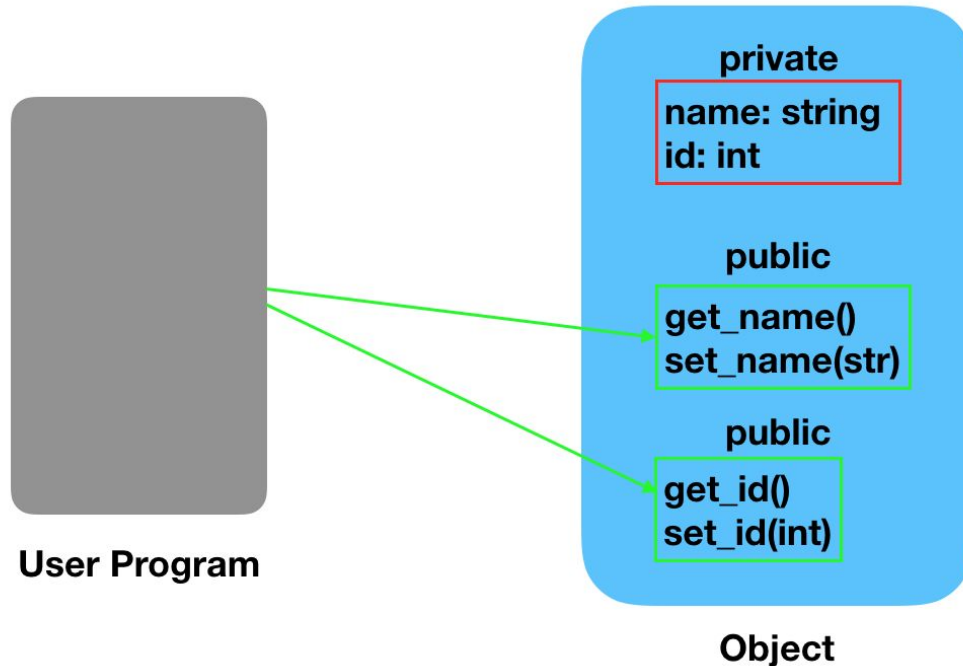


Message

- Real-world objects exhibit many effects on each other.
- These interactions are represented in software as messages (a.k.a. method or function calls).
- A message has these three things:
 - *sender*: the initiator of the message
 - *receiver*: the recipient of the message
 - *arguments*: data required by the receiver

Data Abstraction

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.



Encapsulation

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as **data-hiding**.

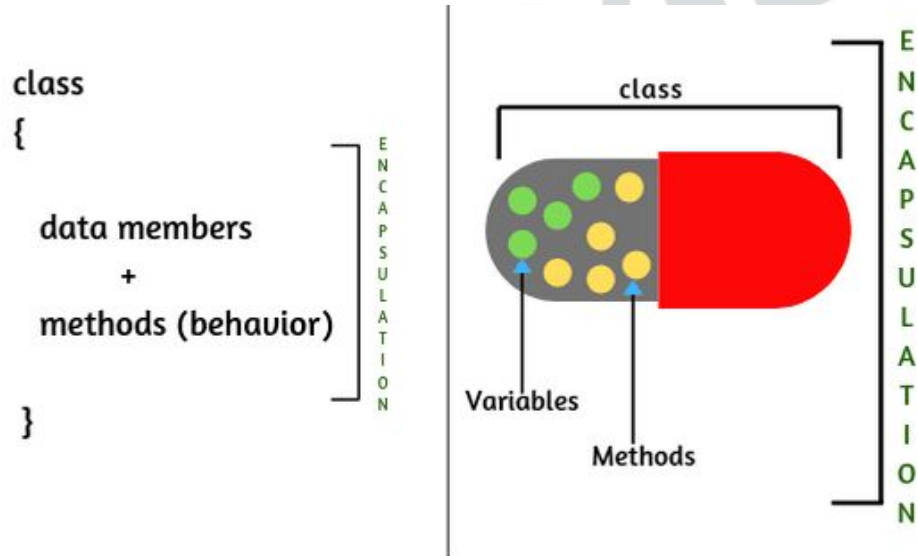
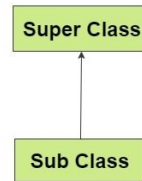


Fig: Encapsulation

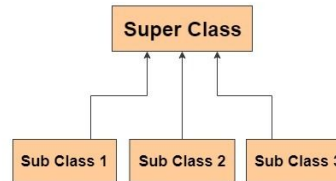
Inheritance

Inheritance is an important pillar of OOP(Object-Oriented Programming). The capability of a class to derive properties and characteristics from another class is called Inheritance. When we write a class, we inherit properties from other classes. So when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and reduce its redundancy.

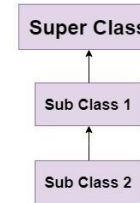
Single Inheritance



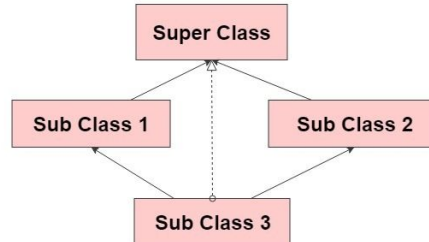
Hierarchial Inheritance



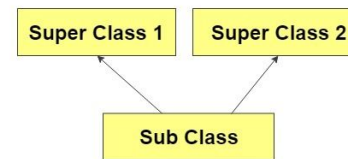
MultiLevel Inheritance



Hybrid Inheritance



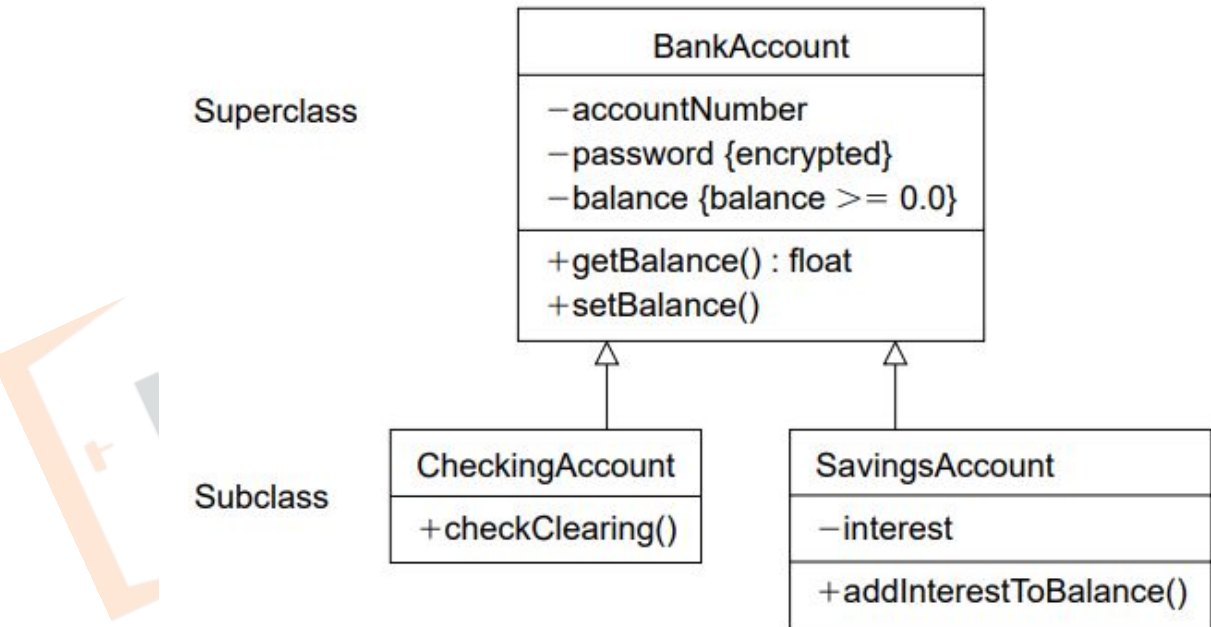
Multiple Inheritance



Properties of Inheritance

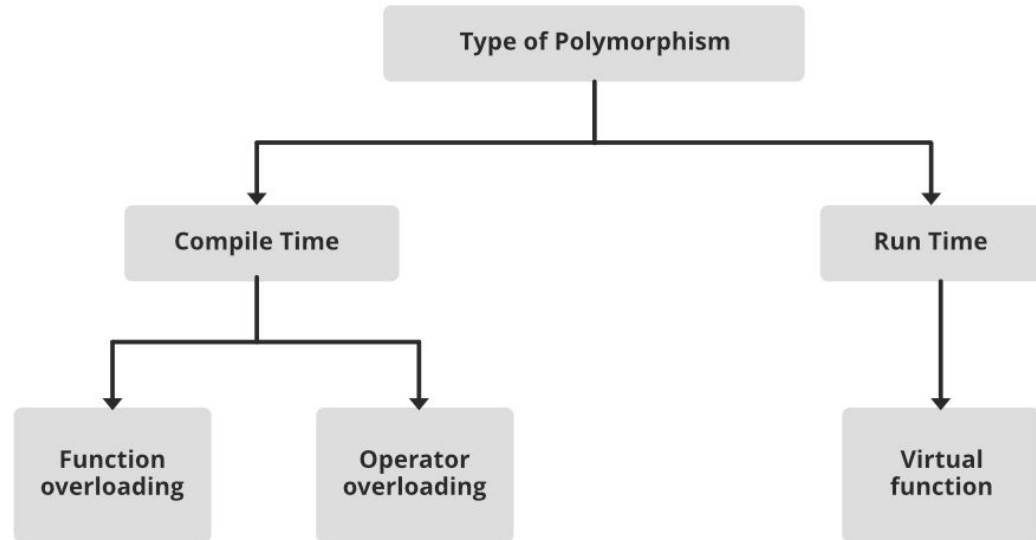
Generalization

The purpose of this property is to distribute the commonalities from the superclass among a group of similar subclasses. The subclass inherits all the superclass's (base class) operations and attributes. That is, whatever the superclass possesses, the derived class (subclass) does as well.



Polymorphism

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

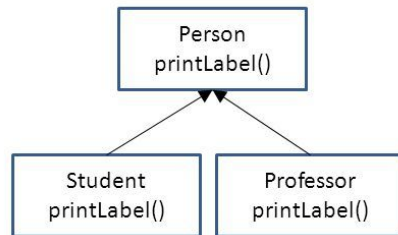


Dynamic Binding

Dynamic binding, also known as late binding or runtime binding, is a programming concept related to polymorphism in object-oriented languages. It refers to the process of determining the specific method or function to be executed at runtime, rather than at compile time. Dynamic binding is a key feature in languages that support polymorphism and inheritance.

Dynamic Method Binding

- Ability to use a derived class in a context that expects its base class



```
Student s = new Student()
Professor p = new Professor()
```

```
Person x = s;
Person y = p;
```

```
s.printLabel();
p.printLabel();
```

```
x.printLabel(); // Which one?
y.printLabel();
```

Q.Object oriented programmers primarily focus on (RPSC Lect. 2011)

- (a) procedure to be performed
- (b) the step by step statements needed to solve a problem
- (c) the physical orientation of objects within a program
- (d) objects and the tasks that must be performed with those objects

Q. Object oriented programmers primarily focus on (RPSC Lect. 2011)

- (a) procedure to be performed
- (b) the step by step statements needed to solve a problem
- (c) the physical orientation of objects within a program
- (d) objects and the tasks that must be performed with those objects

In **Object-Oriented Programming (OOP)**, the focus shifts from *procedures* (as in procedural programming) to *objects* — which are real-world entities represented in software.

Q. Which of the following is the main advantage of object-oriented programming?

- A.** Reduces code size only
- B.** Enhances code reusability and maintainability
- C.** Increases program execution speed
- D.** Eliminates the need for testing

Q. Which of the following is the main advantage of object-oriented programming?

- A. Reduces code size only
- B. Enhances code reusability and maintainability
- C. Increases program execution speed
- D. Eliminates the need for testing

Answer: (b)

Explanation: OOP supports *code reusability* through inheritance and *maintainability* through modular design. It helps in organizing large software into manageable objects that can be reused and modified independently.

Q. Object oriented inheritance models: (NIELIT Scientists-B 04.12.2016 (CS))

- A. "is a kind of" relationship**
- B. "has a" relationship**
- C. "want to be" relationship**
- D. "contains" of relationship**

Q. Object oriented inheritance models: (NIELIT Scientists-B 04.12.2016 (CS))

- A. "is a kind of" relationship
- B. "has a" relationship
- C. "want to be" relationship
- D. "contains" of relationship

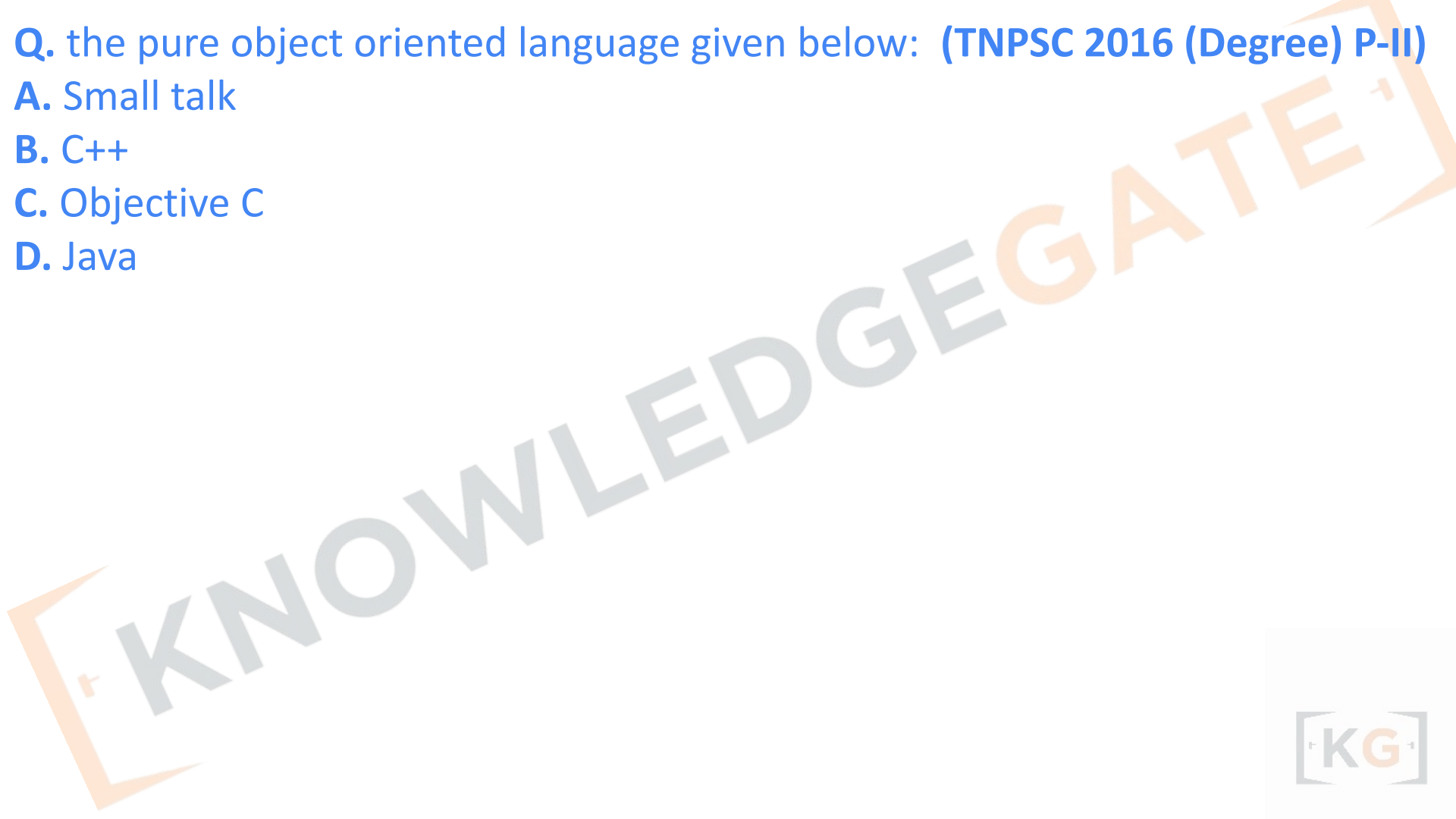
Answer: A

Inheritance → "is-a" relationship

Composition/Aggregation → "has-a" relationship

Q. the pure object oriented language given below: (TNPSC 2016 (Degree) P-II)

- A. Small talk**
- B. C++**
- C. Objective C**
- D. Java**



Q. the pure object oriented language given below: (TNPSC 2016 (Degree) P-II)

- A. Small talk
- B. C++
- C. Objective C
- D. Java

Smalltalk is considered the **purest form of Object-Oriented Programming language**, developed by **Alan Kay** and his team at **Xerox PARC**.

**Q. In the object-oriented design of software, what are the object components?
(APPSC Poly. Lect. 13.03.2020)**

- A. Attributes and name only**
- B. Operation and name only**
- C. Attributes, names and operations**
- D. Attributes and operations only**

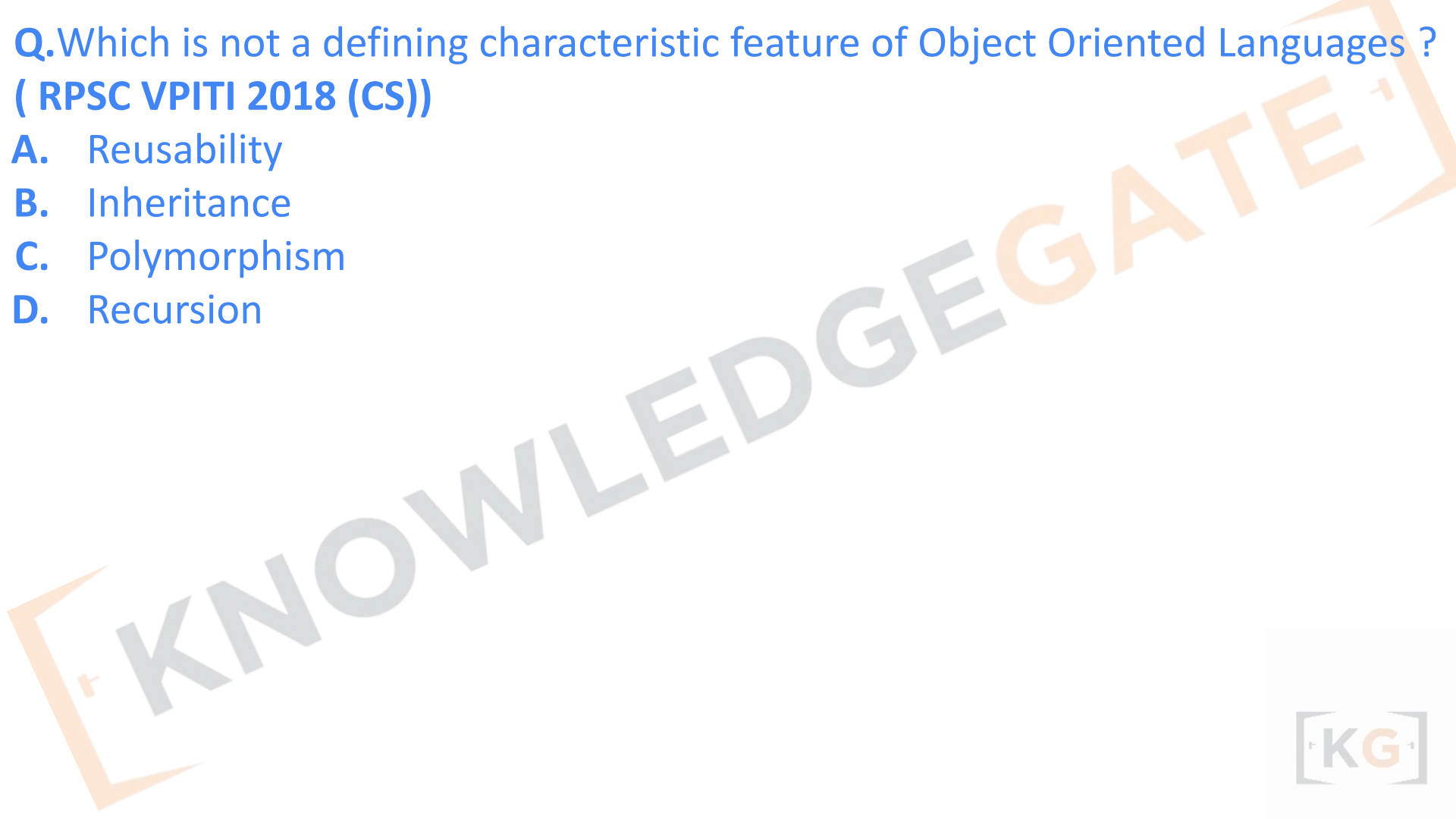
**Q. In the object-oriented design of software, what are the object components?
(APPSC Poly. Lect. 13.03.2020)**

- A. Attributes and name only**
- B. Operation and name only**
- C. Attributes, names and operations**
- D. Attributes and operations only**

Answer: (c) Attributes, names and operations

Q. Which is not a defining characteristic feature of Object Oriented Languages ?
(RPSC VPITI 2018 (CS))

- A. Reusability
- B. Inheritance
- C. Polymorphism
- D. Recursion



Q. Which is not a defining characteristic feature of Object Oriented Languages ?

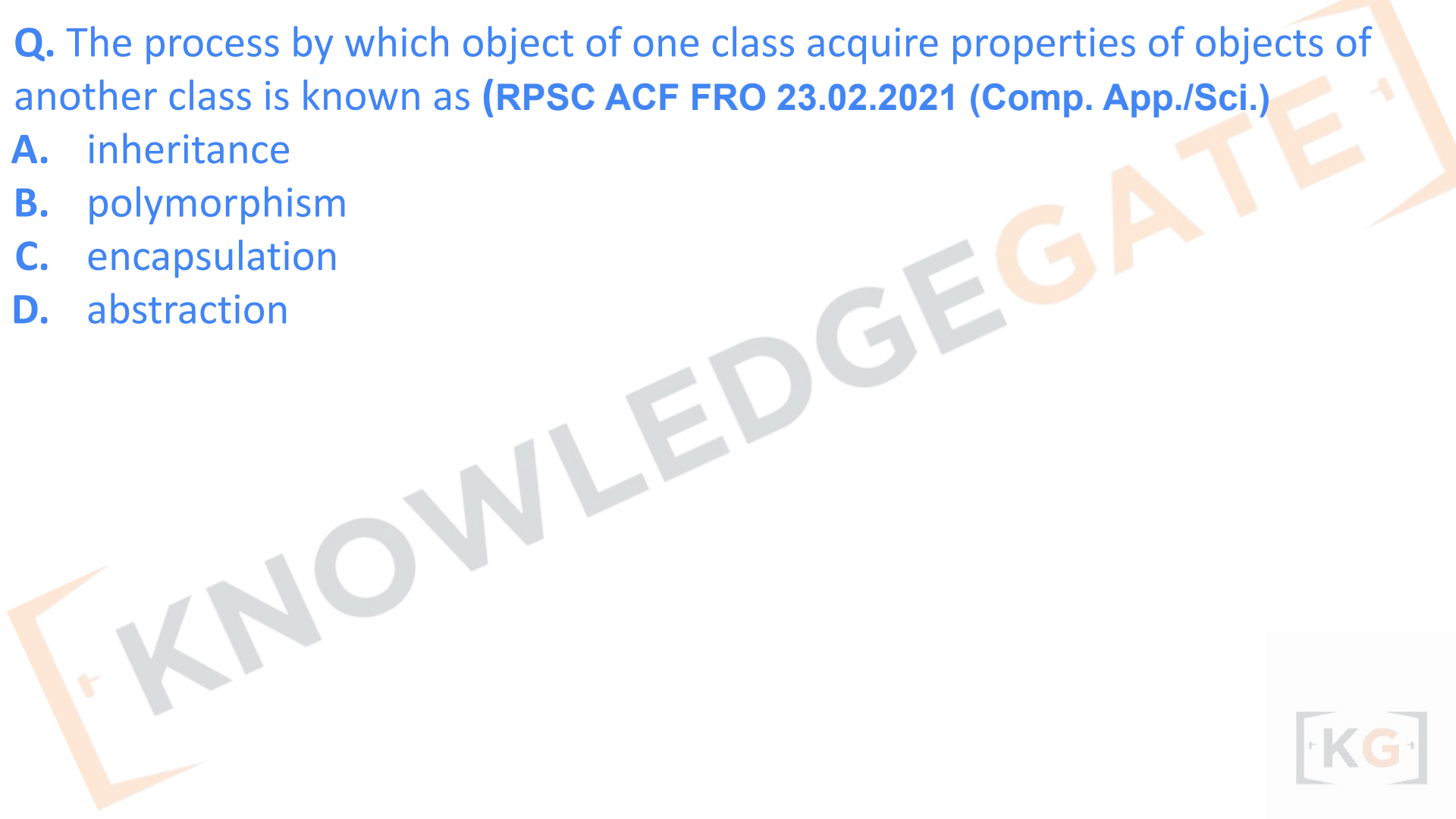
(RPSC VPITI 2018 (CS))

- A. Reusability
- B. Inheritance
- C. Polymorphism
- D. Recursion

Ans. (d)

Q. The process by which object of one class acquire properties of objects of another class is known as **(RPSC ACF FRO 23.02.2021 (Comp. App./Sci.)**

- A.** inheritance
- B.** polymorphism
- C.** encapsulation
- D.** abstraction



Q. The process by which object of one class acquire properties of objects of another class is known as **(RPSC ACF FRO 23.02.2021 (Comp. App./Sci.)**

- A.** inheritance
- B.** polymorphism
- C.** encapsulation
- D.** abstraction

Ans: (a)

Q. Match List I with List II:

	List(I)		List(II)
(A)	Localization	(I)	Encapsulation
(B)	Packaging or binding of a collection of item	(II)	Abstraction
(C)	Mechanism that enables designer to focus on essential details of a program Component.	(III)	Characteristics of software that indicates the manner in which information is concentrated in program.
(D)	Information hiding	(IV)	Suppressing the operational details of a Program component

Choose the correct answer from the options given below:(UGC NET PAPER-2022)

- (a) (A)-(I),(B)-(II),(C)-(III),(D)-(IV)
- (b) (A)-(II),(B)-(I),(C)-(III),(D)-(IV)
- (c) (A)-(III),(B)-(I),(C)-(II),(D)-(IV)
- (d) (A)-(III),(B)-(I),(C)-(IV),(D)-(II)

Q. Match List I with List II:

	List(I)		List(II)
(A)	Localization	(I)	Encapsulation
(B)	Packaging or binding of a collection of item	(II)	Abstraction
(C)	Mechanism that enables designer to focus on essential details of a program Component.	(III)	Characteristics of software that indicates the manner in which information is concentrated in program.
(D)	Information hiding	(IV)	Suppressing the operational details of a Program component

Choose the correct answer from the options given below:(UGC NET PAPER-2022)

- (a) (A)-(I),(B)-(II),(C)-(III),(D)-(IV)
- (b) (A)-(II),(B)-(I),(C)-(III),(D)-(IV)
- ✓ (c) (A)-(III),(B)-(I),(C)-(II),(D)-(IV)
- (d) (A)-(III),(B)-(I),(C)-(IV),(D)-(II)

Localization → Concentration of information

Encapsulation → Binding data and functions

Abstraction → Focusing on essential details

Information hiding → Suppressing operational details

Q: Match List (I) with List (II)

	List (I)		List (II)
A.	If the implementation of generated or derived classes differ only through a parameter we have	I.	To use class hierarchy
B.	If the actual types of objects used cannot be known at compile time, then we have	II.	Improved run-time efficiency
C.	If inline operations are essential and templates are used then we have	III.	To use templates
D.	To gain access to differing instances for derived classes through base we have	IV.	To use explicit casting

Choose the correct answer from the options given below: (UGC NET PAPER-2022)

- (a) A-(II), B-(IV), C-(I), D-(III)
- (b) A-(II), B-(I), C-(IV), D-(III)
- (c) A-(III), B-(I), C-(II), D-(IV)
- (d) A-(I), B-(II), C-(IV), D-(III)

Chapter - 2

Unified Modeling Language, Object Oriented modelling, UML, Structural Modelling, Behavioural. Modelling and Architectural Modelling

Object-Oriented Modelling (OOM)

Object-Oriented Modelling is a **conceptual approach or methodology** used to analyze, design, and represent a system using **object-oriented concepts**. It helps developers **visualize and structure** the system in terms of real-world entities and their interactions before coding.

Phases of OOM:

1. **Object-Oriented Analysis (OOA)** – Identifying objects and relationships from problem statements.
2. **Object-Oriented Design (OOD)** – Refining and organizing them into design-level models.
3. **Object-Oriented Implementation (OOI)** – Coding them in an OOP language (like Java, C++).

Example:

Modeling a *Library System* using objects such as *Book*, *Member*, *Librarian*, and defining their attributes and interactions.

So, **OOM = The process of modelling a system using OOP concepts.**



UML Diagram

- UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems. (used to **express Object-Oriented Models.**)
- UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997. OMG is continuously making efforts to create a truly industry standard.
- UML stands for Unified Modeling Language.
- UML is a pictorial language used to make software blueprints.
- UML can be described as a general purpose visual modeling language to visualize, specify, construct, and document software system.

Design phase- Uses for UML

UML Diagram used to specifying the structure of how a software system will be written and function, without actually writing the complete implementation.

- **as a sketch:** to communicate aspects of system
 - forward design: doing UML before coding
 - backward design: doing UML after coding as documentation
 - used to get rough selective ideas

- **as a blueprint:**

A complete design to be implemented – sometimes done with CASE (Computer-Aided Software Engineering) tools

- **as a programming language:** with the right tools, code can be auto-generated and executed from UML – only good if this is faster than coding in a "real" language

UML Diagram Classification—Static, Dynamic, and Implementation

- A software system can be said to have two distinct characteristics:
a structural, “static” part and a behavioral, “dynamic” part.

In addition to these two characteristics, an additional characteristic that a software system possesses is related to implementation.

- **Static:** The static characteristic of a system is essentially the structural aspect of the system. The static characteristics define what parts the system is made up of.
- **Dynamic:** The behavioral features of a system; for example, the ways a system behaves in response to certain events or actions are the dynamic characteristics of a system.
- **Implementation:** The implementation characteristic of a system is an entirely new feature that describes the different elements required for deploying a system.

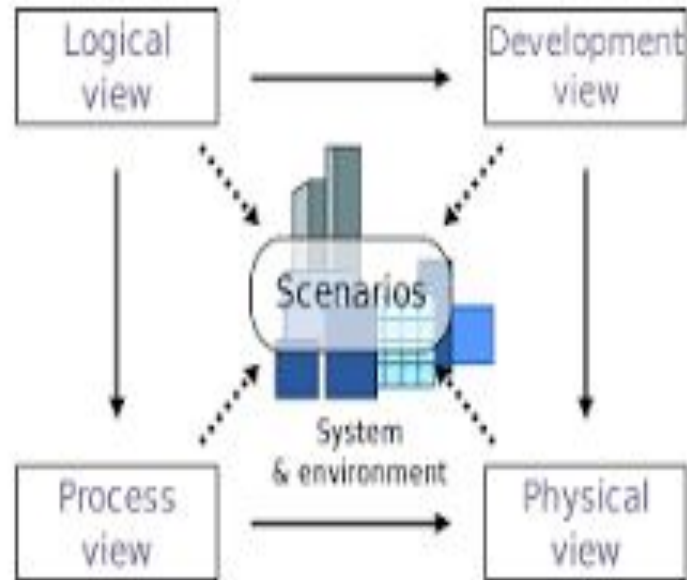
UML Diagram Classification—Static, Dynamic, and Implementation

The UML diagrams that fall under each of these categories are:

- Static
 - Use case diagram
 - Class diagram
- Dynamic
 - Object diagram
 - State diagram
 - Activity diagram
 - Sequence diagram
 - Collaboration diagram
- Implementation
 - Component diagram
 - Deployment diagram

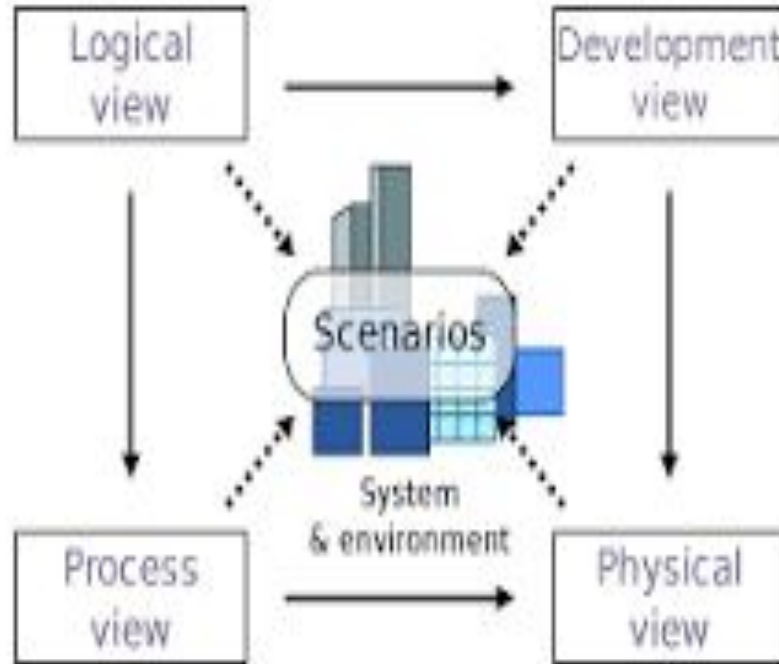
4+1 View

- The 4+1 view offers a different perspective to classify and apply UML diagrams. The 4+1 view is essentially how a system can be viewed from a software life cycle perspective. 4+1 is a view model used for **"describing the architecture of software-intensive systems, based on the use of multiple, concurrent views"**. The views are used to describe the system from the viewpoint of different stakeholders, such as end-users, developers, system engineers, and project managers. Each of these views represents how a system can be modeled. This will enable us to understand where exactly the UML diagrams fit in and their applicability.



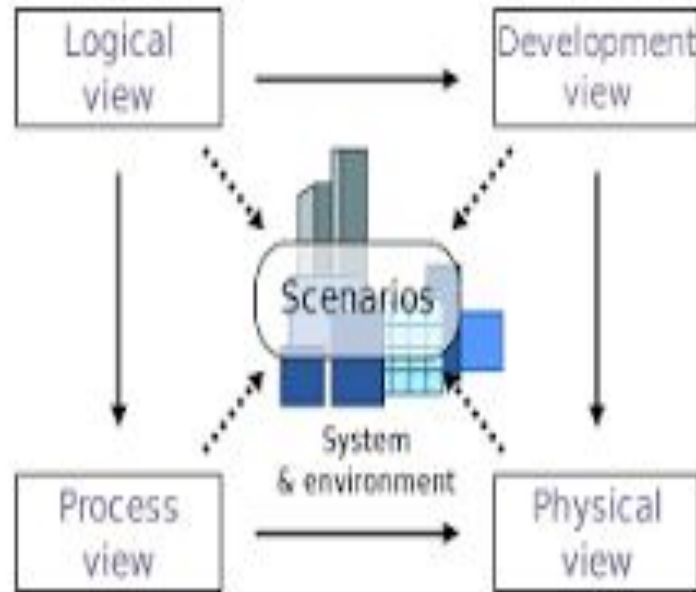
4+1 View

- **Design View:** The *design view* of a system is the structural view of the system. This gives an idea of what a given system is made up of. Class diagrams and object diagrams form the design view of the system.
- **Process View:** The dynamic behavior of a system can be seen using the *process view*. The different diagrams such as the state diagram, activity diagram, sequence diagram, and collaboration diagram are used in this view.



4+1 View

- **Component View:** Next, you have the *component view* that shows the grouped modules of a given system modeled using the component diagram.
- **Deployment View:** The deployment diagram of UML is used to identify the deployment modules for a given system. This is the *deployment view* of the
- **Use case View:** Finally, we have the *use case view*. Use case diagrams of UML are used to view a system from this perspective as a set of discrete activities or transactions.



Q. Which of the following UML diagrams has a static view? (UGC NET OCT-2020)

- A. Collaboration diagram**
- B. Use-case diagram**
- C. State chart diagram**
- D. Activity diagram**

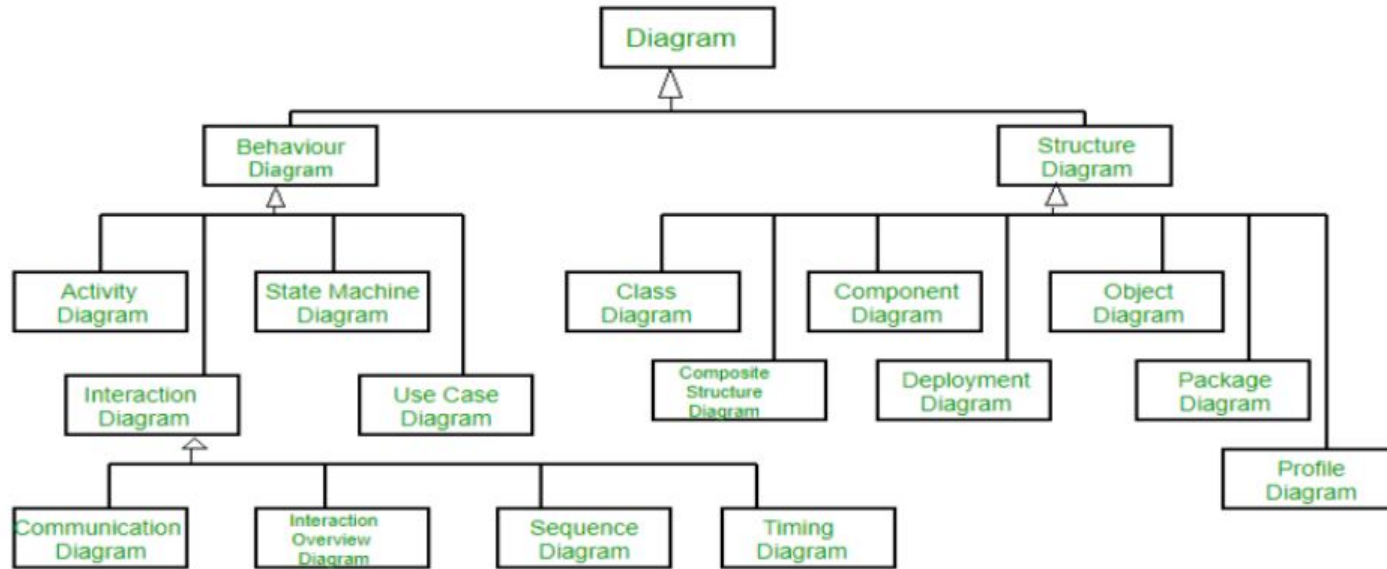
Q. Which of the following UML diagrams has a static view? (UGC NET OCT-2020)

- A. Collaboration diagram**
- B. Use-case diagram**
- C. State chart diagram**
- D. Activity diagram**

Correct: B

Structural diagrams and behavioural or interaction Modeling

**UML diagrams are organized into two distinct groups:
Structural diagrams and behavioural or interaction diagrams.**

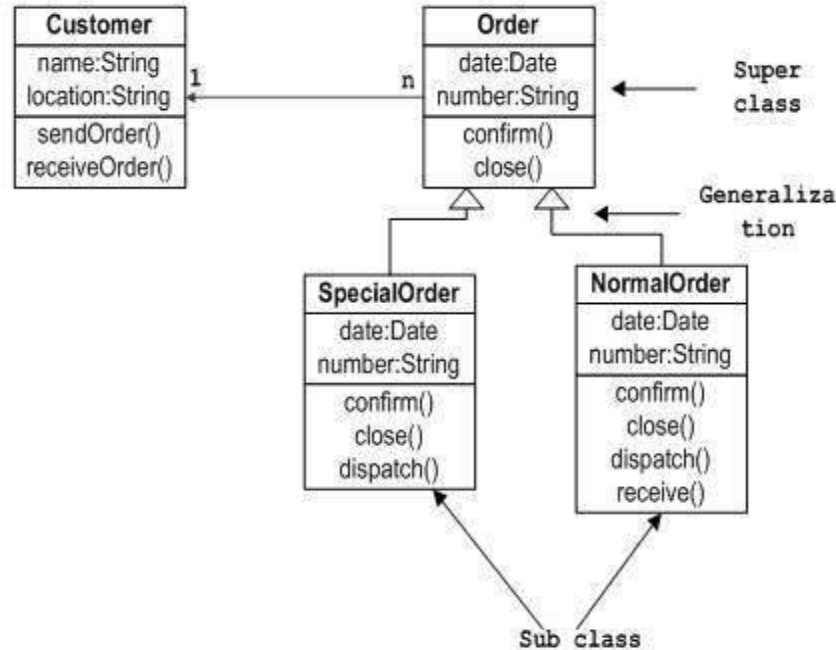


Types of UML Diagrams Class diagram

Class diagrams are the backbone of almost every object-oriented method, including UML. They describe the static structure of a system.

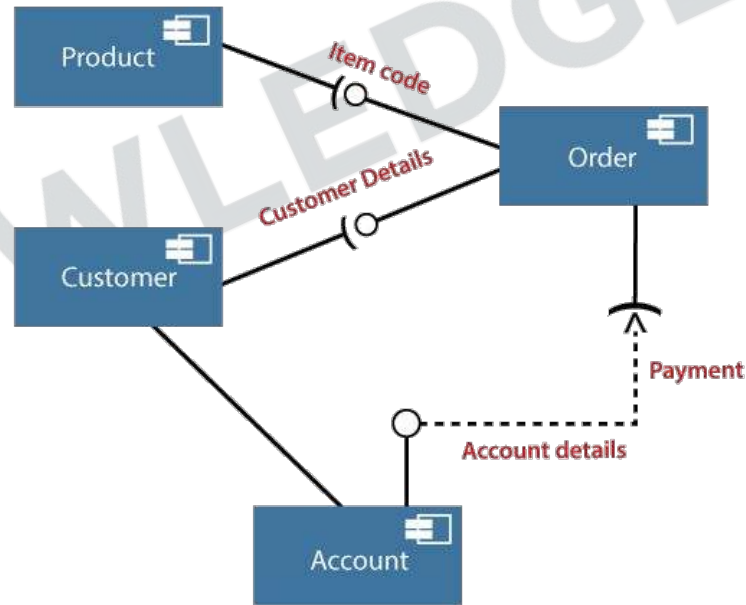
In most modeling tools, a class has three parts. Name at the top, attributes in the middle and operations or methods at the bottom. In a large system with many related classes, classes are grouped together to create class diagrams.

Sample Class Diagram



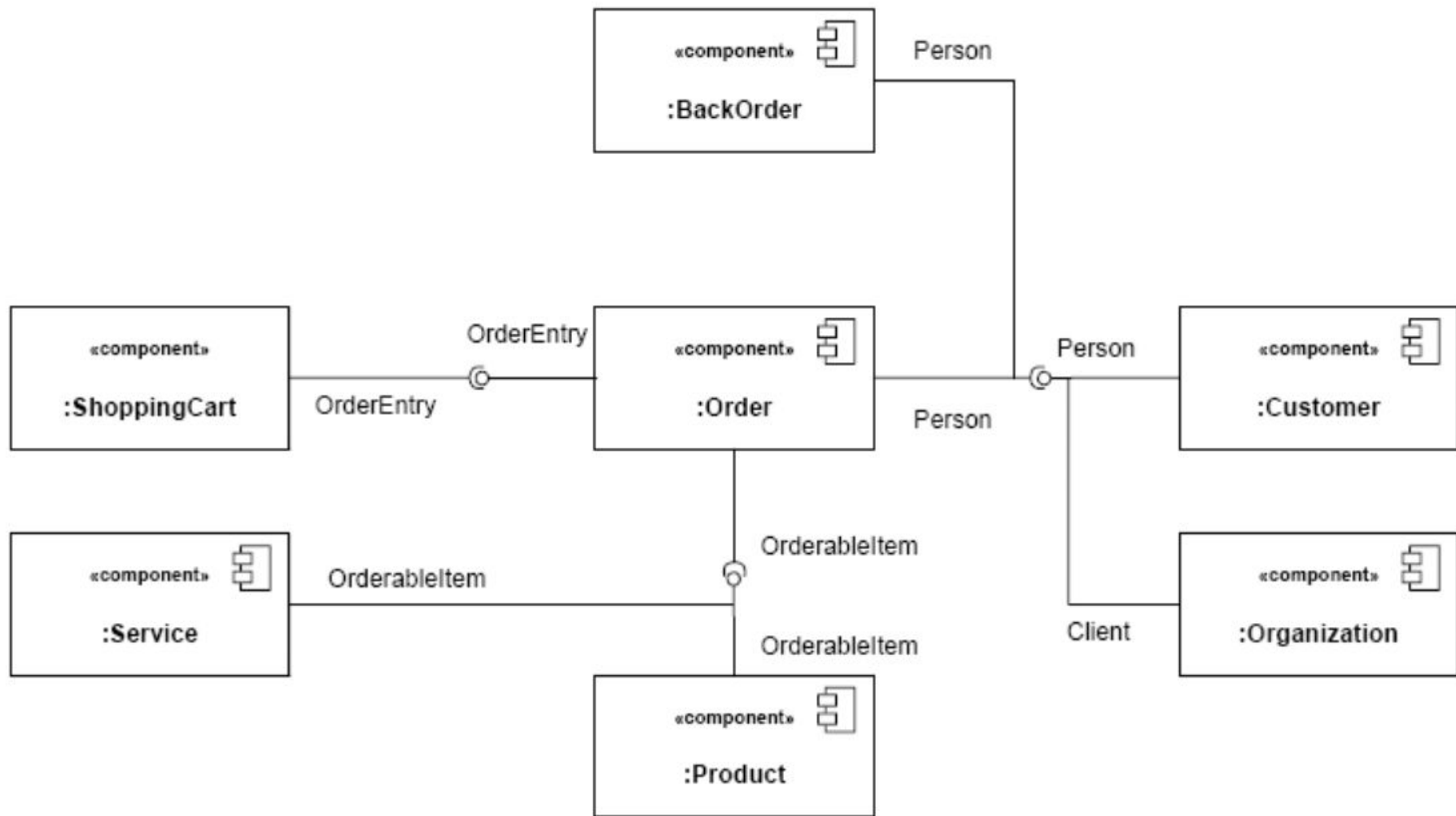
Component Diagram

- A component diagram displays the structural relationship of components of a software system.
- Component diagrams describe the organization of physical software components, including source code, run-time (binary) code, and executables.
- These are mostly used when working with complex systems with many components. Components communicate with each other using interfaces.
- The interfaces are linked using connectors.



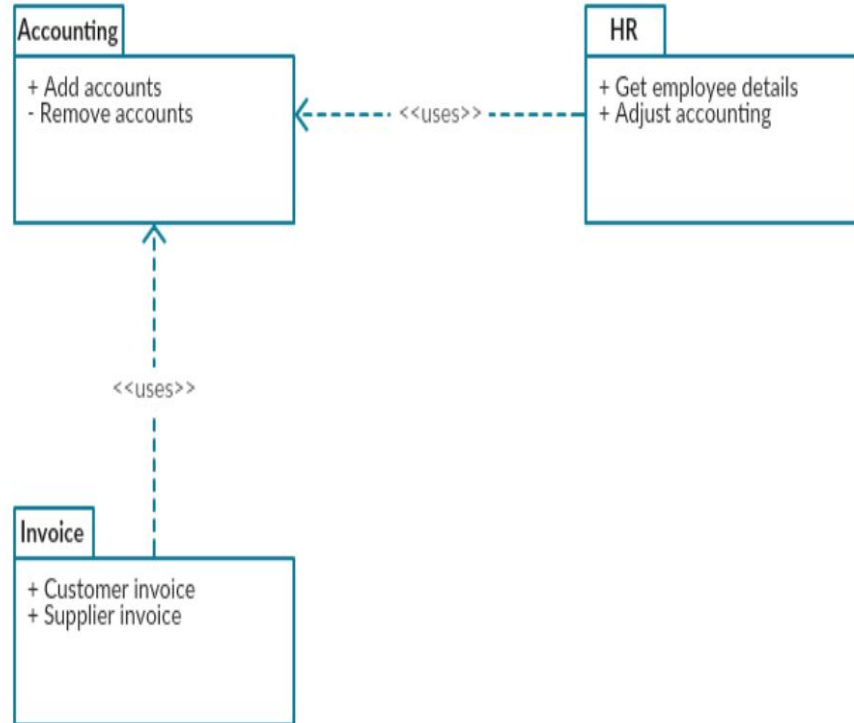
Composite Structure Diagram

- Composite structure diagrams are used to show the internal structure of a class.
- A composite structure diagram is a UML structural diagram that contains classes, interfaces, packages, and their relationships, and that provides a logical view of all, or part of a software system.
- It shows the internal structure (including parts and connectors) of a structured classifier or collaboration.



Package Diagram

- Package diagrams are a subset of class diagrams, but developers sometimes treat them as a separate technique.
- Package diagrams organize elements of a system into related groups to minimize dependencies between packages.



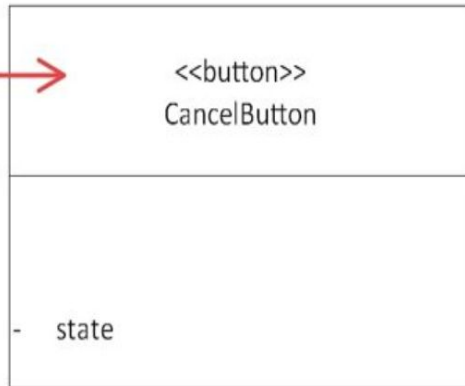
- **Profile diagrams** are an extension of Unified Modelling Language (UML), which are a structural diagram that use stereotypes, tagged values and constraints to extend and customize UML.

- Stereotyped depict the way that each object or element is to be extended as part of a profile
- It can never be used alone, and it cannot be extended by another stereotype.
- Tagged values allow you to include supplementary information about the specifications of a model element that can't otherwise be included in UML. A name or value represents them in brackets.

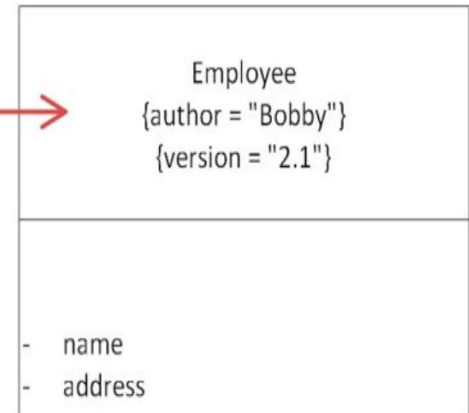
Stereotype



Stereotype

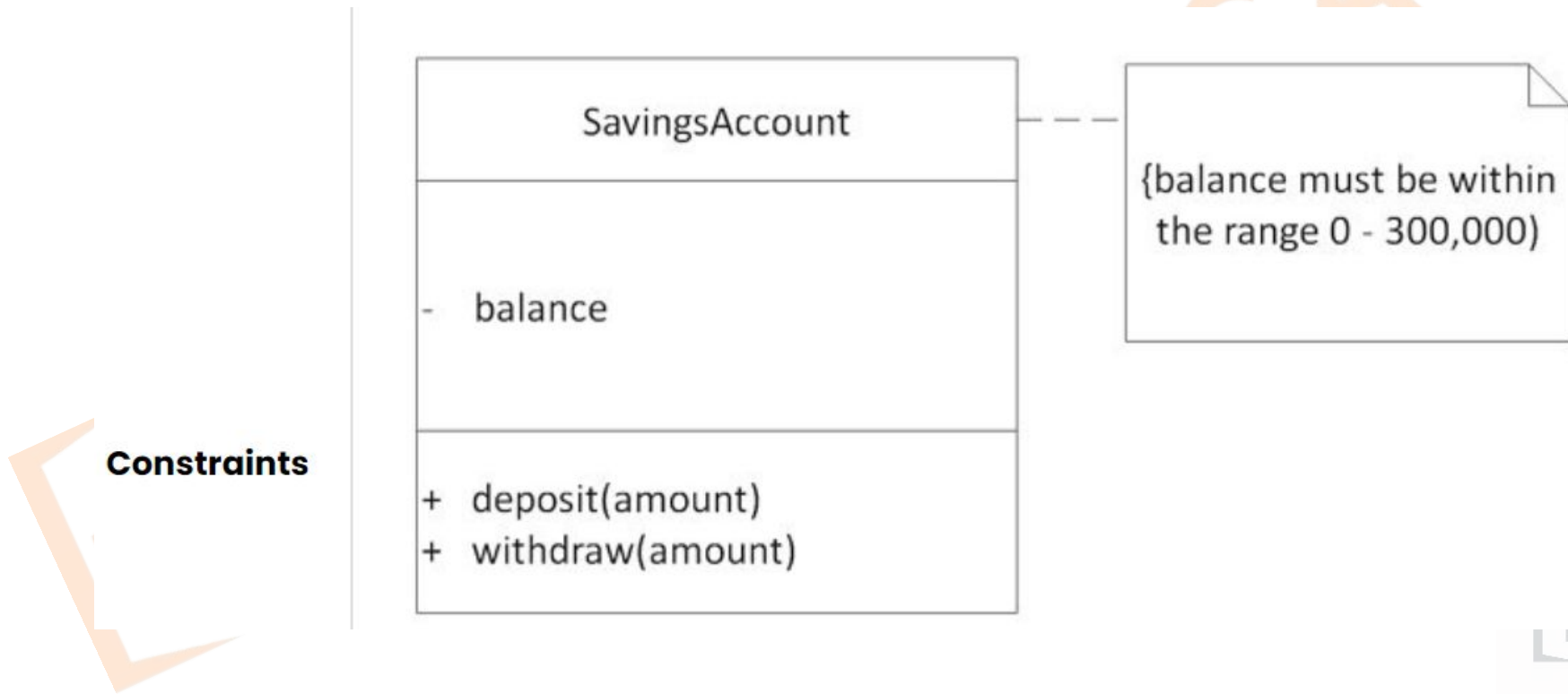


Two tagged
values



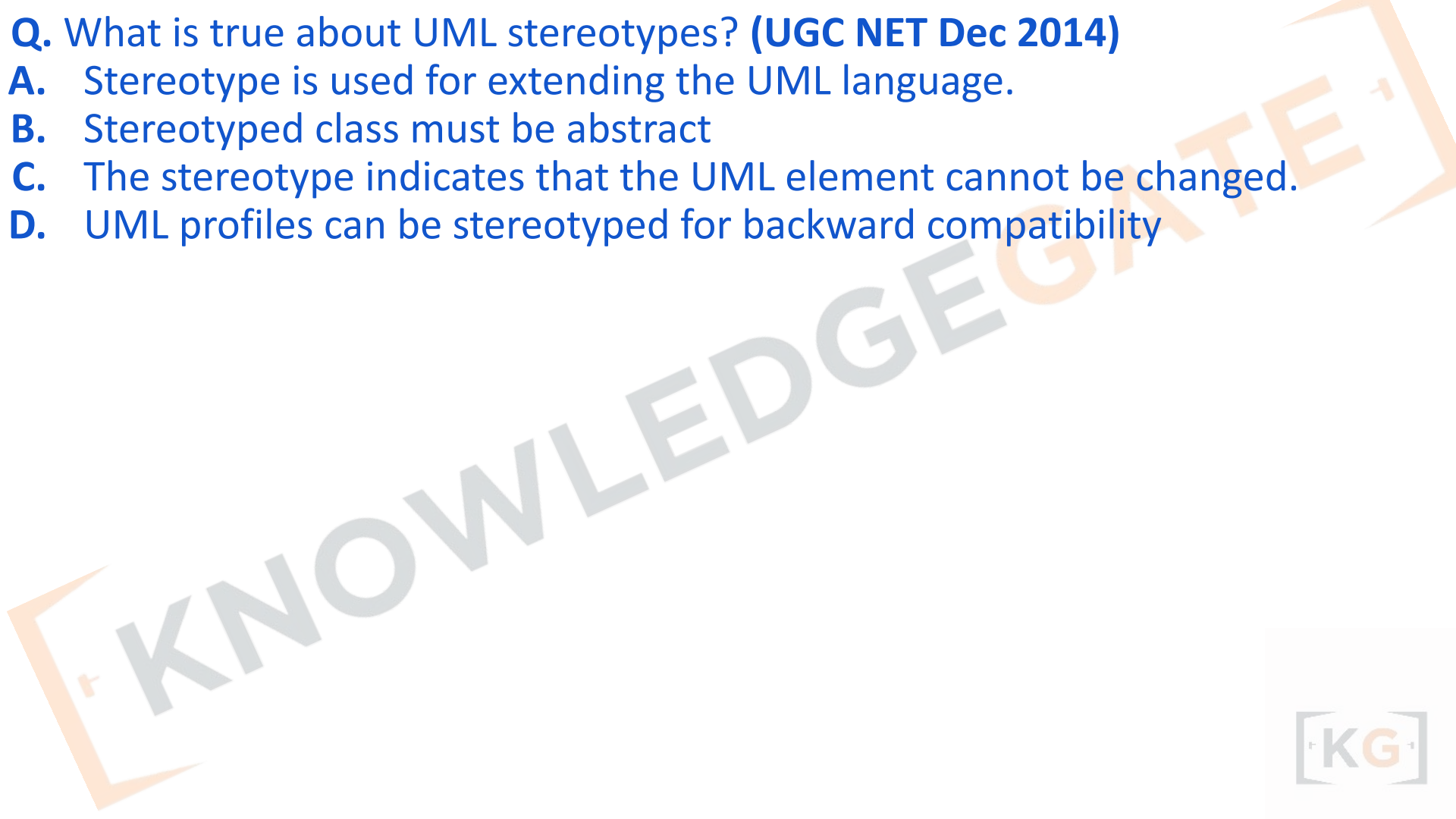
Profile Diagram

- Constraints represent the conditions that must remain constant throughout the entire system or process. They are typically shown via a note.



Q. What is true about UML stereotypes? (UGC NET Dec 2014)

- A. Stereotype is used for extending the UML language.**
- B. Stereotyped class must be abstract**
- C. The stereotype indicates that the UML element cannot be changed.**
- D. UML profiles can be stereotyped for backward compatibility**



Q. What is true about UML stereotypes? (UGC NET Dec 2014)

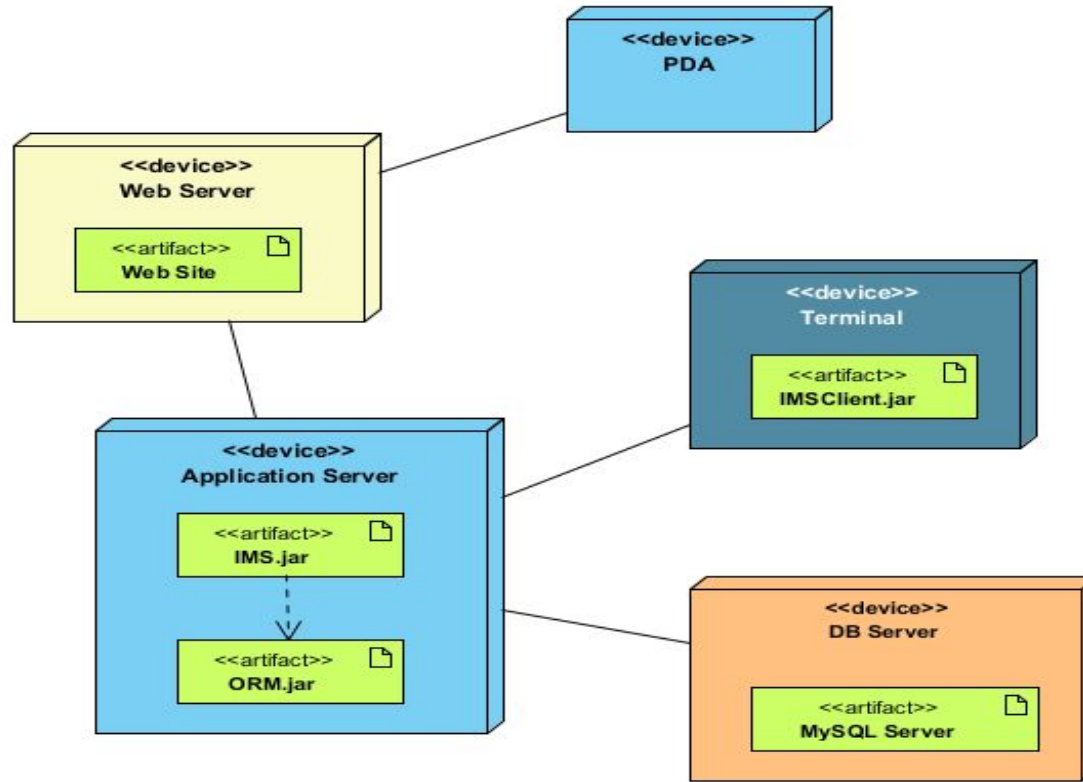
- A. Stereotype is used for extending the UML language.**
- B. Stereotyped class must be abstract**
- C. The stereotype indicates that the UML element cannot be changed.**
- D. UML profiles can be stereotyped for backward compatibility**

Correct: A

- Stereotype, tags and constraints are used for extending the UML.
- They allow designers to extend the vocabulary of UML in order to create new model elements, derived from existing ones.
- a stereotype is rendered as a name enclosed by << >>
- A stereotype cannot be used by itself, but must always be used with one of the metaclasses it extends.
- Stereotype cannot be extended by another stereotype.

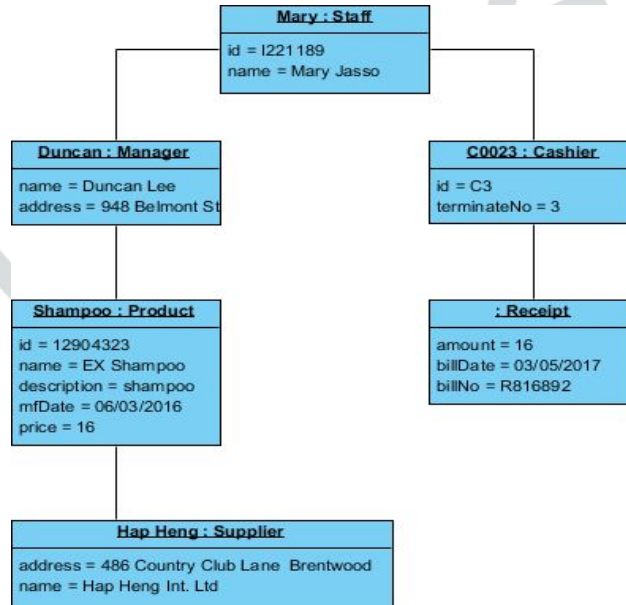
Deployment Diagram

- A deployment diagram shows the hardware of your system and the software in that hardware. Deployment diagrams are useful when your software solution is deployed across multiple machines with each having a unique configuration.



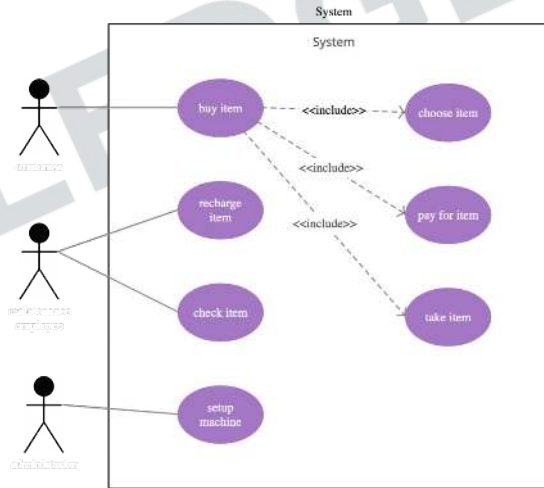
Object Diagram

- Object Diagrams, sometimes referred to as Instance diagrams are very similar to class diagrams. Like class diagrams, they also show the relationship between objects but they use real-world examples.
- They show what a system will look like at a given time. Because there is data available in the objects, they are used to explain complex relationships between objects.



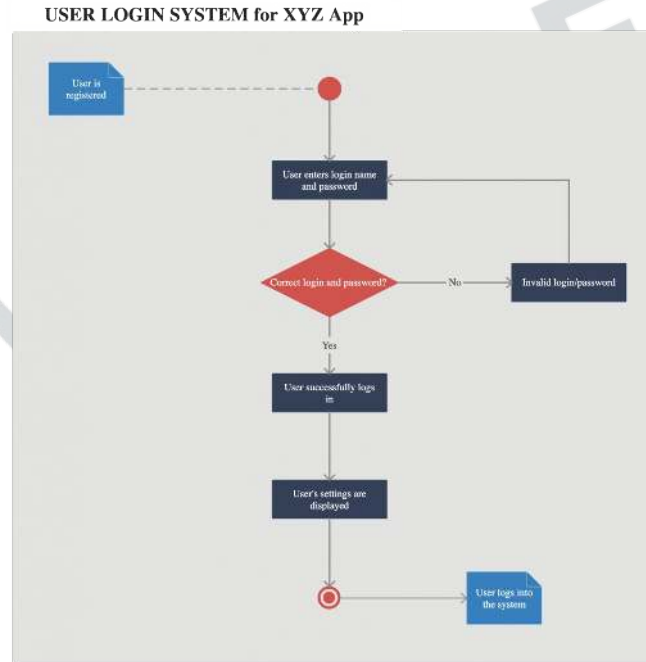
Use Case Diagram

- As the most known diagram type of the behavioral UML types, Use case diagrams give a graphic overview of the actors involved in a system, different functions needed by those actors and how these different functions interact.
- It's a great starting point for any project discussion because you can easily identify the main actors involved and the main processes of the system



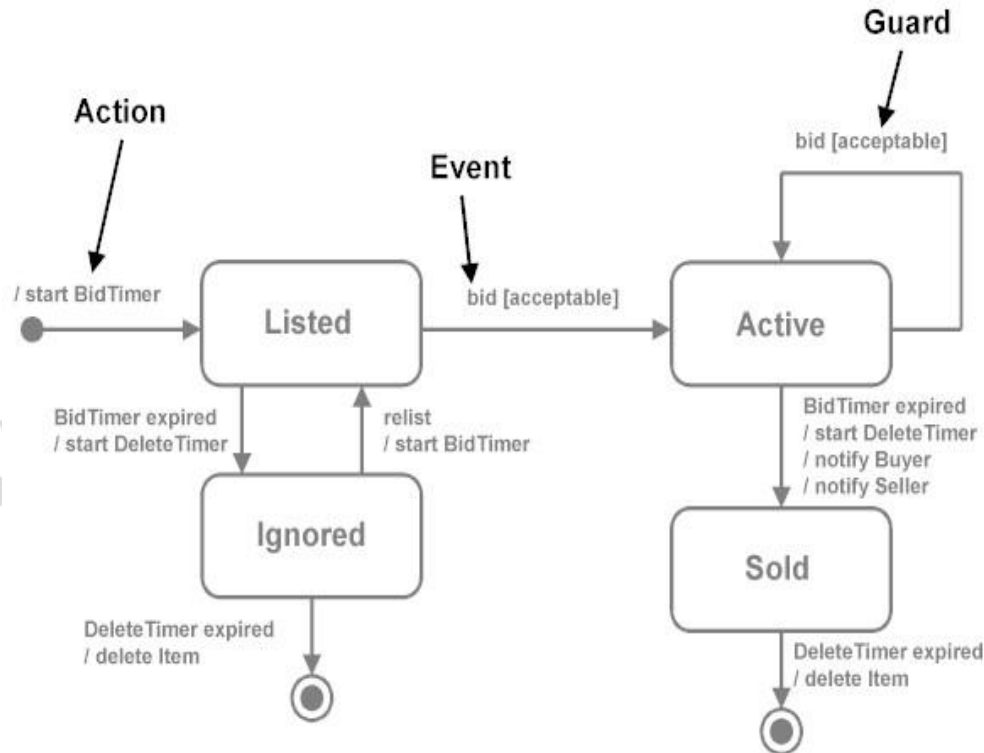
Activity Diagram

- Activity diagrams represent workflows in a graphical way. They can be used to describe the business workflow or the operational workflow of any component in a system. Sometimes activity diagrams are used as an alternative to State machine diagrams.
- An activity represents an operation on some class in the system that results in a change in the state of the system.



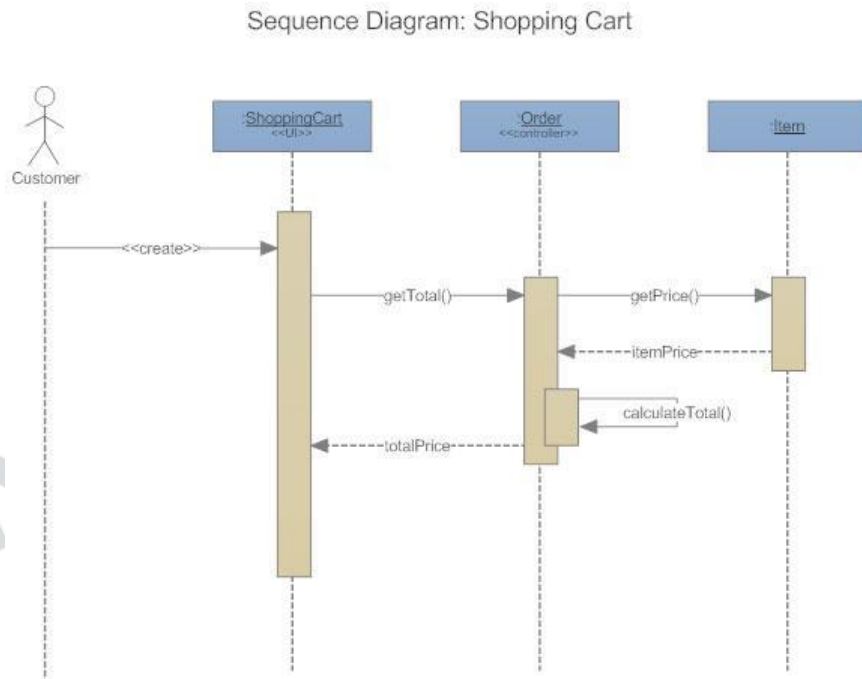
State Machine Diagram

- State machine diagrams are similar to activity diagrams, although notations and usage change a bit.
- They are sometimes known as state diagrams or state chart diagrams as well. These are very useful to describe the behavior of objects that act differently according to the state they are in at the moment.



Sequence Diagram

Sequence diagrams in UML show how objects interact with each other and the order those interactions occur. It's important to note that they show the interactions for a particular scenario. The processes are represented vertically and interactions are shown as arrows.



Q. The sequence diagram given in the Figure 1 for the weather information System takes place when an external system requests the summarized data from the weather station. The increasing order of lifeline for the objects in the system are: **(UGC NET Dec 2019)**

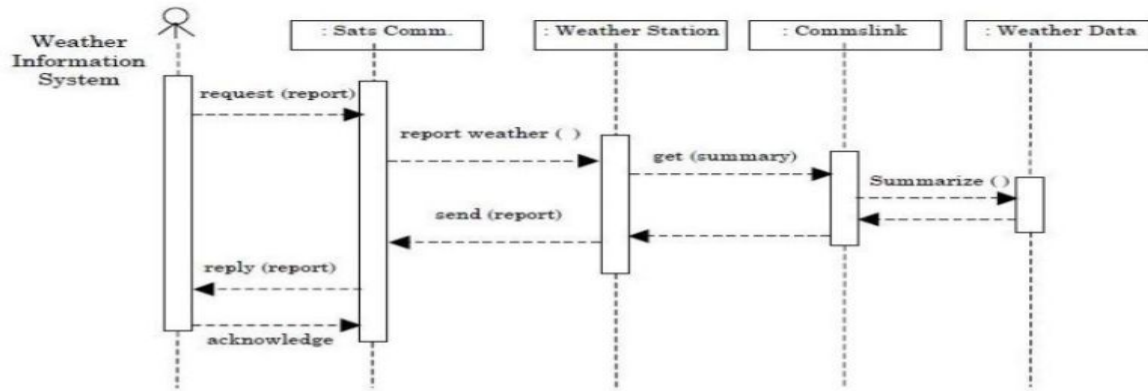


Figure 1

- A. Sat comms -> weather station -> comms link -> weather data
- B. Sat comms -> comms link -> weather station -> weather data
- C. weather data -> comms link -> weather station -> Sat Comms
- D. weather data -> weather station -> comms link -> Sat Comms

Q. In a system for a restaurant, the main scenario for placing order is given below: **(UGC NET Dec 2019)**

a. Customer reads menu

- a. Customer reads menu
- b. Customer places order
- c. Order is sent to kitchen for preparation
- d. Ordered items are served
- e. Customer requests for a bill for the order
- f. Bill is prepared for this order
- g. Customer is given the bill
- h. Customer pays the bill

A sequence diagram for the scenario will have at least how many objects among whom the messages will be exchanged?

A. 3

B. 4

C. 5

D. 6



Q. In a system for a restaurant, the main scenario for placing order is given below: **(UGC NET Dec 2019)**

a. Customer reads menu

a. Customer reads menu

b. Customer places order

c. Order is sent to kitchen for preparation

d. Ordered items are served

e. Customer requests for a bill for the order

f. Bill is prepared for this order

g. Customer is given the bill

h. Customer pays the bill

A sequence diagram for the scenario will have at least how many objects among whom the messages will be exchanged?

A. 3

B. 4

C. 5

D. 6

Correct: C



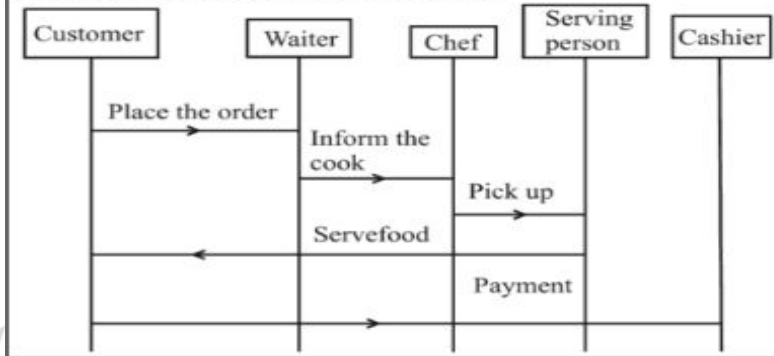
Explanation:

Ans. (c) : There are five objects among whom the message will be exchanged. :

These are :

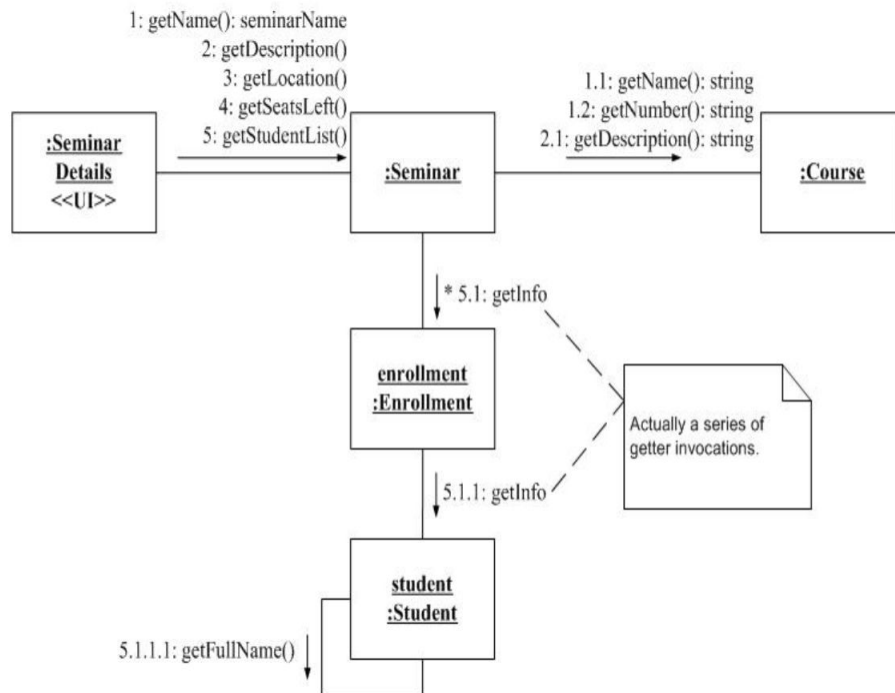
1. Customer
2. Person who takes the order
3. Chef
4. Serving person
5. Cashier

Process with the help of diagram is :



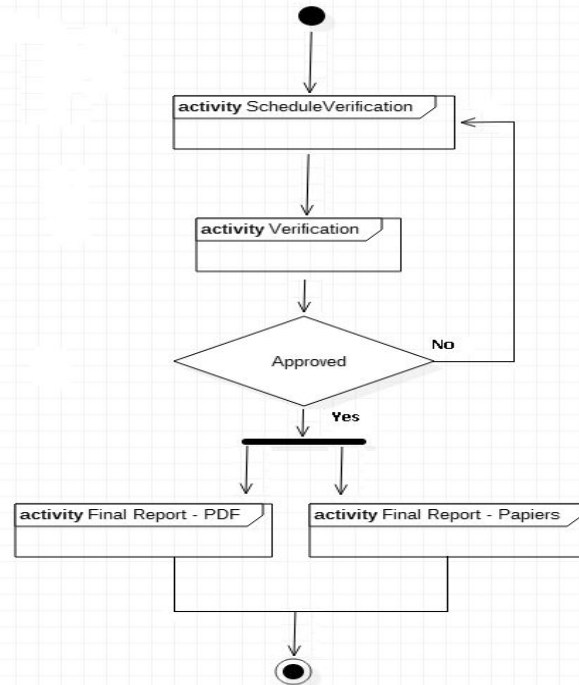
Communication Diagram

- In UML 1 they were called collaboration diagrams. Communication diagrams are similar to sequence diagrams, but the focus is on messages passed between objects. The same information can be represented using a sequence diagram and different objects.



Interaction Overview Diagram

- Interaction overview diagrams are a combination of activity and sequence diagrams.
- They model a sequence of actions and let you deconstruct more complex interactions into manageable occurrences.
- Interaction overview diagrams are very similar to activity diagrams. While activity diagrams show a sequence of processes, Interaction overview diagrams show a sequence of interaction diagrams.

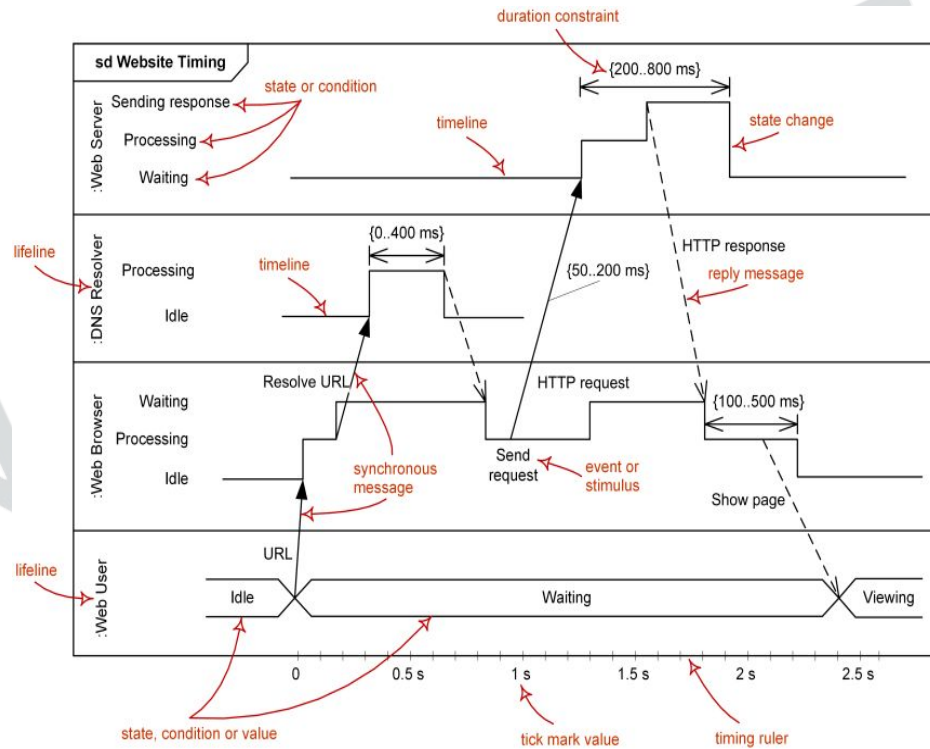


Timing Diagram

Timing diagrams are very similar to sequence diagrams.

They represent the behavior of objects in a given time frame.

If it's only one object, the diagram is straightforward. But, if there is more than one object is involved, a Timing diagram is used to show interactions between objects during that time frame.



Q. Match each UML diagram in List I to its appropriate description in List II (UGC NET Dec 2018)

List I

List II

- | | |
|----------------------|---|
| (a) State Diagram | (i) Describes how the external entities (people, devices) can interact with the System |
| (b) Use Case Diagram | (ii) Used to describe the static or structural view of a system |
| (c) Class Diagram | (iii) Used to show the flow of a business process, the steps of a use-case or the logic of an object behaviour |
| (d) Activity Diagram | (iv) Used to describe the dynamic behaviour of objects and could also be used to describe the entire system behaviour |

- A.** (a)-(i); (b)-(iv); (c)-(ii); (d)-(iii)
B. (a)-(i); (b)-(iv); (c)-(i); (d)-(iii)
C. (a)-(ii); (b)-(iv); (c)-(i); (d)-(iii)
D. (a)-(iv); (b)-(ii); (c)-(ii); (d)-(iii)

Q. Match each UML diagram in List I to its appropriate description in List II (UGC NET Dec 2018)

List I

List II

- | | |
|----------------------|---|
| (a) State Diagram | (i) Describes how the external entities (people, devices) can interact with the System |
| (b) Use Case Diagram | (ii) Used to describe the static or structural view of a system |
| (c) Class Diagram | (iii) Used to show the flow of a business process, the steps of a use-case or the logic of an object behaviour |
| (d) Activity Diagram | (iv) Used to describe the dynamic behaviour of objects and could also be used to describe the entire system behaviour |

A. (a)-(i); (b)-(iv); (c)-(ii); (d)-(iii)

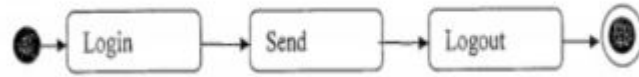
B. (a)-(i); (b)-(iv); (c)-(i); (d)-(iii)

C. (a)-(ii); (b)-(iv); (c)-(i); (d)-(iii)

D. (a)-(iv); (b)-(ii); (c)-(ii); (d)-(iii)

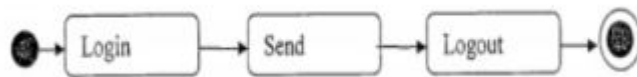
Correct :D

Q. The above figure represents which one of the following UML diagrams for a single send session of an online chat system? **(ISRO - 2011)**



- A.** Package diagram
- B.** Activity diagram
- C.** Class diagram
- D.** Sequence diagram

Q. The above figure represents which one of the following UML diagrams for a single send session of an online chat system? **(ISRO - 2011)**



- A.** Package diagram
- B.** Activity diagram
- C.** Class diagram
- D.** Sequence diagram

Correct : B

Q. In UML diagram of a class (ISRO - 2007)

- A. State of object cannot be represented**
- B. State is irrelevant**
- C. State is represented as an attribute**
- D. State is represented as a result of an operation**

Q. In UML diagram of a class (ISRO - 2007)

- A. State of object cannot be represented**
- B. State is irrelevant**
- C. State is represented as an attribute**
- D. State is represented as a result of an operation**

Correct :C

Q. The diagram that helps in understanding and representing user requirements for a software project using UML (unified Model Language) is (**GATE IT 2004**)

- A.** Entity Relationship Diagram
- B.** Deployment Diagram
- C.** Data Flow Diagram
- D.** Use case Diagram

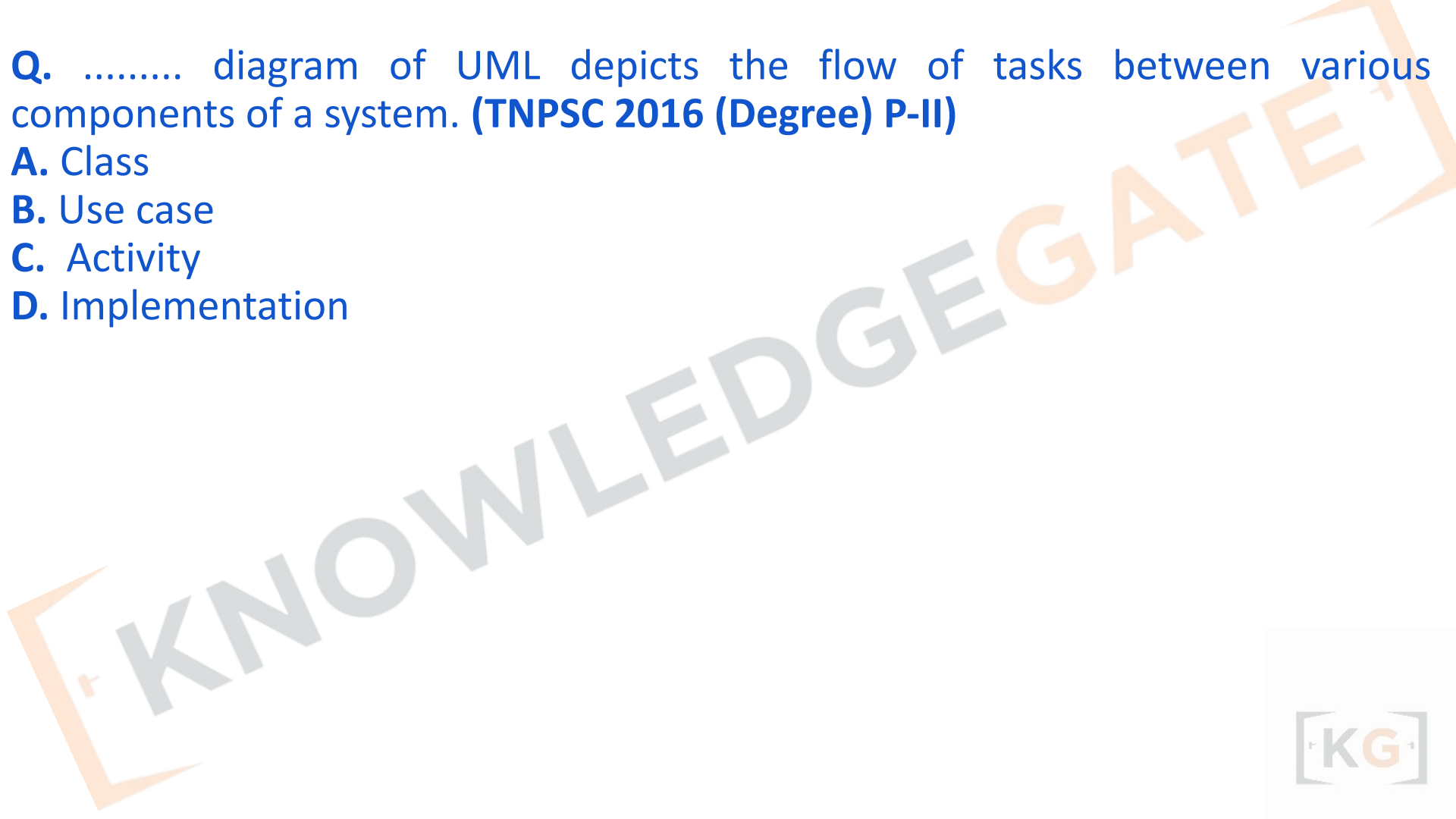
Q. The diagram that helps in understanding and representing user requirements for a software project using UML (unified Model Language) is (**GATE IT 2004**)

- A.** Entity Relationship Diagram
- B.** Deployment Diagram
- C.** Data Flow Diagram
- D.** Use case Diagram

Correct : D

Q. diagram of UML depicts the flow of tasks between various components of a system. **(TNPSC 2016 (Degree) P-II)**

- A. Class
- B. Use case
- C. Activity
- D. Implementation



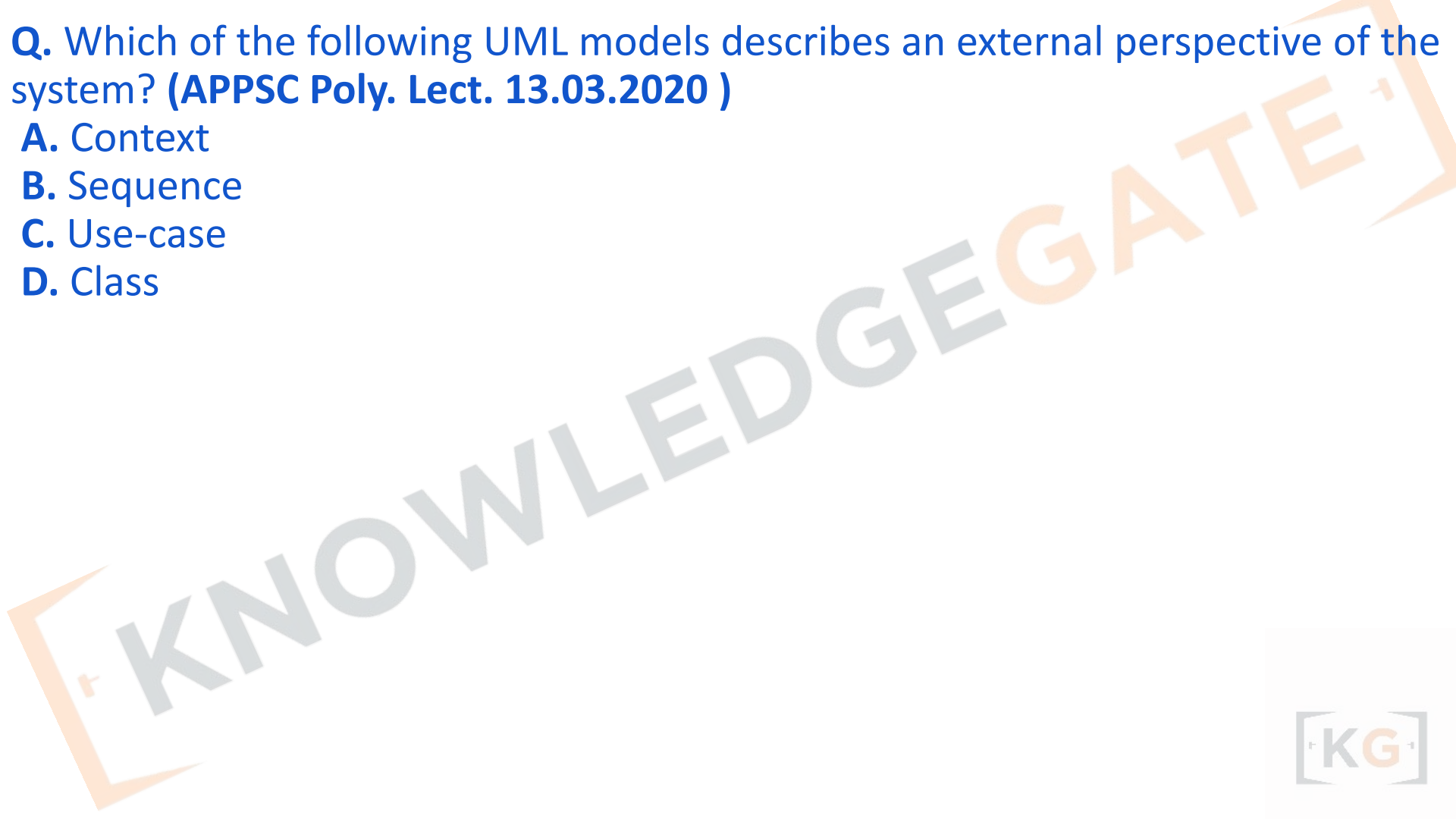
Q. diagram of UML depicts the flow of tasks between various components of a system. (TNPSC 2016 (Degree) P-II)

- A. Class**
- B. Use case**
- C. Activity**
- D. Implementation**

Ans. (c)

Q. Which of the following UML models describes an external perspective of the system? (APPSC Poly. Lect. 13.03.2020)

- A. Context**
- B. Sequence**
- C. Use-case**
- D. Class**



Q. Which of the following UML models describes an external perspective of the system? (APPSC Poly. Lect. 13.03.2020)

- A. Context
- B. Sequence
- C. Use-case
- D. Class

Ans. A : A **Context Model (Diagram)** in UML (or sometimes used alongside UML) defines the **external environment** of a system — showing the **system boundary** and all **external entities** that interact with it.

It helps analysts understand **system scope** and **external interfaces**, hence it represents the **external perspective** of the system.



Q. Which of the following UML 2.0 diagrams capture behavioural aspects of a system?

- A.** Use Case Diagram, Object Diagram, Activity Diagram and State Machine Diagram
- B.** Use Case diagram, Activity Diagram, and State Machine Diagram
- C.** Object Diagram, Communication Diagram, Timing Diagram, and Interaction diagram Computer Science
- D.** Object Diagram, Composite Structure Diagram, Package Diagram, and Deployment Diagram

Q. Which of the following UML 2.0 diagrams capture behavioural aspects of a system?

- A.** Use Case Diagram, Object Diagram, Activity Diagram and State Machine Diagram
- B.** Use Case diagram, Activity Diagram, and State Machine Diagram
- C.** Object Diagram, Communication Diagram, Timing Diagram, and Interaction diagram Computer Science
- D.** Object Diagram, Composite Structure Diagram, Package Diagram, and Deployment Diagram

Answer: (b) Use Case Diagram, Activity Diagram, and State Machine Diagram

Q. The UML supports event-based modeling using _____ diagrams. (GPSC Asstt. Prof. 30.06.2016)

- A. Deployment**
- B. Collaboration**
- C. State chart**
- D. All of the above**

Q. The UML supports event-based modeling using _____ diagrams. (GPSC Asstt. Prof. 30.06.2016)

- A. Deployment**
- B. Collaboration**
- C. State chart**
- D. All of the above**

Ans. (c) :

Unified Modeling Language (UML) diagrams are valuable tools in various phases of the Software Development Life Cycle (SDLC) as they help visualize, document, and communicate different aspects of a software system. Here's a summary of UML diagram usage in different SDLC phases:

1. Requirements Gathering and Analysis:

- a. Use Case Diagrams: Identify and depict the interactions between actors (users or external systems) and the system to understand the functional requirements.

2. System Design:

- a. Class Diagrams: Illustrate the static structure of the system, including classes, attributes, methods, and their relationships.
- b. Object Diagrams: Show specific instances of classes and their relationships, aiding in understanding object structures during design.

3. Architecture Design:

- a. Component Diagrams: Describe the high-level components/modules of the system and their dependencies, helping to design the system's architecture.
- b. Deployment Diagrams: Depict the physical deployment of software components on hardware nodes or devices.

- 4. Detailed Design:**
- a. Sequence Diagrams: Visualize the interactions between different objects and the sequence of messages exchanged, describing the dynamic behavior of the system.
 - b. Activity Diagrams: Represent workflows or processes to demonstrate the steps and decisions in complex scenarios.
- 5. Implementation and Coding:**
- a. Class Diagrams: Continue to be useful during the coding phase to define class structures and maintain code organization.
 - b. Package Diagrams: Organize classes into packages or modules, providing a high-level view of the codebase's organization.
- 6. Testing and Quality Assurance:**
- a. State Machine Diagrams: Model the behavior of an object or system in response to different events, useful for testing state transitions.
 - b. Communication Diagrams: Similar to sequence diagrams, they illustrate object interactions, helping in test case generation.

7. **Maintenance and Documentation:**

- a. Use Case Diagrams: Act as a reference for understanding system functionality when maintaining and updating the software.
- b. Class Diagrams: Support code maintenance and refactoring efforts, providing an overview of class relationships.

Throughout the SDLC, UML diagrams serve as a common language that aids communication between stakeholders, developers, testers, and other team members. They help improve the understanding of the software system, leading to better design decisions and a more efficient development process.

Q. In software development, UML diagrams are used during..... (APPSC Lect. 2017 (Degree College) (CS))

- A. requirements analysis**
- B. system/module design**
- C. system integration**
- D. All the given options**

Q. In software development, UML diagrams are used during..... (APPSC Lect. 2017 (Degree College) (CS))

- A. requirements analysis**
- B. system/module design**
- C. system integration**
- D. All the given options**

Ans. (d)

Chapter - 3

Object Oriented Analysis, Object oriented design, Object design. Structured analysis and structured design (SA/SD), Jackson Structured Development (JSD)

Object Oriented Analysis:

Object oriented design, Object design, Combining three models, Designing algorithms, design optimization, Implementation of control, Adjustment of inheritance, Object representation, Physical packaging, Documenting design considerations. Structured analysis and structured design (SA/SD), Jackson Structured Development (JSD). Mapping object oriented concepts using non-object oriented language, Translating classes into data structures, Passing arguments to methods, Implementing inheritance, associations Encapsulation.

Object oriented programming style: reusability, extensibility, robustness, programming in the large. Procedural v/s OOP, Object oriented language features. Abstraction and Encapsulation.

Object Oriented Analysis:

1. In the system analysis or object-oriented analysis phase of software development, the system requirements are determined, the classes are identified and the relationships among classes are Identified.
2. The three analysis techniques that are used in conjunction with each other for object-oriented analysis are object modeling, dynamic modeling, and functional modeling.

Object Oriented Analysis:

Understand **problem domain** in terms of objects, classes, and relationships.

- Identify system requirements.
- Define classes, attributes, operations, and associations.
- Model interactions among objects.

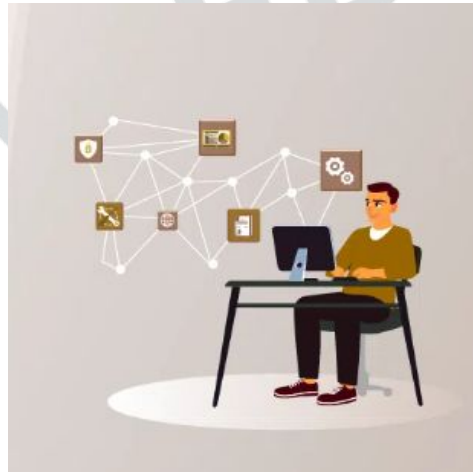
Three core models (Rumbaugh's OMT):

- **Object Model** → Static structure (classes, attributes, relationships)
- **Dynamic Model** → Object interactions and state changes
- **Functional Model** → Data transformation and system functionality



Object Oriented Analysis:

1. In the system analysis or object-oriented analysis phase of software development, the system requirements are determined, the classes are identified and the relationships among classes are identified.
2. The three analysis techniques that are used in conjunction with each other for object-oriented analysis are object modeling, dynamic modeling, and functional modeling.

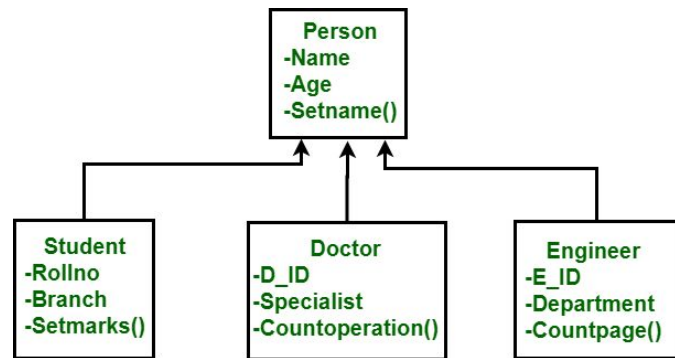


Object Modeling

Object modeling develops the static structure of the software system in terms of objects. It identifies the objects, the classes into which the objects can be grouped into and the relationships between the objects. It also identifies the main attributes and operations that characterize each class.

The process of object modeling can be visualized in the following steps –

- Identify objects and group into classes
- Identify the relationships among classes
- Create user object model diagram
- Define user object attributes
- Define the operations that should be performed on the classes
- Review glossary



Example: Library Management System

Class	Attributes	Methods	Relationships
Book	bookID, title, author, price, status	issue(), return(), reserve()	Associated with <i>Member</i>
Member	memberID, name, address, phone	borrowBook(), returnBook()	Associated with <i>Book</i>
Librarian	empID, name, shift	issueBook(), collectFine()	Supervises <i>Book</i> and <i>Member</i>
Library	libraryName, address	addBook(), removeBook()	Contains <i>Book</i> objects

Relationships:

- **Association:** Member ↔ Book (a member borrows a book)
- **Aggregation:** Library → Book (Library *has many* books)
- **Inheritance:** Staff → Librarian (Librarian *is a kind of* Staff)

Interpretation:

This model defines the **classes and structure** of the system — it's *static* because it does not change with time or events.



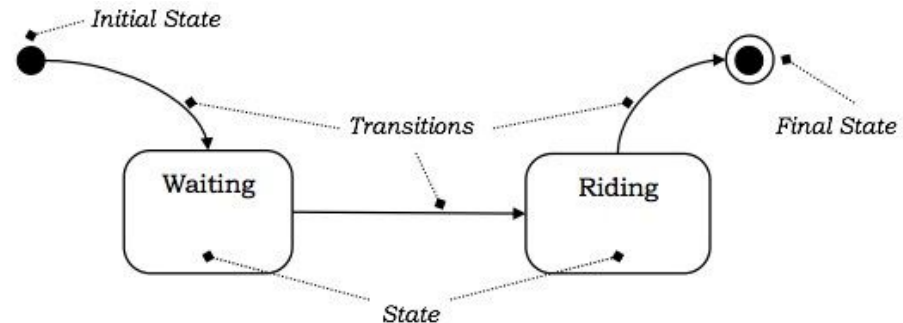
Dynamic Modelling

After the static behavior of the system is analyzed, its behavior with respect to time and external changes needs to be examined. This is the purpose of dynamic modeling.

Dynamic Modelling can be defined as “a way of describing how an individual object responds to events, either internal events triggered by other objects, or external events triggered by the outside world”.

The process of dynamic modeling can be visualized in the following steps –

- Identify states of each object
- Identify events and analyze the applicability of actions
- Construct dynamic model diagram, comprising of state transition diagrams
- Express each state in terms of object attributes
- Validate the state–transition diagrams drawn



Example: “Book Borrowing” Process in Library System : State Diagram (for Book object):

States:

1. **Available**
2. **Reserved**
3. **Issued**
4. **Returned**

Events / Transitions:

- *reserve()* → Available → Reserved
- *issue()* → Reserved → Issued
- *return()* → Issued → Returned → Available

Explanation:

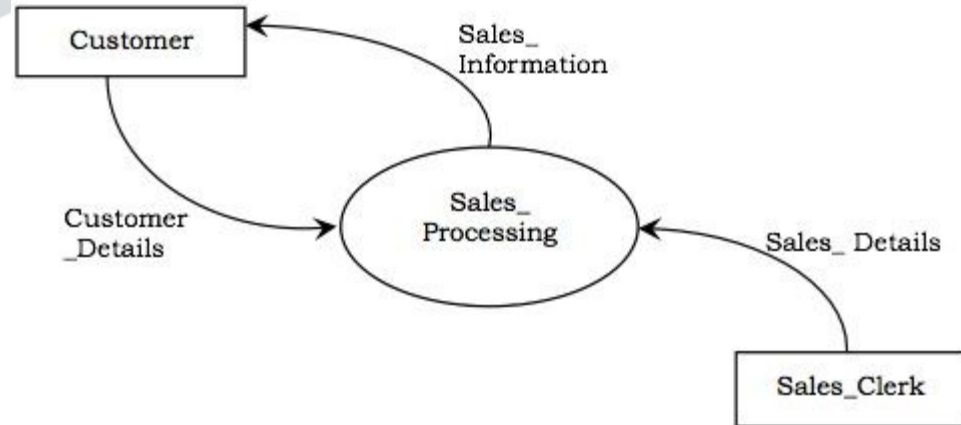
When a book is **issued**, its state changes from *Available* to *Issued*. When the book is **returned**, it transitions back to *Available*.

Functional Modeling

Functional Modeling is the final component of object-oriented analysis. The functional model shows the processes that are performed within an object and how the data changes as it moves between methods. It specifies the meaning of the operations of object modeling and the actions of dynamic modeling. The functional model corresponds to the data flow diagram of traditional structured analysis.

The process of functional modeling can be visualized in the following steps –

- Identify all the inputs and outputs
- Construct data flow diagrams showing functional dependencies
- State the purpose of each function
- Identify constraints
- Specify optimization criteria



Data Flow:

Member Details → IssueBook Process → Book Record → Transaction Record → Confirmation

Functions Involved:

- validateMember() → verifies the member's validity
- checkBookAvailability() → ensures the book isn't already issued
- updateBookStatus() → marks the book as issued
- recordTransaction() → logs the issue details

Q. According to Rumbaugh's Object Modeling Technique (OMT), object-oriented analysis consists of which three core models?

- A. Object Model, Dynamic Model, Functional Model
- B. Object Model, Data Model, Flow Model
- C. Structural Model, Process Model, Functional Model
- D. Entity Model, State Model, Control Model

Q. According to Rumbaugh's Object Modeling Technique (OMT), object-oriented analysis consists of which three core models?

- A. Object Model, Dynamic Model, Functional Model
- B. Object Model, Data Model, Flow Model
- C. Structural Model, Process Model, Functional Model
- D. Entity Model, State Model, Control Model

Answer: (a)

Explanation:

Rumbaugh's OMT defines **three complementary models** for analyzing and designing systems:

- **Object Model** → structure of the system (classes, relationships)
- **Dynamic Model** → behavior over time (state transitions)
- **Functional Model** → data flow and transformations (functions/processes).

Q. In Rumbaugh's OMT, which model describes the static structure of objects, classes, attributes, and relationships?

- A.** Dynamic Model
- B.** Object Model
- C.** Functional Model
- D.** State Model

Q. In Rumbaugh's OMT, which model describes the static structure of objects, classes, attributes, and relationships?

- A.** Dynamic Model
- B.** Object Model
- C.** Functional Model
- D.** State Model

Answer: (b)

Explanation:

The **Object Model** represents the **static structure** of the system — classes, attributes, inheritance, and associations.

It is often represented using **class diagrams**.

Q. Which OMT model is primarily represented by *state transition diagrams*?

- A.** Object Model
- B.** Functional Model
- C.** Dynamic Model
- D.** Process Model

Q. Which OMT model is primarily represented by *state transition diagrams*?

- A. Object Model
- B. Functional Model
- C. Dynamic Model
- D. Process Model

Answer: (c)

Explanation:

The **Dynamic Model** describes how objects change states and interact over time in response to events.

It is modeled using **state transition diagrams** or **statecharts**.

Q. In Rumbaugh's OMT, which model focuses on what data flows between functions and how input is transformed into output?

- A.** Object Model
- B.** Functional Model
- C.** Dynamic Model
- D.** Interaction Model

Q. In Rumbaugh's OMT, which model focuses on what data flows between functions and how input is transformed into output?

- A.** Object Model
- B.** Functional Model
- C.** Dynamic Model
- D.** Interaction Model

Answer: (b)

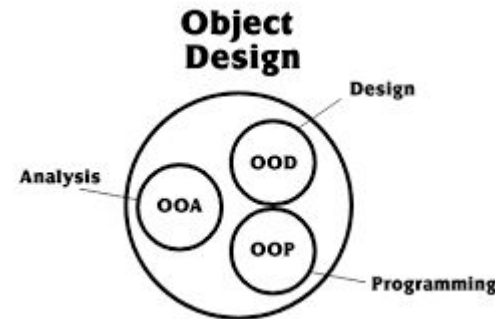
Explanation:

The **Functional Model** specifies **data transformations and flow** between processes, often represented using **Data Flow Diagrams (DFDs)**.

It answers "*What functions does the system perform?*"

□ Object-Oriented Design (OOD):

1. An analysis model created using object-oriented analysis is transformed by object-oriented design into a design model that works as a plan for software creation.
2. OOD results in a design having several different levels of modularity i.e., The major system components are partitioned into subsystems (a system-level “modular”), and data manipulation operations are encapsulated into objects (a modular form that is the building block of an OO system.).
3. In addition, OOD must specify some data organization of attributes and a procedural description of each operation.



Object Design

After the hierarchy of subsystems has been developed, the objects in the system are identified and their details are designed. Here, the designer details out the strategy chosen during the system design. The emphasis shifts from application domain concepts toward computer concepts. The objects identified during analysis are etched out for implementation with an aim to minimize execution time, memory consumption, and overall cost.

Object design includes the following phases –

- Object identification
- Object representation, i.e., construction of design models
- Classification of operations
- Algorithm design
- Design of relationships
- Implementation of control for external interactions
- Package classes and associations into modules



Object Identification

The first step of object design is object identification. The objects identified in the object oriented analysis phases are grouped into classes and refined so that they are suitable for actual implementation.

The functions of this stage are –

- Identifying and refining the classes in each subsystem or package
- Defining the links and associations between the classes
- Designing the hierarchical associations among the classes, i.e., the generalization/specialization and inheritances
- Designing aggregations

Object Representation

Once the classes are identified, they need to be represented using object modeling techniques. This stage essentially constructs UML diagrams.

There are two types of design models that need to be produced –

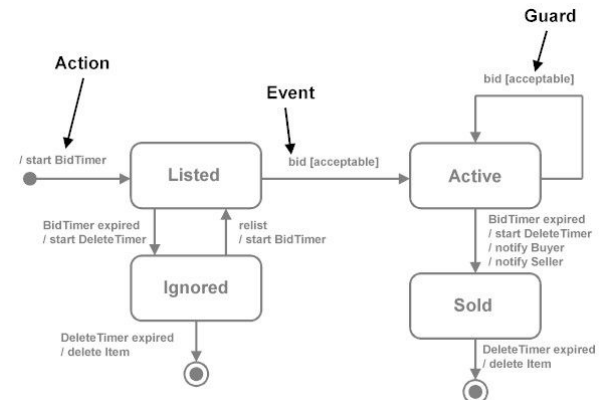
- **Static Models** – To describe the static structure of a system using class **diagrams** and **object diagrams**.
- **Dynamic Models** – To describe the dynamic structure of a system and show the interaction between classes using interaction **diagrams** and **state-chart diagrams**.

Classification of Operations

In this step, the operation to be performed on objects are defined by combining the three models developed in the OOA phase, namely, object model, dynamic model, and functional model. An operation specifies what is to be done and not how it should be done.

The following tasks are performed regarding operations –

- The state transition diagram of each object in the system is developed.
- Operations are defined for the events received by the objects.
- Cases in which one event triggers other events in the same or different objects are identified.
- The sub-operations within the actions are identified.
- The main actions are expanded to data flow diagrams.



Algorithm Design

The operations in the objects are defined using algorithms. An algorithm is a stepwise procedure that solves the problem laid down in an operation. There may be more than one algorithm corresponding to a given operation. Once the alternative algorithms are identified, the optimal algorithm is selected for the given problem domain.

The metrics for choosing the optimal algorithm are –

- **Computational Complexity** – Complexity determines the efficiency of an algorithm in terms of computation time and memory requirements.
- **Flexibility** – Flexibility determines whether the chosen algorithm can be implemented suitably, without loss of appropriateness in various environments.
- **Understandability** – This determines whether the chosen algorithm is easy to understand and implement.

Design of Relationships

The strategy to implement the relationships needs to be chalked out during the object design phase. **The main relationships that are addressed are associations, aggregations, and inheritances.**

The designer should do the following regarding associations –

- Identify whether an association is unidirectional or bidirectional.
- Analyze the path of associations and update them if necessary.
- Implement the associations as a distinct object, in case of many-to-many relationships; or as a link to other objects in case of one-to-one or one-to-many relationships. Regarding inheritances, the designer should do the following –
 - Adjust the classes and their associations.
 - Identify abstract classes.
 - Make provisions so that behaviors are shared when needed.

Implementation of Control

The object designer may incorporate refinements in the strategy of the state-chart model. In system design, a basic strategy for realizing the dynamic model is made. During object design, this strategy is aptly embellished for appropriate implementation.

The approaches for implementation of the dynamic model are –

- **Represent State as a Location within a Program** – This is the traditional procedure-driven approach whereby the location of control defines the program state. A finite state machine can be implemented as a program. A transition forms an input statement, the main control path forms the sequence of instructions, the branches form the conditions, and the backward paths form the loops or iterations.
- **State Machine Engine** – This approach directly represents a state machine through a state machine engine class. This class executes the state machine through a set of transitions and actions provided by the application.
- **Control as Concurrent Tasks** – In this approach, an object is implemented as a task in the programming language or the operating system. Here, an event is implemented as an inter-task call. It preserves inherent concurrency of real objects.

Packaging Classes

In any large project, meticulous partitioning of an implementation into modules or packages is important. During object design, classes and objects are grouped into packages to enable multiple groups to work cooperatively on a project.

The different aspects of packaging are –

- **Hiding Internal Information from Outside View** – It allows a class to be viewed as a “black box” and permits class implementation to be changed without requiring any clients of the class to modify code.
- **Coherence of Elements** – An element, such as a class, an operation, or a module, is coherent if it is organized on a consistent plan and all its parts are intrinsically related so that they serve a common goal.
- **Construction of Physical Modules** – The following guidelines help while constructing physical modules –
 - o Classes in a module should represent similar things or components in the same composite object.
 - o Closely connected classes should be in the same module.
 - o Unconnected or weakly connected classes should be placed in separate modules.
 - o Modules should have good cohesion, i.e., high cooperation among its components.
 - o A module should have low coupling with other modules, i.e., interaction or interdependence between modules should be minimum.

Design Optimization

The analysis model captures the logical information about the system, while the design model adds details to support efficient information access. Before a design is implemented, it should be optimized so as to make the implementation more efficient. The aim of optimization is to minimize the cost in terms of time, space, and other metrics. However, design optimization should not be excess, as ease of implementation, maintainability, and extensibility are also important concerns. It is often seen that a perfectly optimized design is more efficient but less readable and reusable. So the designer must strike a balance between the Two.

The various things that may be done for design optimization are –

- Add redundant associations
- Omit non-usable associations
- Optimization of algorithms
- Save derived attributes to avoid re-computation of complex expressions

Design Documentation

Documentation is an essential part of any software development process that records the procedure of making the software. The design decisions need to be documented for any non-trivial software system for transmitting the design to others.

Usage Areas

Though a secondary product, a good documentation is indispensable, particularly in the following areas

–

- In designing software that is being developed by a number of developers
- In iterative software development strategies
- In developing subsequent versions of a software project
- For evaluating a software
- For finding conditions and areas of testing
- For maintenance of the software.

Contents

A beneficial documentation should essentially include the following contents –

- **High-level system architecture** – Process diagrams and module diagrams
- **Key abstractions and mechanisms** – Class diagrams and object diagrams.
- **Scenarios that illustrate the behavior of the main aspects** – Behavioural diagrams

Features

The features of a good documentation are –

- Concise and at the same time, unambiguous, consistent, and complete
- Traceable to the system's requirement specifications
- Well-structured
- Diagrammatic instead of descriptive

Q. Object-Oriented Design (OOD) transforms:

- A.** A procedural model into a logical model
- B.** An object-oriented analysis model into an implementation model
- C.** A relational model into an object model
- D.** A database schema into a class structure

Q. Object-Oriented Design (OOD) transforms:

- A.** A procedural model into a logical model
- B.** An object-oriented analysis model into an implementation model
- C.** A relational model into an object model
- D.** A database schema into a class structure

Answer: (b)

Explanation:

Object-Oriented Design (OOD) is the **process of refining the object-oriented analysis (OOA) model** into a form suitable for implementation.

It focuses on defining **classes, interfaces, inheritance, and collaborations** to meet system requirements.

Q. In Object-Oriented Design, the term "object" represents:

- A.** A function that manipulates data
- B.** A real-world entity encapsulating data and behavior
- C.** A logical process flow
- D.** A data structure without behavior

Q. In Object-Oriented Design, the term "object" represents:

- A. A function that manipulates data
- B. A real-world entity encapsulating data and behavior
- C. A logical process flow
- D. A data structure without behavior

Answer: (b)

Explanation:

An **object** in OOD represents a **real-world entity** that encapsulates **data (attributes)** and **behavior (methods)** in a single unit, ensuring abstraction and modularity.

Q. What is the primary focus of Object Design in Rumbaugh's OMT?

- A.** Identifying real-world objects and relationships
- B.** Designing the logical data flow of the system
- C.** Refining analysis models to include data structures, algorithms, and representations for implementation
- D.** Describing system use cases

Q. What is the primary focus of Object Design in Rumbaugh's OMT?

- A.** Identifying real-world objects and relationships
- B.** Designing the logical data flow of the system
- C.** Refining analysis models to include data structures, algorithms, and representations for implementation
- D.** Describing system use cases

Answer: (c)

Explanation:

Object Design refines the analysis model into a **ready-to-code design** by specifying internal data structures, algorithmic details, inheritance adjustments, and object representations.

Q. In Object Design, what does “Design Optimization” aim to achieve?

- A.** Eliminate classes with inheritance
- B.** Improve efficiency by restructuring the class and object model
- C.** Remove redundant data attributes only
- D.** Simplify DFD representations

Q. In Object Design, what does “Design Optimization” aim to achieve?

- A.** Eliminate classes with inheritance
- B.** Improve efficiency by restructuring the class and object model
- C.** Remove redundant data attributes only
- D.** Simplify DFD representations

Answer: (b)

Explanation:

Design Optimization involves refining object and class structures to improve **runtime performance**, **memory use**, and **code efficiency** without changing system behavior.

Q. In Object-Oriented Design, physical packaging of classes means:

- A.** Storing objects in a database
- B.** Grouping related classes into modules or files for implementation
- C.** Integrating multiple applications
- D.** Mapping classes directly to network layers

Q. In Object-Oriented Design, physical packaging of classes means:

- A.** Storing objects in a database
- B.** Grouping related classes into modules or files for implementation
- C.** Integrating multiple applications
- D.** Mapping classes directly to network layers

Answer: (b)

Explanation:

Physical packaging involves organizing classes into **packages or files** (e.g., Java packages, C++ header and source files) for **modularity and easier maintenance** during implementation.

Structured Analysis vs. Object Oriented Analysis

The Structured Analysis/Structured Design (SASD) approach is the traditional approach of software development based upon the waterfall model. The phases of development of a system using SASD are –

- Feasibility Study
- Requirement Analysis and Specification
- System Design
- Implementation
- Post-implementation Review

Advantages/Disadvantages of Object Oriented Analysis

Advantages	Disadvantages
Focuses on data rather than the procedures as in Structured Analysis.	Functionality is restricted within objects. This may pose a problem for systems which are intrinsically procedural or computational in nature.
The principles of encapsulation and data hiding help the developer to develop systems that cannot be tampered by other parts of the system.	It cannot identify which objects would generate an optimal system design.
The principles of encapsulation and data hiding help the developer to develop systems that cannot be tampered by other parts of the system.	The object-oriented models do not easily show the communications between the objects in the system.
It allows effective management of software complexity by the virtue of modularity.	All the interfaces between the objects cannot be represented in a single diagram.

Advantages/Disadvantages of Structured Analysis

Advantages	Disadvantage
As it follows a top-down approach in contrast to bottom-up approach of object-oriented analysis, it can be more easily comprehended than OOA.	In traditional structured analysis models, one phase should be completed before the next phase. This poses a problem in design, particularly if errors crop up or requirements change.
It is based upon functionality. The overall purpose is identified and then functional decomposition is done for developing the software. The emphasis not only gives a better understanding of the system but also generates more complete systems.	The initial cost of constructing the system is high, since the whole system needs to be designed at once leaving very little option to add functionality later.
The specifications in it are written in simple English language, and hence can be more easily analyzed by non-technical personnel.	It does not support reusability of code. So, the time and cost of development is inherently high.

Structured Analysis and Structured Design (SA/SD)

Structured Analysis and Structured Design (SA/SD) is a diagrammatic notation that is designed to help people understand the system. The basic goal of SA/SD is to improve quality and reduce the risk of system failure. It establishes concrete management specifications and documentation. It focuses on the solidity, pliability, and maintainability of the system. Basically, the approach of SA/SD is based on the Data Flow Diagram. It is easy to understand SA/SD but it focuses on well-defined system boundaries whereas the JSD approach is too complex and does not have any graphical representation.

SA/SD is combined known as SAD and it mainly focuses on the following 3 points:

1. System
2. Process
3. Technology

SA/SD involves 2 phases:

1. **Analysis Phase:** It uses Data Flow Diagram, Data Dictionary, State Transition diagram and ER diagram.
2. **Design Phase:** It uses Structure Chart and Pseudo Code.

1. **Analysis Phase:** Analysis Phase involves data flow diagram, data dictionary, state transition diagram, and entity-relationship diagram.

Data Flow Diagram:

In the data flow diagram, the model describes how the data flows through the system. We can incorporate the Boolean operators and & or link data flow when more than one data flow may be input or output from a process.

For example, if we have to choose between two paths of a process we can add an operator or and if two data flows are necessary for a process we can add an operator. The input of the process “check-order” needs the credit information and order information whereas the output of the process would be a cash-order or a good-credit-order.

Data Dictionary: The content that is not described in the DFD is described in the data dictionary. It defines the data store and relevant meaning. A physical data dictionary for data elements that flow between processes, between entities, and between processes and entities may be included. This would also include descriptions of data elements that flow external to the data stores. A logical data dictionary may also be included for each such data element. All system names, whether they are names of entities, types, relations, attributes, or services, should be entered in the dictionary.

State Transition Diagram: State transition diagram is similar to the dynamic model. It specifies how much time the function will take to execute and data access triggered by events. It also describes all of the states that an object can have, the events under which an object changes state, the conditions that must be fulfilled before the transition will occur and the activities were undertaken during the life of an object.

ER Diagram: ER diagram specifies the relationship between data stores. It is basically used in database design. It basically describes the relationship between different entities.

2. Design Phase:

Design Phase involves structure chart and pseudocode.

1. Structure Chart:

It is created by the data flow diagram. Structure Chart specifies how DFS's processes are grouped into tasks and allocate to the CPU. The structured chart does not show the working and internal structure of the processes or modules and does not show the relationship between data or data-flows. Similar to other SASD tools, it is time and cost-independent and there is no error-checking technique associated with this tool. The modules of a structured chart are arranged arbitrarily and any process from a DFD can be chosen as the central transform depending on the analysts own perception. The structured chart is difficult to amend, verify, maintain, and check for completeness and consistency.

Pseudo Code:

It is the actual implementation of the system. It is an informal way of programming that doesn't require any specific programming language or technology.

Difference between Structured and Object-Oriented Analysis

Analysis simply means to study or examine the structure of something, elements, and system requirements in detail, and methodical way. Structured analysis and Object-oriented analysis both are important for software development and are analysis techniques used in software engineering. But both are different from each other.

1. Structured Analysis :

Structured analysis is a method of development that allows and gives permission to the analyst to understand and know about the system and all of its activities in a logical way. It is simply a graphic that is used to specify the presentation of the application.

2. Object-Oriented Analysis :

Object-Oriented Analysis (OOA) is a technical approach generally used for analyzing and application designing, system designing, or even business designing just by applying object oriented programming even with the use of visual modeling throughout the process of development to just simply guide the stakeholder communication and quality of the product. It is actually a process of discovery where a team of developers understands and models all the requirements of the system.

Difference Between Structured and Object-oriented analysis :

Structured Analysis	Object-Oriented Analysis
The main focus is on the process and procedures of the system.	The main focus is on data structure and real world objects that are important.
It uses System Development Life Cycle (SDLC) methodology for different purposes like planning, analyzing, designing, implementing, and supporting an information system.	It uses Incremental or Iterative methodology to refine and extend our design.
It is suitable for well-defined projects with stable user requirements.	It is suitable for large projects with changing user requirements.
This technique is old and is not usually preferred.	This technique is new and is mostly preferred.

Difference Between Structured and Object-oriented analysis :

Structured Analysis	Object-Oriented Analysis
Risk while using this analysis technique is high and reusability is also low.	Risk while using this analysis technique is low and reusability is also high.
Structuring requirements include DFDs (Data Flow Diagram), Structured Analysis, ER (Entity Relationship) diagram, CFD (Control Flow Diagram), Data Dictionary, Decision table/tree, and the State transition diagram.	Requirement engineering includes the Use case model (find Use cases, Flow of events, Activity Diagram), the Object model (find Classes and class relations, Object interaction, Object to ER mapping), Statechart Diagram, and deployment diagram.
This technique is old and is not usually preferred.	This technique is new and is mostly preferred.

Q. Structured Analysis (SA) is primarily concerned with:

- A.** How the system will be implemented
- B.** What the system must do (logical design)
- C.** The coding structure of the system
- D.** The database schema

Q. Structured Analysis (SA) is primarily concerned with:

- A.** How the system will be implemented
- B.** What the system must do (logical design)
- C.** The coding structure of the system
- D.** The database schema

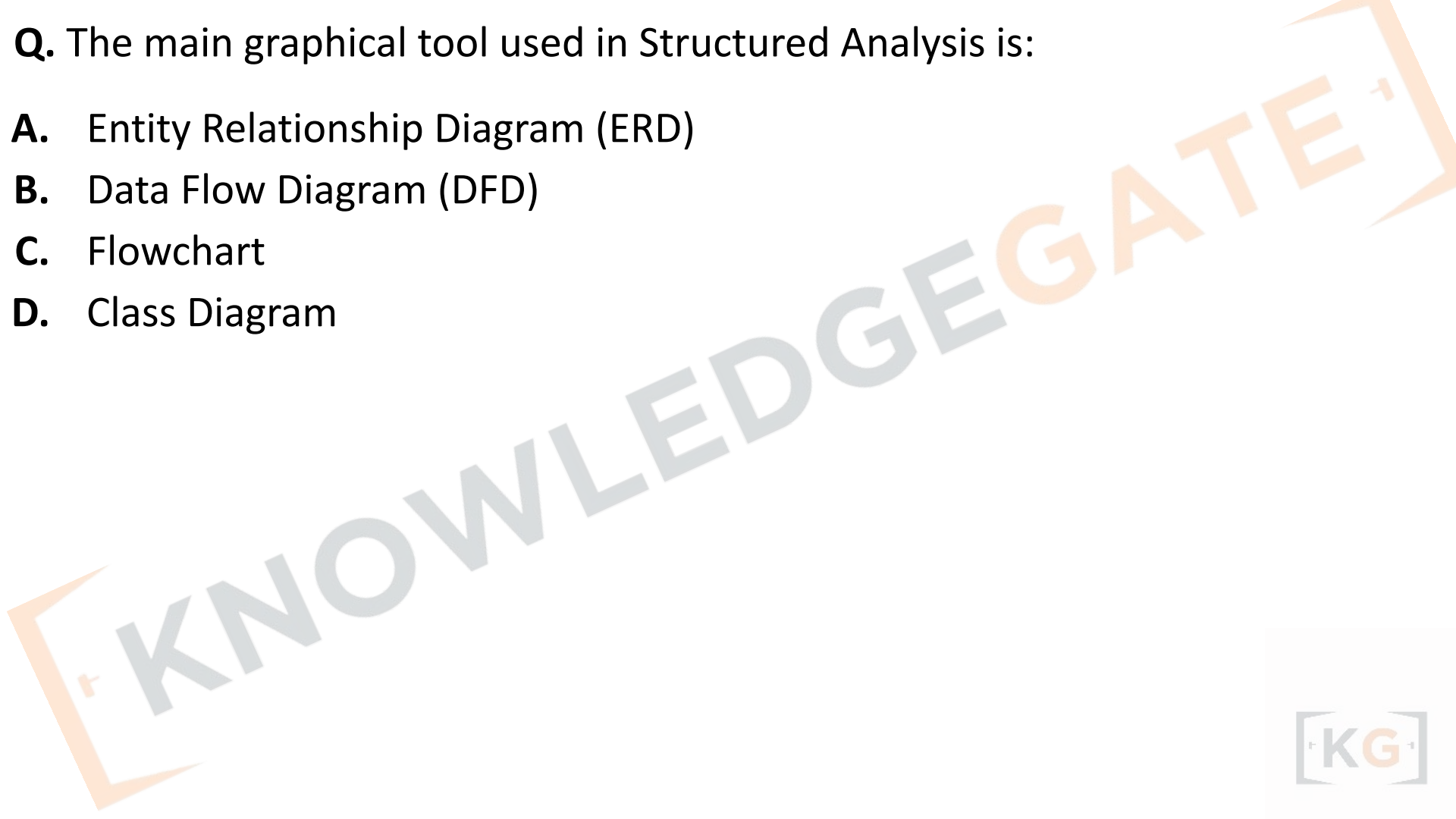
Answer: (b)

Explanation:

Structured Analysis focuses on **what the system must do**, describing the system in terms of **processes, data, and data flows**. It uses models like **Data Flow Diagrams (DFDs)** to represent system functionality logically — not physically.

Q. The main graphical tool used in Structured Analysis is:

- A.** Entity Relationship Diagram (ERD)
- B.** Data Flow Diagram (DFD)
- C.** Flowchart
- D.** Class Diagram



Q. The main graphical tool used in Structured Analysis is:

- A.** Entity Relationship Diagram (ERD)
- B.** Data Flow Diagram (DFD)
- C.** Flowchart
- D.** Class Diagram

Answer: (b)

Explanation:

The **Data Flow Diagram (DFD)** is the primary tool in Structured Analysis. It shows how **data moves** through processes and how it is stored, transformed, and output in a system.

Q. Structured Design (SD) focuses on:

- A.** Logical representation of processes
- B.** Physical implementation of data stores
- C.** Defining program modules and control relationships
- D.** Describing real-world entities as objects

Q. Structured Design (SD) focuses on:

- A.** Logical representation of processes
- B.** Physical implementation of data stores
- C.** Defining program modules and control relationships
- D.** Describing real-world entities as objects

Answer: (c)

Explanation:

Structured Design (SD) converts the **DFD from analysis into a hierarchy of modules**, showing how program components are related and controlled. It uses **structure charts** to represent this relationship.

Q. In Structured Analysis, the highest-level DFD is called:

- A.** Context Diagram
- B.** Level-1 DFD
- C.** Data Dictionary
- D.** Structure Chart

Q. In Structured Analysis, the highest-level DFD is called:

- A.** Context Diagram
- B.** Level-1 DFD
- C.** Data Dictionary
- D.** Structure Chart

Answer: (a)

Explanation:

The **Context Diagram** represents the **entire system as a single process**, showing its interactions with external entities (users, systems, etc.) — this is the top-level DFD.

Q. In structured design, good design aims to have:

- A.** High coupling and low cohesion
- B.** High cohesion and low coupling
- C.** Low cohesion and low coupling
- D.** No relationship between cohesion and coupling

Q. In structured design, good design aims to have:

- A.** High coupling and low cohesion
- B.** High cohesion and low coupling
- C.** Low cohesion and low coupling
- D.** No relationship between cohesion and coupling

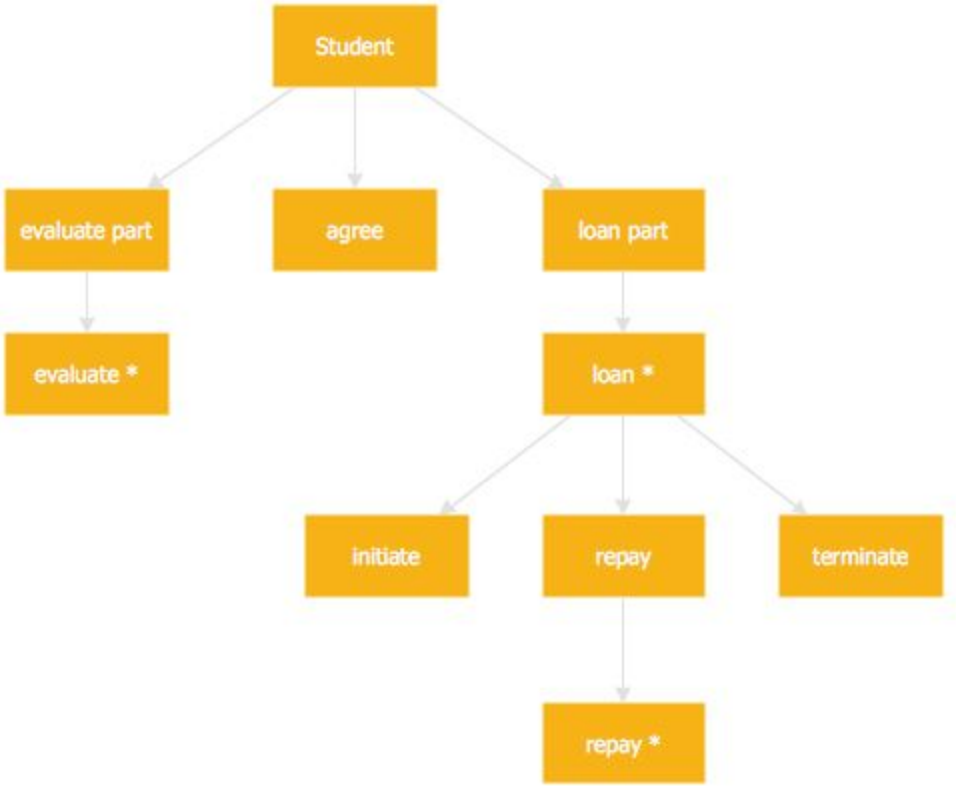
Answer: (b)

Explanation:

A well-structured system has **high cohesion** (modules do one clear task) and **low coupling** (modules are independent).

This improves **modularity, maintainability, and reusability.**

Jackson System Development (JSD)



Jackson System Development (JSD)

Jackson System Development (JSD) is a method of system development that covers the software life cycle either directly or by providing a framework into which more specialized techniques can fit. This article focuses on discussing JSD in detail.

What is Jackson System Development (JSD)?

Jackson system development is a linear system development approach that lets one describe and model the real world. JSD can start from the stage in a project when there is only a general statement of requirements.

1. However many projects that have used JSD started slightly later in the life cycle, doing the first steps largely from existing documents rather than directly with the users.
2. The later steps of JSD produce the code of the final system.
3. The output of the earlier steps is a set of program design problems.

Phases of JSD:

JSD has 3 phases:

1. Modeling Phase: In the modeling phase of JSD, the designer creates a collection of entity structure diagrams and identifies the entities in the system, the actions they perform, the attributes of the actions and the time order of the actions in the life of the entities.

2. Specification Phase: This phase focuses on actually what is to be done? Previous phase provides the basics for this phase. A sufficient model of a time-ordered world must itself be time-ordered. Major goal is to map progress in the real world to progress in the system that models it.

3. Implementation Phase: In the implementation phase JSD determines how to obtain the required functionality. Implementation way of the system is based on the transformation of the specification into an efficient set of processes. The processes involved in it should be designed in such a manner that it would be possible to run them on available software and hardware.

JSD Steps:

Initially there were six steps when it was originally presented by Jackson, they were as below:

1. Entity/action step
2. Initial model step
3. Interactive function step
4. Information function step
5. System timing step
6. System implementation step

Later some steps were combined to create method with only three steps:

1. Modeling Step
2. Network Step
3. Implementation Step

Merits of JSD:

- It is designed to solve real-time problems.
- JSD modeling focuses on time.
- It considers simultaneous processing and timing.
- Provides functionality in the real world.
- It is a better approach for microcode applications.

Demerits of JSD:

- It is a poor methodology for high level analysis and database design.
- JSD is a complex methodology due to pseudo code representation.
- It is less graphically oriented as compared to SA/SD or OMT.
- It is a bit complex and difficult to understand.

□ Mapping object oriented concepts using non-object oriented language

Implementing an object-oriented concept in a non-object oriented language requires the following steps:

1. Translate classes into data structures:

- i. Each class is implemented as a single contiguous block of attributes. Each attribute contains a variable. Now an object has state and identity and is subject to side effects.
- ii. A variable that identifies an object must therefore be implemented as a sharable reference.

2. Pass arguments to methods:

- i. Every method has at least one argument. In a non-object-oriented language, the argument must be made explicit.
- ii. Methods can contain additional objects as arguments. In passing an object as an argument to a method, a reference to the object must be passed if the value of the object can be updated within the method.

3. Allocate storage for objects:

- i. Objects can be allocated statically, dynamically or on a stack.
- ii. Most temporary and intermediate objects are implemented as stack-based variables.
- iii. Dynamically allocated objects are used when their number is not known at compile time.
- iv. A general object can be implemented as a data structure allocated on request at run time from a heap.

4 Implement inheritance in data structures: Following ways are use to implement data structure for inheritance in non object oriented programming language

- Avoid it.
- Flatten the class hierarchy.
- Break out separate objects.

5 Implement method resolutions: Method resolution is one. main features of an object-oriented language that is lacking in a non object-oriented language. Method resolution can be implemented in a following ways

- Avoid it.
- Resolve methods at compile time.
- Resolve methods at run time.

6. Implement associations: Implementing associations in a non oriented language can be done by :

- Mapping them into pointers.
- Implementing them directly as association container objects.

7. Deal with concurrency:

- Most languages do not explicitly support concurrency.
- Concurrency is usually needed only when more than one external event occurs, and the behavior of the program depends on their timing.

8. Encapsulate internal details of classes:

- Object-oriented languages provide constructs to encapsulate implementation.
- Some of this encapsulation is lost when object-oriented concepts translated into a non object- oriented language, but we can still take advantage of the encapsulation facilities provided by language

Can a class be a data structure?

A class is a syntactic way that some languages offer to group data and methods. Classes often lend themselves to being used to implement data structures, but **it would be incorrect to say that a class == a data structure.**

□ **Translating classes into data structures**

Normally you will implement each class as a single contiguous block of attributes—a record structure. Each attribute has a declared type, which can be a primitive type, such as integer, real or character or can be a structured value, such as an embedded record structure or a fixed-length array. A variable that identifies an object must therefore be implemented as a sharable reference and not simply by copying the values of an object's attributes.

Translating Classes into C Struct Declarations

Each class in the design becomes a C struct. Each attribute defined in the class becomes a field of the C struct. The structure for the Window class is declared as:

```
struct Window
```

```
{  
    Length xmin;  
    Length ymin;  
    Length xmax;  
    Length ymax;  
};
```

Length is a C type (not a class) defined with a C typedef statement to provide greater modularity in the definition of values:

```
typedef float Length;
```

Q. Jackson System Development (JSD) is primarily based on:

- A.** Function-oriented design
- B.** Data structure-oriented design
- C.** Object-oriented design
- D.** Process flow-based design

Q. Jackson System Development (JSD) is primarily based on:

- A.** Function-oriented design
- B.** Data structure-oriented design
- C.** Object-oriented design
- D.** Process flow-based design

Answer: (b)

Explanation:

JSD (Jackson System Development) is a **data structure-oriented software development method**.

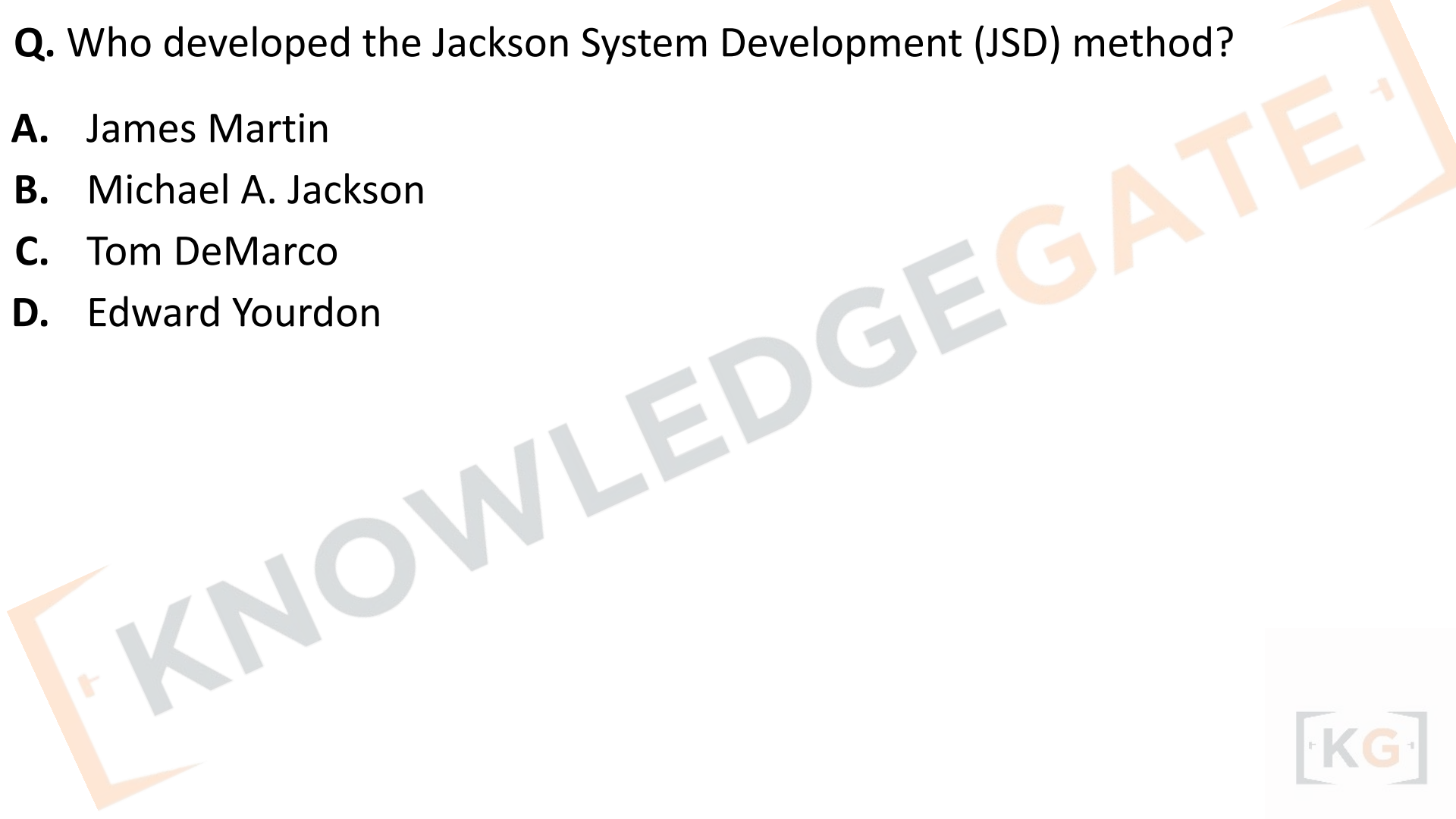
It begins by analyzing **real-world entities and their data structures**, and then derives system processes from these structures.

It emphasizes **correspondence between data and program structure**.



Q. Who developed the Jackson System Development (JSD) method?

- A.** James Martin
- B.** Michael A. Jackson
- C.** Tom DeMarco
- D.** Edward Yourdon



Q. Who developed the Jackson System Development (JSD) method?

- A.** James Martin
- B.** Michael A. Jackson
- C.** Tom DeMarco
- D.** Edward Yourdon

Answer: (b)

Explanation:

Michael A. Jackson developed JSD in the **1980s** as an evolution of his earlier **Jackson Structured Programming (JSP)**.

While JSP focused on program design, JSD extended the concept to **full system development**.

Q. Which of the following is NOT a main phase of Jackson System Development (JSD)?

- A.** System Specification
- B.** Implementation
- C.** Object Modeling
- D.** Program Testing

Q. Which of the following is NOT a main phase of Jackson System Development (JSD)?

- A.** System Specification
- B.** Implementation
- C.** Object Modeling
- D.** Program Testing

Answer: (d)

Explanation:

The **main phases of JSD** are:

- 1. Entity Modeling (Object Modeling)**
- 2. System Timing (Modeling process sequences)**
- 3. System Functioning (Design and Implementation)**

Program Testing is a general step in software life cycles but **not a specific JSD phase**.

Q. The main modeling technique used in JSD for representing system entities and actions is:

- A.** Data Flow Diagram (DFD)
- B.** Entity-Structure Diagram (ESD)
- C.** Class Diagram
- D.** Control Flow Graph

Q. The main modeling technique used in JSD for representing system entities and actions is:

- A.** Data Flow Diagram (DFD)
- B.** Entity-Structure Diagram (ESD)
- C.** Class Diagram
- D.** Control Flow Graph

Answer: (b)

Explanation:

JSD uses **Entity-Structure Diagrams (ESDs)** to model **real-world entities and their life histories (actions and events)**.

An ESD shows how data entities evolve over time — similar to an object's lifecycle in OOP.

Q. In Jackson System Development (JSD), “System timing” refers to:

- A.** Scheduling of CPU processes
- B.** Determining sequence and concurrency of entity actions
- C.** Determining data type definitions
- D.** Designing user interface timing constraints

Q. In Jackson System Development (JSD), “System timing” refers to:

- A.** Scheduling of CPU processes
- B.** Determining sequence and concurrency of entity actions
- C.** Determining data type definitions
- D.** Designing user interface timing constraints

Answer: (b)

Explanation:

System Timing in JSD deals with **temporal behavior of entities**, i.e., how entity actions occur **sequentially or concurrently** over time.

It defines the **timing relationships** among processes derived from entities.

Chapter - 4

Object oriented programming style Introduction
To Java



Java's Lineage

- Java is related to C++, which is a direct descendant of C.
- Much of the character of Java is inherited from these two languages.
- From C, Java derives its syntax.
- Many of Java's object-oriented features were influenced by C++.

C++: The Next Step

- C is a successful and useful language, you might ask *why a need for something else* existed. The answer is *complexity*.
- To solve this problem, a new way to program was invented, called *object-oriented programming (OOP)*.
- C++ was invented by Bjarne Stroustrup in 1979, while he was working at Bell Laboratories in Murray Hill, New Jersey.
- Stroustrup initially called the new language “C with Classes.”
- However, in 1983, the name was changed to C++. C++ extends C by adding object-oriented features.

The Stage Is Set for Java

- By the end of the 1980s and the early 1990s, object-oriented programming using C++ took hold.
- Within a few years, the World Wide Web and the Internet would reach critical mass.
- This event would precipitate another revolution in programming.

The Creation of Java

- The second force was, of course, the World Wide Web.
- The emergence of the World Wide Web, Java was propelled to the forefront of computer language design, because the Web, too, demanded portable programs.
- The Internet ultimately led to Java's large-scale success.
- Java is simply the "Internet version of C++."
- Java was to Internet programming what C was to system programming: a revolutionary force that changed the world.

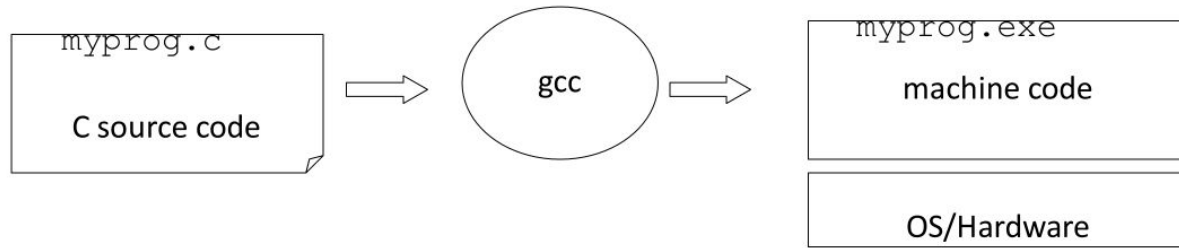
How Java Changed the Internet

- Java innovated a new type of networked program called the applet that changed the way the online world thought about content.
- Java also addressed some of the thorniest issues associated with the Internet: portability and security.

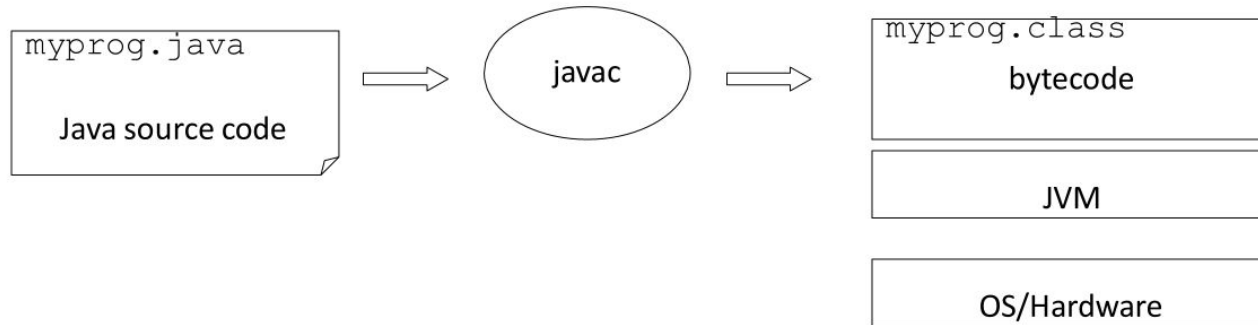
JVM

- A **Virtual Machine** is a software implementation of a physical machine. Java was developed with the concept of **WORA (Write Once Run Anywhere)**, which runs on a **VM**.
- The **compiler** compiles the Java file into a Java **.class** file, then that **.class** file is input into the JVM, which loads and executes the class file.

Platform Dependent



Platform Independent



Java Program

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java

Compiler

Interpreter

Interpreter

Interpreter

Hello
World!

Win32

Hello
World!

Solaris

Hello
World!

MacOS

The Java Buzzwords

- The key considerations were summed up by the Java team in the following list of buzzwords:
 - Simple
 - Secure
 - Portable
 - Object-oriented
 - Robust
 - Multithreaded
 - Architecture-neutral
 - Interpreted
 - High performance
 - Distributed
 - Dynamic

The Java Buzzwords : Simple

- Based on popular languages called C and C++
- C: old, pretty bare bones language
- C++: newer, more complicated language
- Start from C and add some of C++'s more useful features

“Java omits many rarely used, poorly understood, confusing features of C++ that in our experience bring more grief than benefits” (Gosling)

The Java Buzzwords : Object Oriented

- The **object-oriented** paradigm
 - problems and their solutions are packaged in terms of *classes*
 - the information in a class is the *data*
 - the functionality in a class is the *method*
 - a class provides the framework for building *objects*
- Object-oriented programming (OOP) allows pieces of programs to be used in other contexts more easily

The Java Buzzwords : Robust

- Program must execute reliably in a variety of systems.
- Java is a strictly typed language, it checks your code at compile time. However, it also checks your code at run time.
- A **robust** program may not do exactly what it is supposed to do, but it should not bring down other unrelated programs down with it.
- Reliability
 - early (compile time) checking
 - dynamic (runtime) checking
 - eliminating situations that are error prone.
- Example: Two of the main reasons for program failure: memory management mistakes and mishandled exceptional conditions (that is, run-time errors).

The Java Buzzwords : Multithread

- Java supports multithreaded programming, which allows you to write programs that do many things simultaneously.
- A thread is a part of the program that can operate independently of its other parts.
- **Multi-threaded** programs can do multiple things at once
 - Example:
 - Download a file from the web while still looking at other web pages.
- Question: What is the problem with multiple agents working at the same time?
 - Synchronization

The Java Buzzwords : Architecture-Neutral

- Goal was “write once; run anywhere, any time, forever.”
- One of the main problems facing programmers is that no guarantee exists that if you write a program today, it will run tomorrow—even on the same machine.
- Operating system upgrades, processor upgrades, and changes in core system resources can all combine to make a program malfunction.

The Java Buzzwords : Interpreted and High Performance

- Java enables the creation of cross-platform programs by compiling into an intermediate representation called Java bytecode. This code can be executed on any system that implements the Java Virtual Machine.
- Most previous attempts at cross-platform solutions have done so at the expense of performance.
- The Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a just-in-time compiler (JIT).
- Java run-time systems that provide this feature lose none of the benefits of the platform-independent code.

The Java Buzzwords : Interpreted and High Performance

- Java enables the creation of cross-platform programs by compiling into an intermediate representation called Java bytecode. This code can be executed on any system that implements the Java Virtual Machine.
- Most previous attempts at cross-platform solutions have done so at the expense of performance.
- The Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a just-in-time compiler (JIT).
- Java run-time systems that provide this feature lose none of the benefits of the platform-independent code.

The Java Buzzwords: Dynamic

- Java programs carry with them substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time.
- This makes it possible to dynamically link code in a safe and expedient manner.
- This is crucial to the robustness of the Java environment, in which small fragments of bytecode may be dynamically updated on a running system.

The Evolution of Java

- JDK 1.02 (1995)
- JDK 1.1 (1996)
- Java 2 SDK v 1.2 (a.k.a JDK 1.2, 1998)
- Java 2 SDK v 1.3 (a.k.a JDK 1.3, 2000)
- Java 2 SDK v 1.4 (a.k.a JDK 1.4, 2002)
- Java Standard Edition (J2SE)
 - J2SE can be used to develop client-side standalone applications or applets.
- Java Enterprise Edition (J2EE)
 - J2EE can be used to develop server-side applications such as Java servlets and Java ServerPages.
- Java Micro Edition (J2ME).
 - J2ME can be used to develop applications for mobile devices

Types of Java Programs

- **Standalone applications (J2SE)**

- AWT (Abstract Window Toolkit) and Swing are used to create standalone applications.

- **Web applications (JSP)**

- Applet, Servlet, JSP, JSF are used to create web applications.

- **Enterprise applications (J2EE)**

- Designed for corporate sides such as banking business systems.

- **Mobile applications (J2ME)**

- Mobile applications are developed to run on mobile phones and tablets.

Java IDE Tools

- Forte by Sun Microsystems
- Borland JBuilder
- Microsoft Visual J++
- WebGain Café
- IBM Visual Age for Java
- ECLIPSE

Object-Oriented Programming

- **Two Paradigms**

- ❑ All computer programs consist of two elements: **code** and **data**.
- Furthermore, a program can be conceptually organized around its code or around its data. That is, some programs are written around “**what is happening**” and others are written around “**who is being affected.**”
- These are the two paradigms that govern how a program is constructed.

Object-Oriented Programming

process-oriented model

- The process-oriented model can be thought of as *code acting on data*.
- C employs this model.

object-oriented programming

- Object-oriented programming organizes a program around its data (that is, **objects**) and a set of well-defined interfaces to that data.
- An object-oriented program can be characterized as **data controlling access to code**.

Abstraction

- An essential element of object-oriented programming is *abstraction*.
- *Humans manage* complexity through abstraction.

Example: people do not think of a car as a set of tens of thousands of individual parts. Instead, they are free to utilize the object as a whole.

- A powerful way to manage abstraction is through the use of **hierarchical classifications**.
- This allows you to layer the semantics of complex systems, **breaking** them into more manageable pieces.

Abstraction

- Hierarchical abstractions of complex systems can also be **applied** to computer programs.
- The data from a traditional process-oriented program can be transformed by abstraction into its component objects.
- A sequence of process steps can become a collection of messages between these objects. Thus, each of these objects describes its own **unique behavior**.
- You can treat these objects as **concrete entities** that respond to messages telling them to *do something*.
- This is the essence of object-oriented programming.
- Object-oriented programming is a powerful and natural paradigm for creating programs that survive the inevitable changes accompanying the life cycle of any major software project, including conception, growth, and aging.

The Three OOP Principles

1. Encapsulation
2. Inheritance
3. Polymorphism

Encapsulation

- *Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.*
- One way to think about encapsulation is as a **protective wrapper** that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.
- To relate this to the real world: you can't affect the transmission by using the turn signal or windshield wipers, for example.

Encapsulation

- In Java, the basis of encapsulation is the **class**.
- A *class defines* the structure and behavior (data and code) that will be shared by a set of **objects**.
- Each object of a given class contains the structure and behavior defined by the class, as if it were stamped out by a mold in the shape of the class.
- For this reason, objects are sometimes referred to as **instances of a class**.
- *Thus, a class is a logical construct; an object has physical reality.*

Encapsulation

- When you create a class, you will specify the **code** and **data** that constitute that class. Collectively, these elements are called *members of the class*.
- Specifically, the data defined by the class are referred to as **member variables** or **instance variables**. The code that operates on that data is referred to as **member methods** or just **methods**.
- Since the purpose of a class is to encapsulate complexity, there are mechanisms for **hiding** the complexity of the implementation inside the class.
- Each method or variable in a class may be marked

Object

Class



Nancy
1999

James
1876

Goldy
1654

Gabby
1659

Class Student

name

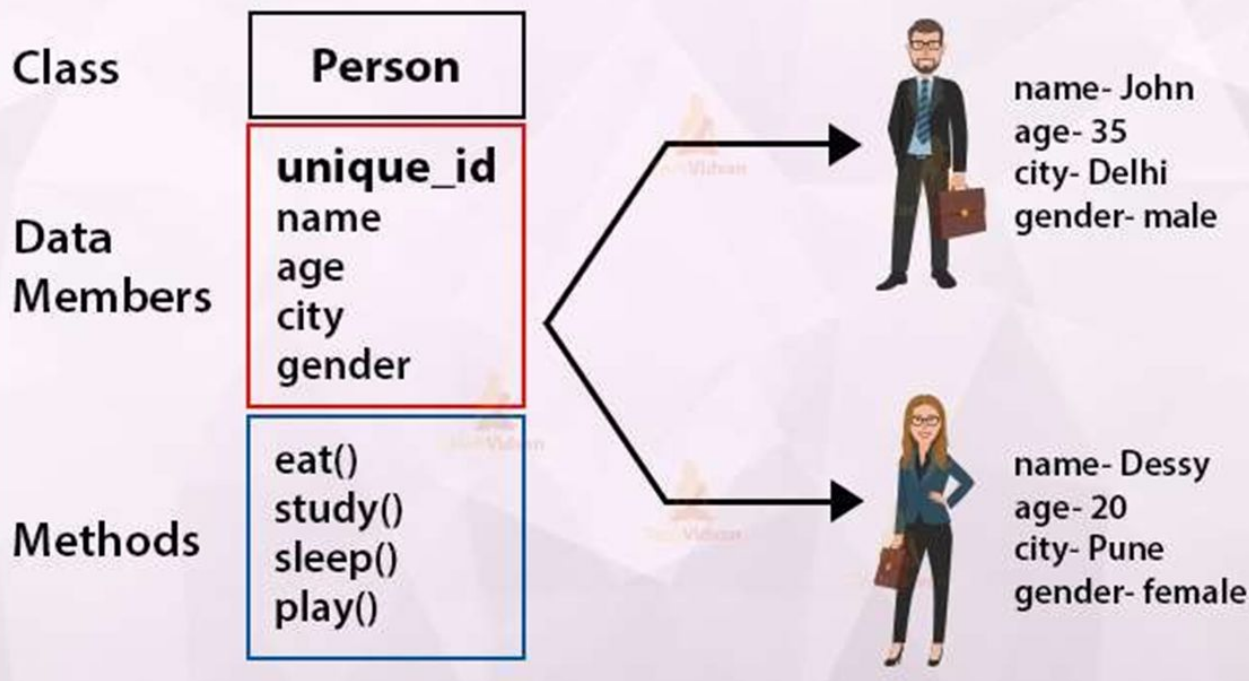
rollNo

setName()

setRollNo()

Blueprint

Java Class & Objects



```
class
{
    //member data
    int x;
    int y;

    //member method
    public void getxy()
    {
        //code
    }
    public void putxy()
    {
        //code
    }
}
```

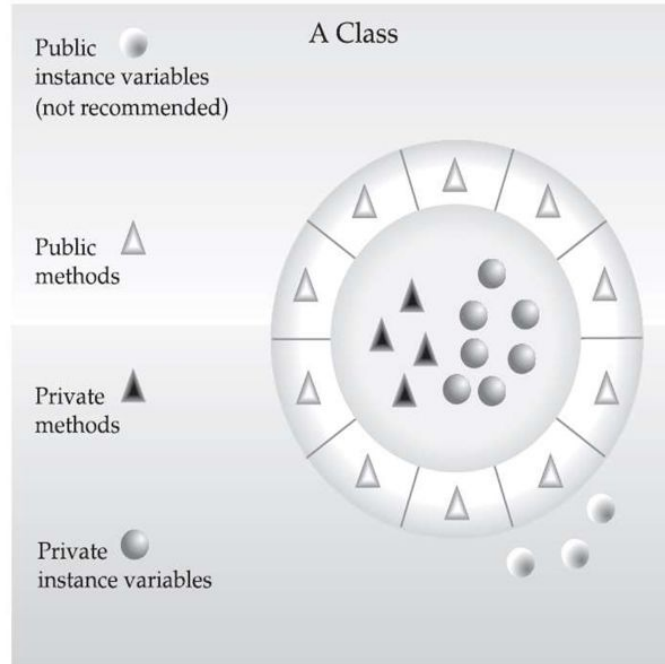
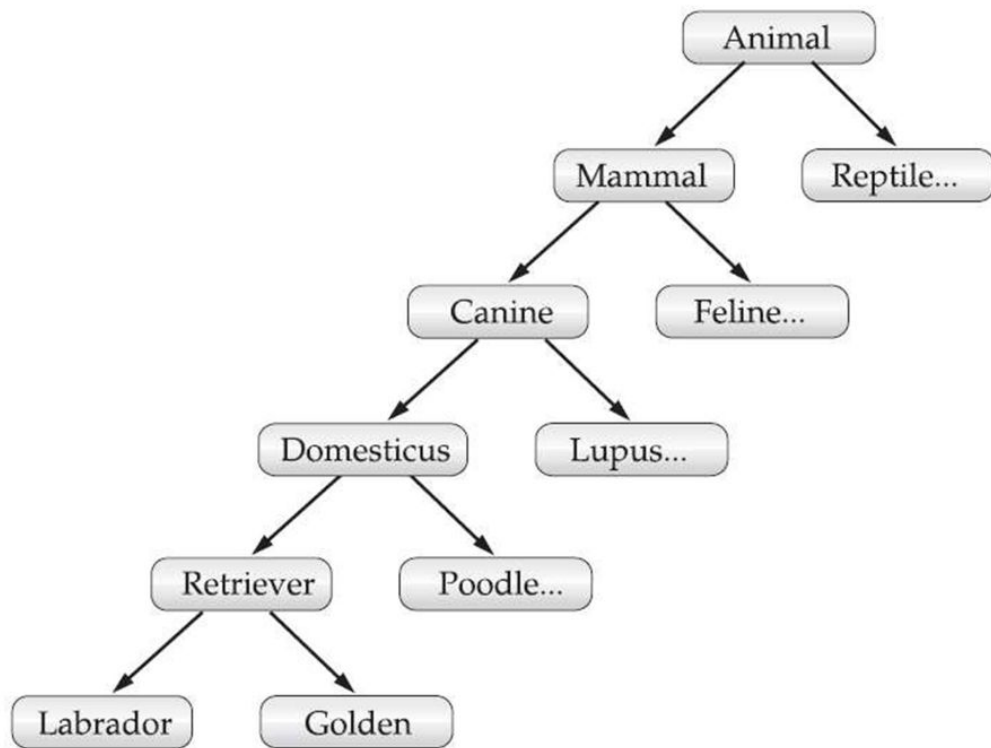


FIGURE 2-1 Encapsulation: public methods can be used to protect private data

Inheritance

- *Inheritance is the process by which one object acquires the properties of another object.* This is important because it supports the concept of **hierarchical classification**.
- With inheritance, an object need only define those qualities that make it **unique** within its class. It can inherit its general attributes from its parent.
- Thus, it is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case.
- A deeply inherited subclass inherits all of the attributes from each of its ancestors in the *class hierarchy*.+



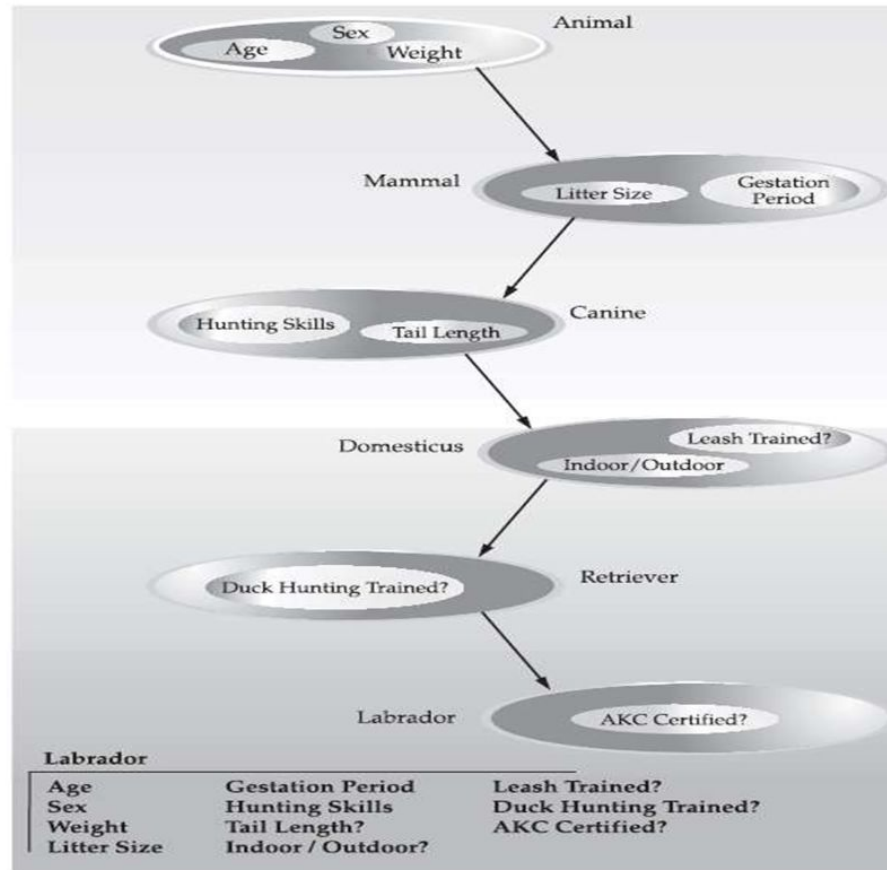


FIGURE 2-2 Labrador inherits the encapsulation of all its superclasses

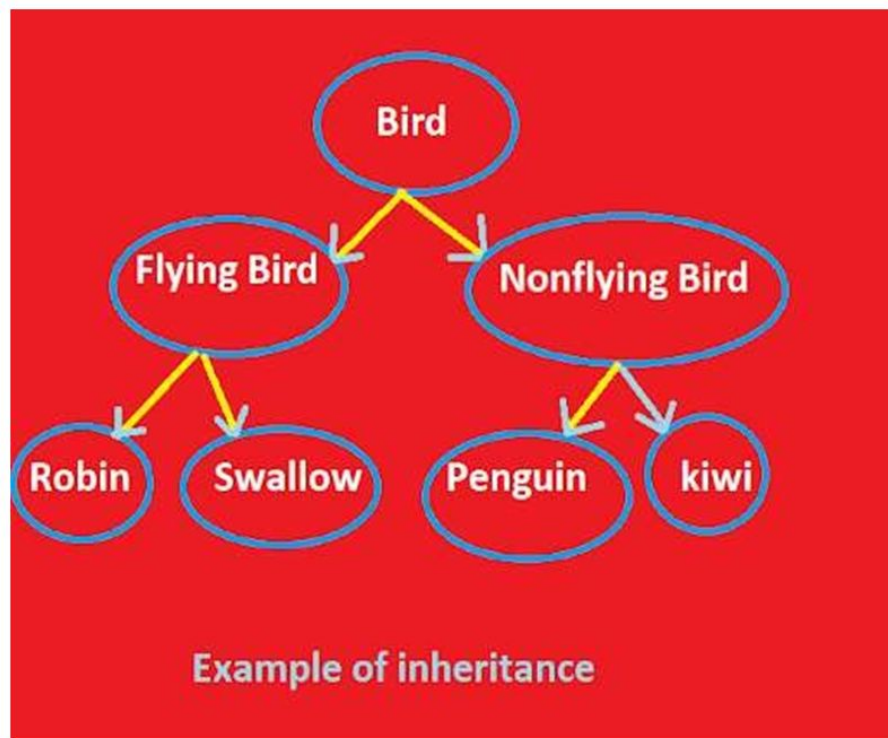
Grand Parent



Parent



Child



Inheritance

- Inheritance **interacts** with encapsulation as well. If a given class encapsulates some attributes, then any subclass will have the same attributes *plus any that it adds as part of its specialization*.
- This is a key concept that lets object-oriented programs **grow** in complexity linearly rather than geometrically.
- A new subclass **inherits** all of the attributes of all of its ancestors. It does not have unpredictable interactions with the majority of the rest of the code in the system.

Polymorphism

- *Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions.*

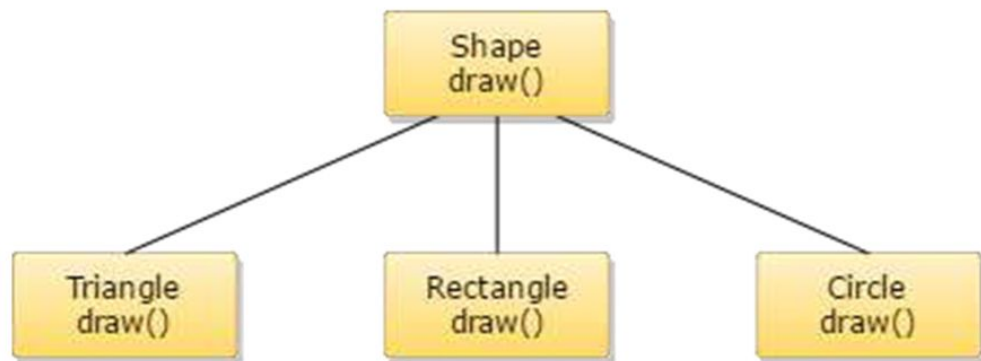
Ex: Consider a stack (which is a last-in, first-out list).

- The concept of polymorphism is often expressed by the phrase “**one interface, multiple methods.**”
- This means that it is possible to design a **generic interface** to a group of related activities.
- This helps reduce complexity by allowing the same interface to be used to specify a *general class of action*.
- *It is the compiler’s job to select the specific action* (that is, method) as it applies to each situation. You, the programmer, do not need to make this selection manually. You need only remember and utilize the general interface.

Example: A dog’s sense of smell is polymorphic.

Polymorphism, Encapsulation, and Inheritance Work Together

- When properly applied, polymorphism, encapsulation, and inheritance combine to produce a programming environment that supports the development of far more robust and scalable programs than does the process-oriented model.
- A well-designed hierarchy of classes is the basis for reusing the code in which you have invested time and effort developing and testing.
- Encapsulation allows you to migrate your implementations over time without breaking the code that depends on the public interface of your classes.
- Polymorphism allows you to create clean, sensible, readable, and resilient code.



Polymorphism

A First Simple Program

```
/*  
    This is a simple Java program.  
    Call this file "Example.java".  
*/  
class Example {  
    // Your program begins with a call to main().  
    public static void main(String args[]) {  
        System.out.println("This is a simple Java program.");  
    }  
}
```

- **Enter the program and save as Example.java**
- **Compile the Program**
c:\myjava\javac Example.java
- **Run the Program**
c:\myjava\javac Example

Comments

- The contents of a **comment** are ignored by the compiler. Instead, a comment describes or explains the operation of the program to anyone who is reading its source code.
- Java supports three styles of comments:
 - *multiline comment* (`/* ... */`)
 - *single-line comment* (`// ...`)
 - *documentation comment* (`/** ... */`)
 - This type of comment is used to produce an HTML file that documents your program.

public static void main(String args[])

Here's a brief breakdown of each keyword and part:

- **public** : Access specifier. Allows the method to be called from anywhere, even outside the class. Necessary because the Java Virtual Machine (JVM) needs to call main().
- **static**: Means the method belongs to the class, not an object. JVM can call main() without creating an object of the class.
- **void**: Return type. Indicates that main() does not return any value.
- **main**: The entry point of any Java program. JVM always looks for this method to start execution.
- **String args[]**: Parameter of the method. args is an array of String objects that stores command-line arguments.

Example: If you run `java MyClass hello world`, then `args[0] = "hello"`, `args[1] = "world"`.

System.out.println()

- **println()**

- ☐ It displays the string (or value) passed to it on the console.
- ☐ After printing, it moves the cursor to the next line.

- **System and out**

- ☐ System → a predefined Java class that provides access to system resources.
- ☐ out → a static member (output stream) of System that sends data to the console.

- **Semicolon (;)**

- ☐ Every statement in Java must end with a semicolon.

- **Variable**

- ☐ A variable is a **named memory location** that stores a value.
- ☐ The value can be assigned and changed during program execution.

```
class Example2 {  
    public static void main(String args[]) {  
        int num;        // this declares a variable called num  
        num = 100;      // this assigns num the value 100  
  
        System.out.println("This is num: " + num);  
  
        num = num * 2;  
        System.out.print("The value of num * 2 is ");  
        System.out.println(num);  
    }  
}
```

type var-name;

***type** specifies the type of variable being declared,
and **var-name** is the name of the variable.*

Lexical Issues

- Java programs are a collection of **whitespace, identifiers, literals, comments, operators, separators, and keywords.**
- **Whitespace**
 - Java is a free-form language. This means that you do not need to follow any special indentation rules.
 - In Java, whitespace is a space, tab, or newline.
- **Identifiers**
 - Identifiers are used for class names, method names, and variable names.
 - An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters.
 - They must not begin with a number.
 - Java is case-sensitive.

Lexical Issues

- **Literals**

- A **constant** value in Java is created by using a *literal* representation of it.

- **Comments**

- Three types of comments defined by Java.

Lexical Issues

- Separators

- In Java, there are a few characters that are used as separators.
- The most commonly used separator in Java is the semicolon.

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.

Lexical Issues

- The Java Keywords

- There are 50 keywords currently defined in the Java language
- These keywords cannot be used as names for a variable, class, or method.
- In addition to the keywords, Java reserves the following:
true, false, and null. These are values defined by Java.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

The Java Class Libraries

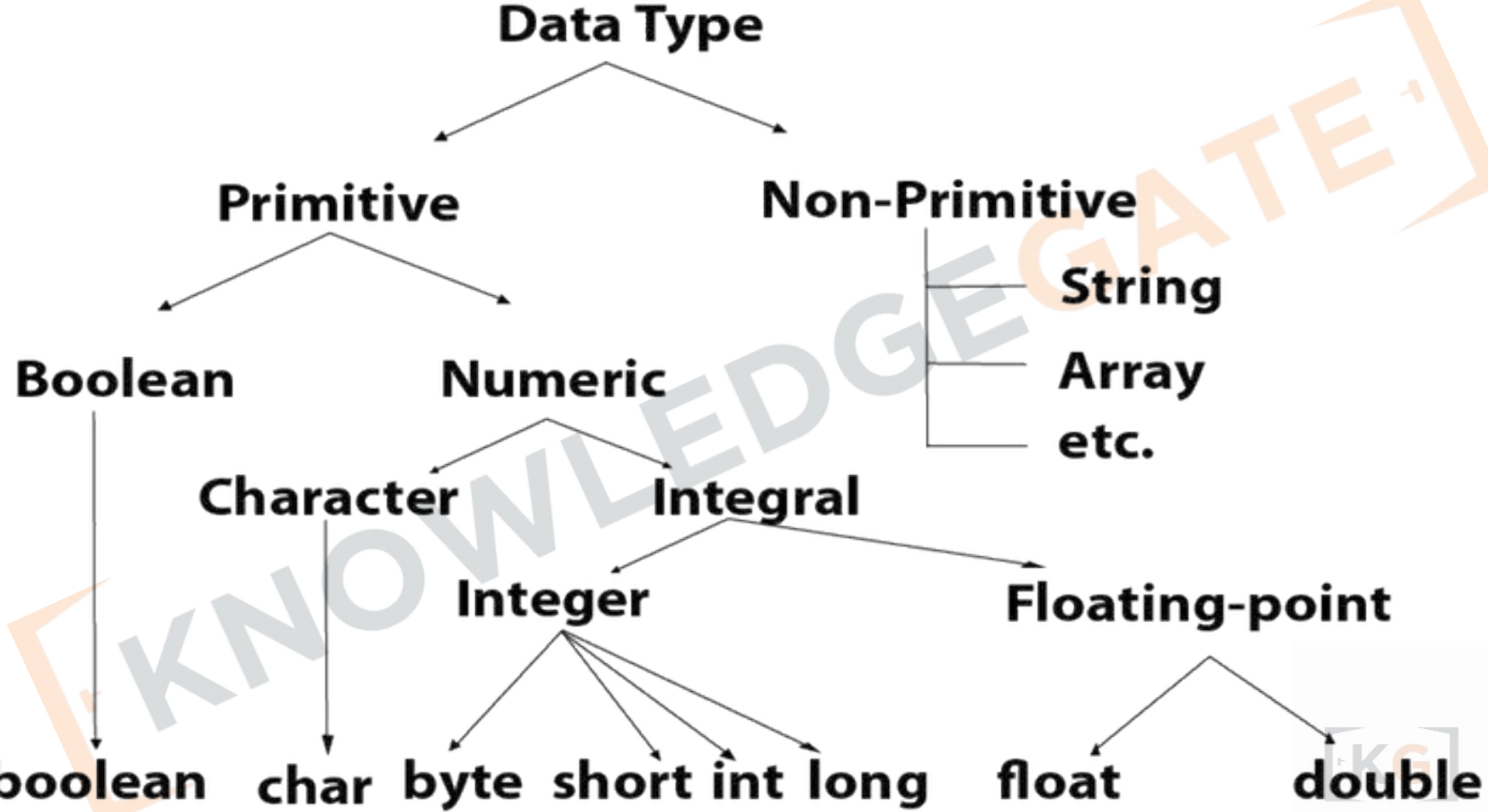
- `println( )` and `print( )`, these methods are members of the **System** class.

Java Is a Strongly Typed Language

- Java is a strongly typed language.
- **First**, every variable has a type, every expression has a type, and every type is strictly defined.
- **Second**, all assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility.
- There are **no automatic coercions or conversions** of conflicting types as in some languages.
- The Java compiler checks all expressions and parameters to ensure that the types are compatible.
- Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

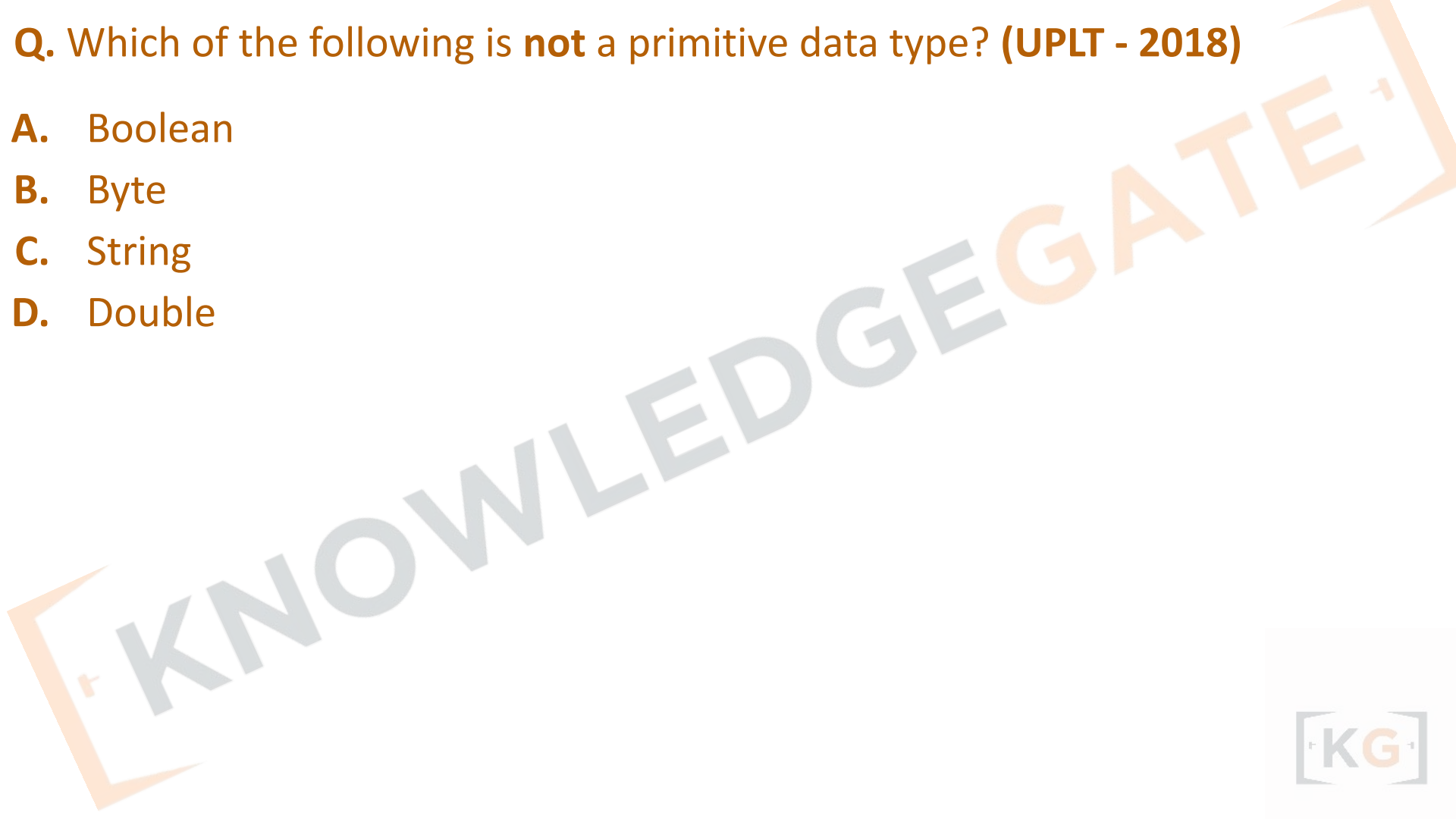
Data Types in Java

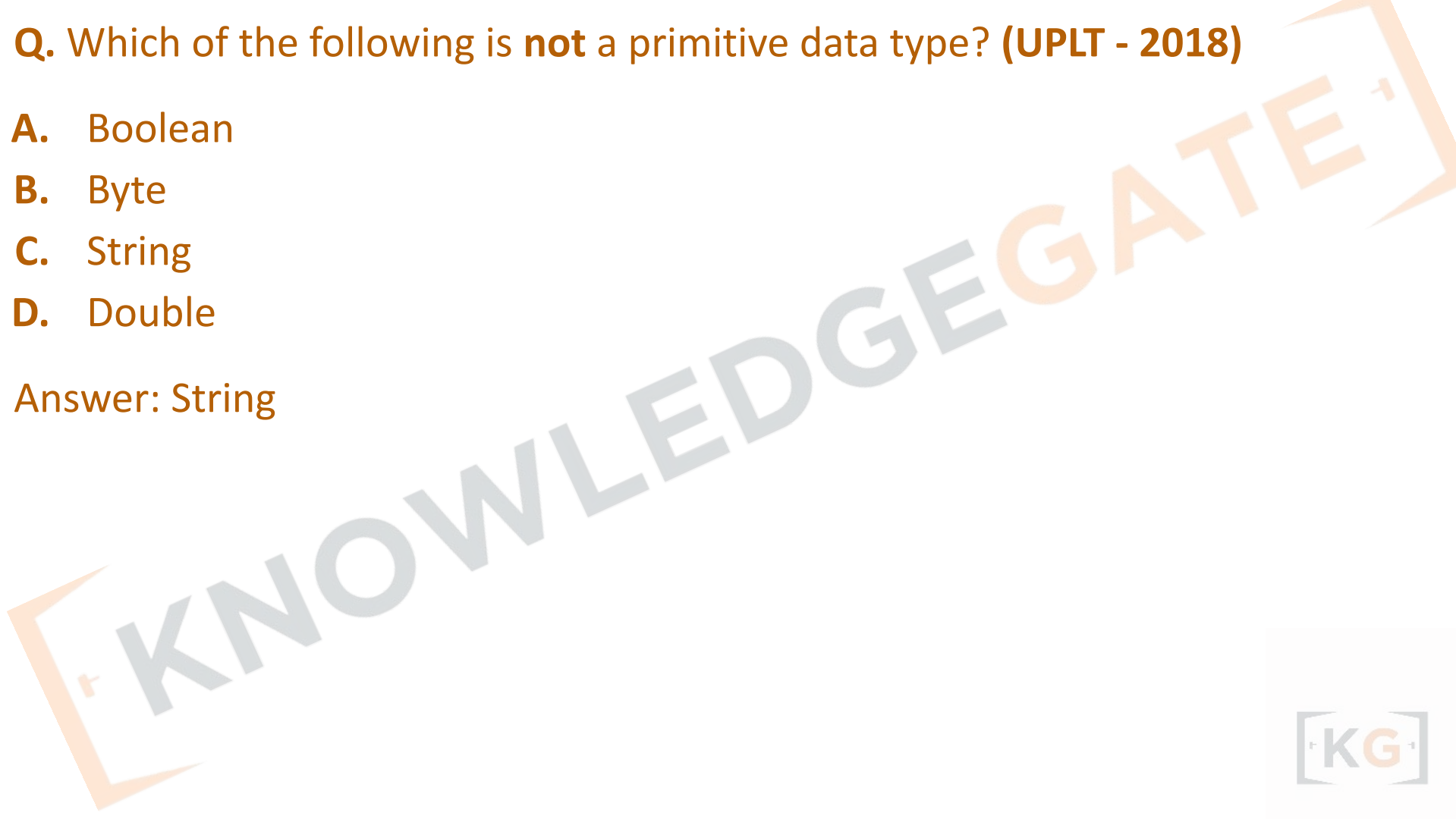
- Data types specify the different sizes and values that can be stored in the variable.
- There are two types of data types in Java:
 1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
 2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.
- Integers: This group includes **byte, short, int, and long**, which are for whole-valued signed numbers.
- Floating-point numbers: This group includes **float and double**, which represent numbers with fractional precision.
- Characters :This group includes **char**, which represents symbols in a character set, like letters and numbers.
- Boolean: This group includes **boolean**, which is a special type for representing true/false values.



Q. Which of the following is **not** a primitive data type? (UPLT - 2018)

- A. Boolean
- B. Byte
- C. String
- D. Double





Q. Which of the following is **not** a primitive data type? (UPLT - 2018)

- A. Boolean
- B. Byte
- C. String
- D. Double

Answer: String



Q. `string var1[ ];`
`var1 = new string[3];`
`system.out.println(var1[1]);`
Print **(UPLT - 2018)**

- A. an empty string
- B. null
- C. 0
- D. a garbage character

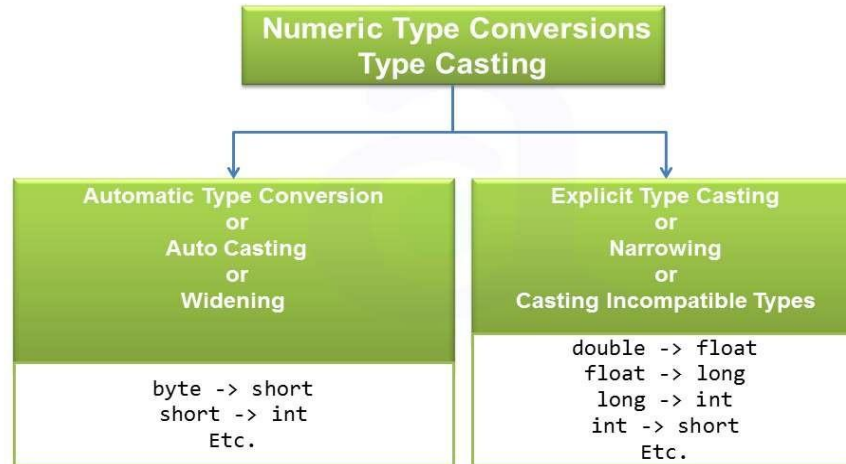
Q. `string var1[ ];`
`var1 = new string[3];`
`system.out.println(var1[1]);`
Print (UPLT - 2018)

- A.** an empty string
- B.** null
- C.** 0
- D.** a garbage character

Answer: B

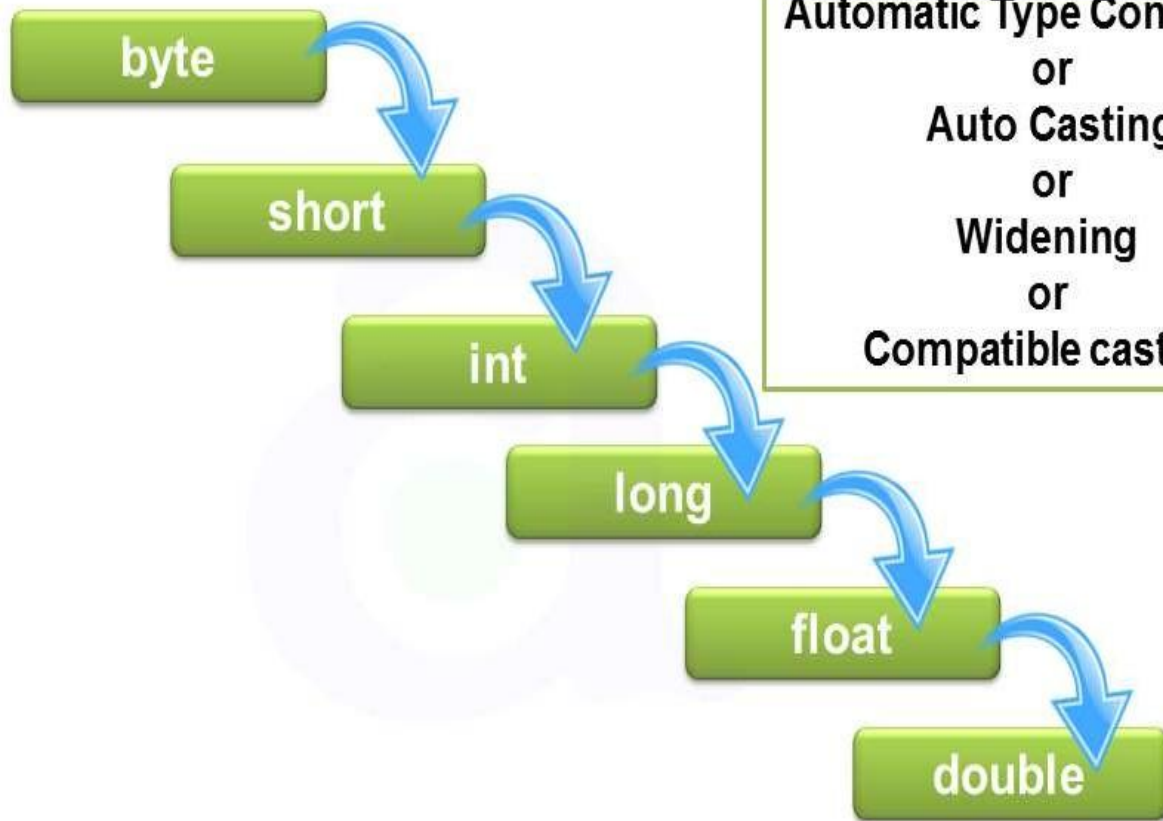
Type Conversion and Casting

- If the two types are compatible, then Java will perform the conversion **automatically**.
- Ex: assign an **int** value to a **long** variable
- no automatic conversion defined from **double to byte**.
- Conversion between incompatible types, use a **cast**, which performs **an explicit conversion** between incompatible types.



Java's Automatic Conversions

- When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:
 - The two types are **compatible**.
 - The destination type is **larger** than the source type.
- Java also performs an automatic type conversion when storing a literal integer constant into variables of type **byte, short, long, or char**.



**Automatic Type Conversion
or
Auto Casting
or
Widening
or
Compatible casting**

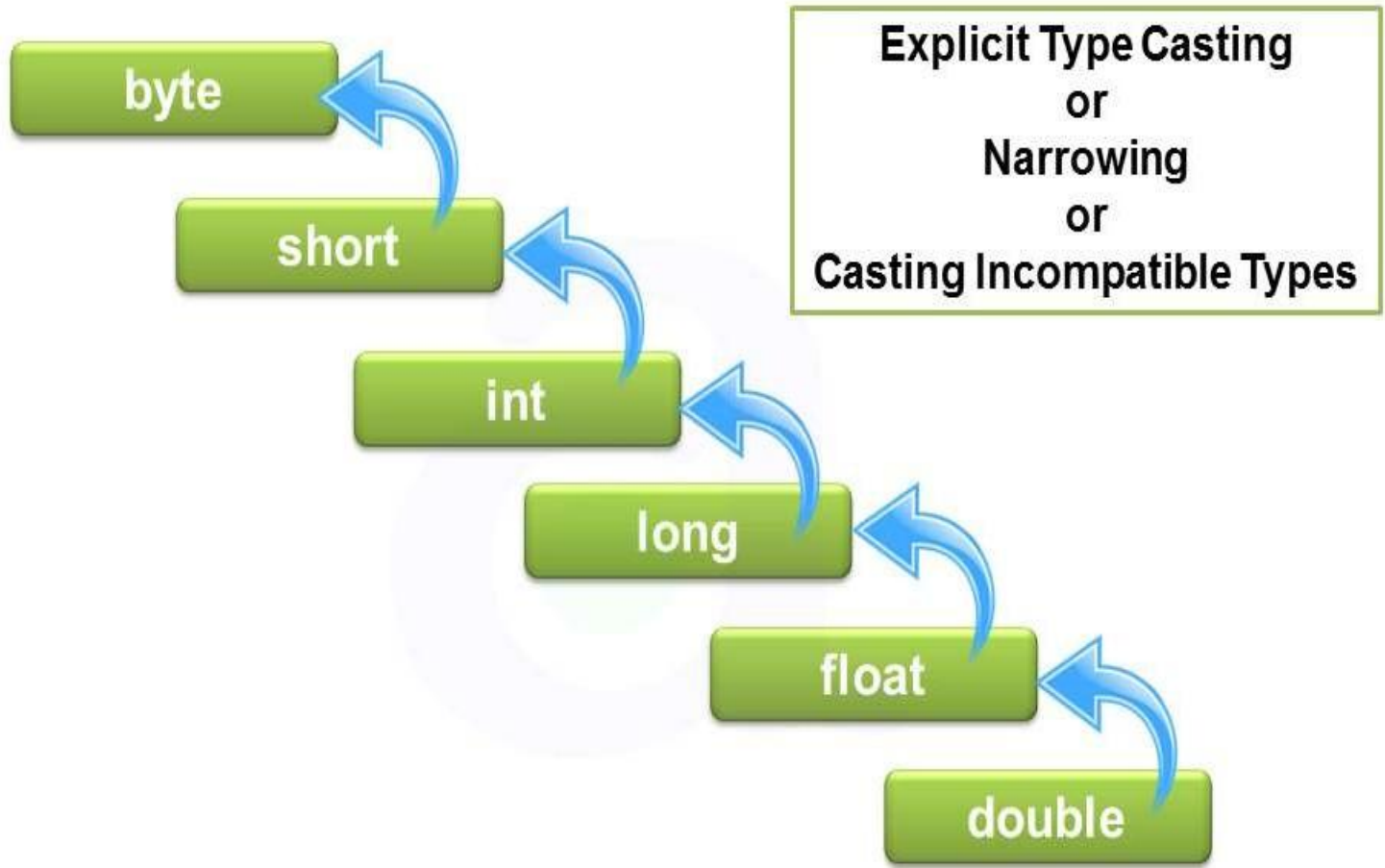
Casting Incompatible Types

- To create a conversion between two **incompatible types**, you must use a **cast**.
- A *cast* is simply an **explicit type conversion**. Syntax:

(target-type) value

- Assign an int value to a byte variable, byte is smaller than an int, This kind of conversion is sometimes called a **narrowing conversion**
- A different type of conversion will occur when a floating- point value is assigned to an integer type: **truncation**.

```
int a;  
byte b;  
// ...  
b = (byte) a;
```



Operators

- Java provides a rich operator environment. Most of its operators can be divided into the following four groups:
 - arithmetic,
 - bitwise,
 - relational, and
 - logical.
- Java also defines some additional operators that handle certain special situations.

Arithmetic Operators

The operands of the arithmetic operators must be of a numeric type.

You **cannot** use them on boolean types, but you can use them on char types, since the char type in Java is, essentially, a subset of int.

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

The Bitwise Operators

- Java defines several *bitwise operators* that can be applied to the integer types, long, int, short, char, and byte.
- These operators act upon the individual **bits** of their operands.

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Shift Operators

- These operators are used to shift the bits of a number left or right thereby **multiplying** or **dividing** the number by two respectively.
- They can be used when we have to multiply or divide a number by two.
- Syntax

number **shift_op** number_of_shift;

- **Signed Right shift operator (>>)**
- **Unsigned Right shift operator (>>>)**
- **Left shift operator (<<)**

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

The outcome of these operations is a **boolean** value.

Boolean Logical Operators

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

Short-Circuit Logical

Operators

- Java provides two interesting Boolean operators, These are secondary versions of the Boolean **AND** and **OR** operators, and are known as *short-circuit logical operators*.
- *The OR operator results in true when A is true, no matter what B is. Similarly, the AND operator results in false when A is false, no matter what B is.*
- If you use the `||` and `&&` forms, rather than the `|` and `&` forms of these operators, Java will not bother to evaluate the right-hand operand when the outcome of the expression can be determined by the left operand alone.
- This is very useful when the right-hand operand depends on the value of the left one in order to function properly.

Short-Circuit Logical Operators

- `if (denom != 0 && num / denom > 10)`
- Since the short-circuit form of AND (`&&`) is used, there is no risk of causing a run-time exception when `denom` is zero.
- If this line of code were written using the single `&` version of AND, both sides would be evaluated, causing a run-time exception when `denom` is zero.

The Assignment

~~var~~ = expression;

```
int x, y, z;
```

```
x = y = z = 100; // set x, y, and z to 100
```

- This fragment sets the variables x, y, and z to 100 using a single statement.
- This works because the = is an operator that yields the value of the **right-hand expression**.
- Thus, the value of z = 100 is 100, which is then assigned to y, which in turn is assigned to x.
- Using a “**chain of assignment**” is an easy way to set a group of variables to a common value

The ?

Operator

- Java includes a special ternary (*three-way*) operator that can replace certain types of if-then-else statements. This operator is the **?**.

expression1 ? expression2 : expression3

- Here, *expression1* can be any expression that evaluates to a boolean value. If *expression1* is true, then *expression2* is evaluated; otherwise, *expression3* is evaluated. The result of the ? operation is that of the expression evaluated. Both *expression2* and *expression3* are required to return the **same type**, which can't be void.

`k = i < 0 ? -i : i; // get absolute value of i`

Operator Precedence

Highest			
()	[]	.	
++	--	~	!
*	/	%	
+	-		
>>	>>>	<<	
>	>=	<	<=
==	!=		
&			
^			
&&			
?:			
=	op=		
Lowest			

Precedence	Operator	Type	Associativity
15	() [] .	Parentheses Array subscript Member selection	Left to Right
14	++ --	Unary post-increment Unary post-decrement	Right to left
13	++ -- + - ! ~ (<i>type</i>)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
12	* / %	Multiplication Division Modulus	Left to right
11	+ -	Addition Subtraction	Left to right
10	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right

9	<	Relational less than	Left to right
	<=	Relational less than or equal	
	>	Relational greater than	
	>=	Relational greater than or equal	
	instanceof	Type comparison (objects only)	
8	==	Relational is equal to	Left to right
	!=	Relational is not equal to	
7	&	Bitwise AND	Left to right
6	^	Bitwise exclusive OR	Left to right
5		Bitwise inclusive OR	Left to right
4	&&	Logical AND	Left to right
3		Logical OR	Left to right
2	? :	Ternary conditional	Right to left
1	=	Assignment	Right to left
	+=	Addition assignment	
	-=	Subtraction assignment	
	*=	Multiplication assignment	
	/=	Division assignment	
	%=	Modulus assignment	

Java's Selection Statements

- if
- if-else
- if-else-if ladder
- nested if
- switch

- Java's program control statements can be put into the following categories:

- Selection

- *Selection statements allow your program to choose different paths of execution based upon the outcome of an expression or the state of a variable.*

- Iteration

- *Iteration statements enable program execution to repeat one or more statements (that is, iteration statements form loops).*

- jump

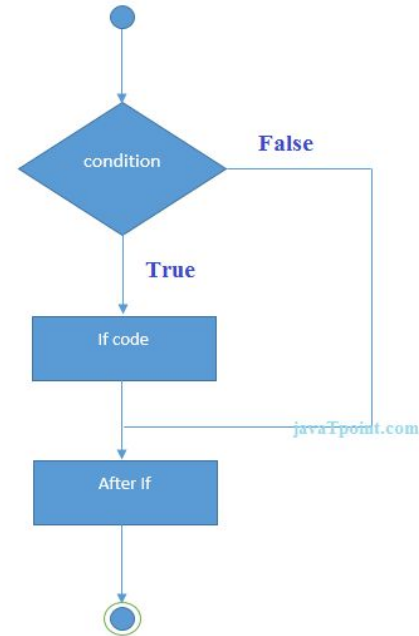
- *Jump statements allow your program to execute in a nonlinear fashion.*

Control Statements

- **The if Statement** : The Java if statement tests the condition.
- It executes the *if block* if condition is true.

```
if(condition) {  
    // statement;  
}
```

```
if(num < 100)  
    System.out.println("less than 100");
```

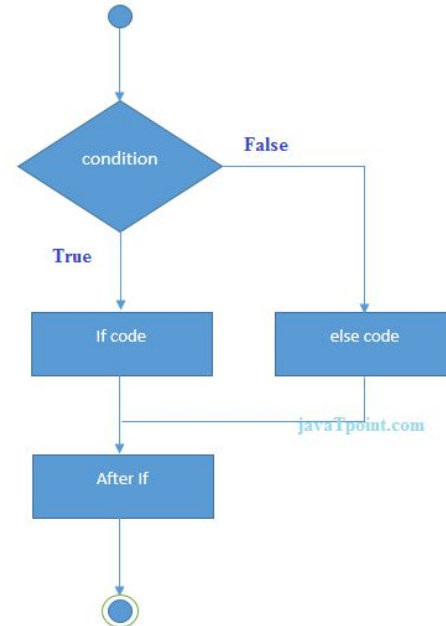


if - else

- The Java if-else statement also tests the condition.
- It executes the *if block* if condition is true otherwise *else block* is execute^d

Syntax:

```
if(condition){  
    //code if condition is true  
}  
else{  
    //code if condition is false  
}
```



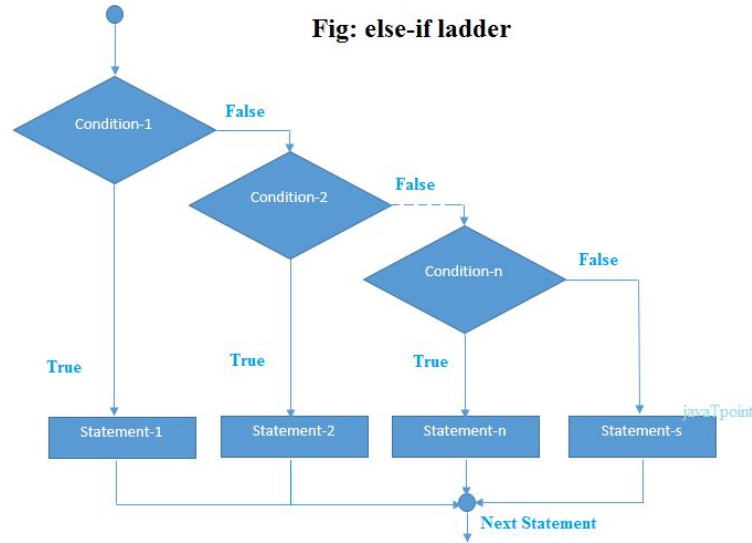
if-else-if ladder

- The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1){  
    //code to be executed if condition1 is true  
}  
else if(condition2){  
    //code to be executed if condition2 is true  
}  
else if(condition3){  
    //code to be executed if condition3 is true  
}  
...  
else{  
    //code to be executed if all the conditions are false  
}
```

Fig: else-if ladder

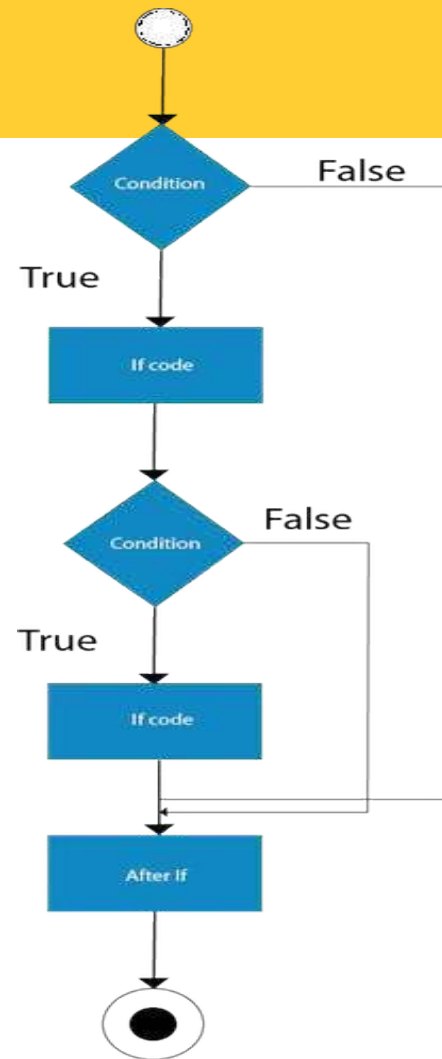


Nested if

- The nested if statement represents the *if block within another if block*.
- Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition){  
    //code to be executed  
    if(condition){  
        //code to be executed  
    }  
}
```



Switch

Statement

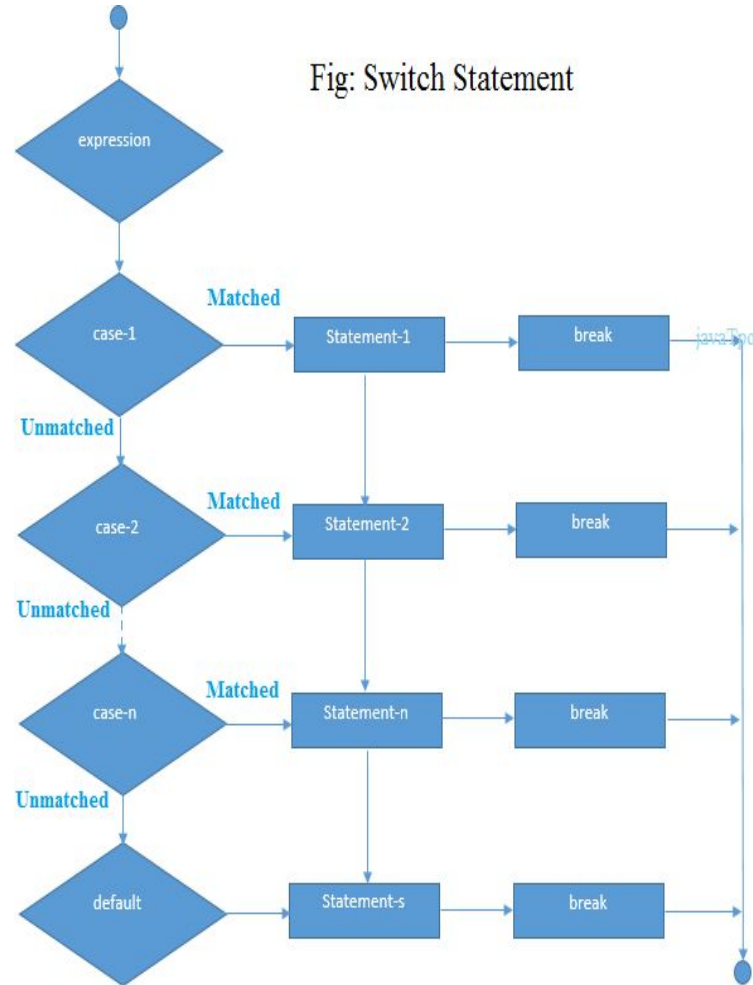
- The Java *switch statement* executes a statement from multiple conditions.
- It is like if-else-if ladder statement.
- The switch statement works with byte, short, int, long, enum types, String and some wrapper types like Byte, Short, Int, and Long.
- There can be *one or N number of case values* for a switch expression.
- The case value must be of switch expression type only. The case value must be *literal or constant*. It *doesn't allow variables*.
- The case values must be *unique*. In case of duplicate value, it renders compile-time error.
- Each case statement can have a *break statement* which is optional. When control reaches to the break statement, it jumps the control after the switch expression. If a break statement is *not found*, it executes the next case.
- The case value can have a *default label* which is optional.

```

switch(expression){
  case value1:
    //code to be executed;
    break; //optional
  case value2:
    //code to be executed;
    break; //optional
  .....
  default:
    code to be executed if all cases
    are not matched;
}

```

Fig: Switch Statement



Iteration Statements

- Java's iteration statements are for, while, and do-while.
- These statements are called *loops*.
- *A loop repeatedly executes the same set of* instructions until a termination condition is met.

while

- The while loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true.

```
while(condition) {  
    // body of loop  
}
```

- The *condition* can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true.
- When *condition* becomes false, control passes to the next line of code immediately following the loop.
- The curly braces are unnecessary if only a single statement is being repeated.

do-while

- The do-while loop always executes its body at least once, because its conditional expression is at the bottom of the loop. Its general form is

```
do {  
    // body of loop  
} while (condition);
```

- Each iteration of the do-while loop first executes the body of the loop and then evaluates the conditional expression. If this expression is true, the loop will repeat. Otherwise, the loop terminates.

for

- There are two forms of the for loop.
- The first is the traditional form of Java.

for(initialization;

condition; iteration) {

// body

- The second is the new “for-each” form. The for- each loop is essentially read-only.

for(type itr-var : collection) { statement-block

}

for Loop Variations

```
for(int a=1, b=4; a<b; a++, b--)  
{  
}
```

```
for(int i=1; !done; i++)  
{  
}
```

```
//infinite loop  
for( ; ; ) {  
    // ...  
}
```

```
// Parts of the for loop can be empty.  
class ForVar {  
    public static void main(String args[]) {  
        int i;  
        boolean done =  
            false; i = 0;  
        for( ; !done; ) {  
            System.out.println("i is " + i);  
            if(i == 10) done = true;  
            i++;  
        }  
    }  
}
```

Jump Statements

- Java supports **three** jump statements: **break**, **continue**, and **return**.
- These statements transfer control to another part of your program.
- *Java supports one other way that you can change your program's flow of execution: through **exception handling**. Exception handling provides a structured method by which run-time errors can be trapped and handled by your program. It is supported by the keywords **try, catch, throw, throws, and finally**.*

Using break

- In Java, the break statement has three uses.
- First, it terminates a statement sequence in a **switch** statement.
- Second, it can be used to exit a **loop**.
- Third, it can be used as a “civilized” form of **goto**. *break label;*

```
// Using break to exit a loop.
class BreakLoop {
    public static void main(String args[]) {
        for(int i=0; i<100; i++) {
            if(i == 10) break; // terminate loop if i is 10
            System.out.println("i: " + i);
        }
        System.out.println("Loop complete.");
    }
}
```

Using continue

- Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration.
- In **while** and **do-while loops**, a **continue** statement causes control to be transferred directly to the conditional expression that controls the loop.
- In a **for loop**, control goes first to the iteration portion of the for statement and then to the conditional expression.
- For all three loops, any intermediate code is bypassed.
- As with the break statement, continue may **specify a label** to describe which enclosing loop to continue.

return

- The return statement is used to explicitly return from a **method**.
- That is, it causes program control to transfer back to the caller of the method.
- The return statement immediately terminates the method in which it is executed.

Class Fundamentals

- A class, defines a new data type.
- Once defined, this new type can be used to create **objects** of that type.
- Thus, a class is a *template for an object*, and an object is an *instance of a class*.
- *Because an object is an instance of a class, the two words object and instance used interchangeably.*

The General Form of a Class

- The data, or variables, defined within a class are called *instance variables*.
- *The code is contained within methods.*
- *Collectively, the methods and variables defined within a class are called members of the class.*
- Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its **own copy** of these variables. Thus, the data for one object is separate and **unique** from the data for another.

A Simple Class

- a **class** declaration only creates a template;

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

```
Box mybox = new Box(); // create a Box object called  
mybox
```

mybox will be an instance of Box. Thus, it will have “physical” reality. To access class variables, use the *dot (.) operator*. *The dot operator links the name of the object with the name of an instance variable.*

```
mybox.width = 100;
```

```
/* A program that uses the Box class. Call this file BoxDemo.java */
class Box {
    double width;
    double height;
    double depth;
}
// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;
        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;
        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Declaring Objects

- Obtaining objects of a class is a two-step process.
- **First**, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can *refer to an object*.

Box mybox; // declare reference to object

- **Second**, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the **new operator**.

mybox = new Box(); // allocate a Box object

- The new operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new.
- This reference is then stored in the variable.

Statement

Box mybox;

Effect

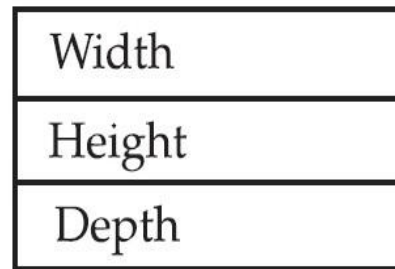
null

mybox

mybox = new Box();



mybox



Box object

What is the value range for "short" data type in Java?

- (a) -2^{15} to $+2^{15}$
- (b) -2^{16} to $+2^{16}$
- (c) -2^{16} to $+2^{16-1}$
- (d) -2^{16-1} to $+2^{16-1}$
- (e) -2^{15} to $+2^{15-1}$

CGPSC Asstt. Prof. 2014 (IT)

What is the value range for "short" data type in Java?

- (a) -2^{15} to $+2^{15}$
- (b) -2^{16} to $+2^{16}$
- (c) -2^{16} to $+2^{16-1}$
- (d) -2^{16-1} to $+2^{16-1}$
- (e) -2^{15} to $+2^{15-1}$

CGPSC Asstt. Prof. 2014 (IT)

Ans. (e) :

In Java, the "DataInputStream" class is defined in package.

- (a) java.lang
- (b) java.util
- (c) java.in
- (d) java.io
- (e) java.out

CGPSC Asstt. Prof. 2014 (IT)

In Java, the "DataInputStream" class is defined in package.

- (a) java.lang
- (b) java.util
- (c) java.in
- (d) java.io
- (e) java.out

CGPSC Asstt. Prof. 2014 (IT)

Ans. (d) :

Earlier name of Java programming language was:

- (a) OAK**
- (b) D**
- (c) Netbean**
- (d) Eclipse**

NIELIT Scientists-B 04.12.2016 (CS)

Earlier name of Java programming language was:

- (a) OAK**
- (b) D**
- (c) Netbean**
- (d) Eclipse**

NIELIT Scientists-B 04.12.2016 (CS)

Ans. (a) :

Java allows programmers to develop the following:

- (a) applets, applications and servlets**
- (b) applets and applications**
- (c) applets, applications, servlets, Java beans and distributed objects**
- (d) none of the above**

TNPSC 2016 (Degree) P-II

Java allows programmers to develop the following:

- (a) applets, applications and servlets**
- (b) applets and applications**
- (c) applets, applications, servlets, Java beans and distributed objects**
- (d) none of the above**

TNPSC 2016 (Degree) P-II

Ans. (a)

Chapter - 5

JavaBeans

INTRODUCTION TO JAVA BEANS

Software components are self-contained software units developed according to the motto “Developed them once, run and reused them everywhere”. Or in other words, reusability is the main concern behind the component model. A software component is a reusable object that can be plugged into any target software application. You can develop software components using various programming languages, such as C, C++, Java, and Visual Basic.

- A “Bean” is a reusable software component model based on sun’s java bean specification that can be manipulated visually in a builder tool.
- The term software component model describe how to create and use reusable software components to build an application
- Builder tool is nothing but an application development tool which lets you both to create new beans or use existing beans to create an application.
- To enrich the software systems by adopting component technology JAVA came up with the concept called Java Beans.
- Java provides the facility of creating some user defined components by means of Bean programming.
- We create simple components using java beans.
- We can directly embed these beans into the software.

Advantages of Java Beans:

- The java beans possess the property of “Write once and run anywhere”.
- Beans can work in different local platforms.
- Beans have the capability of capturing the events sent by other objects and vice versa enabling object communication.
- The properties, events and methods of the bean can be controlled by the application developer.(ex. Add new properties)
- Beans can be configured with the help of auxiliary software during design time.(no hassle at runtime)
- The configuration setting can be made persistent.(reused)
- Configuration setting of a bean can be saved in persistent storage and restored later.

- **Illustration of JavaBean Class**

- A simple example of JavaBean Class is mentioned below:

- `// Java program to illustrate the`
- `// structure of JavaBean class`
- `public class TestBean {`
- `private String name;`
-
- `public void setName (String name) {`
- `this.name = name;`
- `}`
-
- `public String getName () { return name; }`
- `}`
-
-

Setter and Getter Methods in Java

Setter and Getter Methods in Java properties are mentioned below:

Properties for setter methods:

1. It should be public in nature.
2. The return type **a** should be void.
3. The setter method should be prefixed with the set.
4. It should take some argument i.e. it should not be a no-arg method.

Properties for getter methods:

1. It should be public in nature.
2. The return type should not be void i.e. according to our requirement, **return type** we have to give the return type.
3. The getter method should be prefixed with get.
4. It should not take any argument.

```
public class Student implements java.io.Serializable {  
    private int id;  
    private String name;  
  
    // Constructor  
    public Student() {}  
  
    // Setter for Id  
    public void setId(int id) { this.id = id; }  
  
    // Getter for Id  
    public int getId() { return id; }  
  
    // Setter for Name  
    public void setName(String name) { this.name = name; }  
  
    // Getter for Name  
    public String getName() { return name; }  
}
```

What can we do/create by using JavaBean:

There is no restriction on the capability of a Bean.

- It may perform a simple function, such as checking the spelling of a document, or a complex function, such as forecasting the performance of a stock portfolio. A Bean may be visible to an end user. One example of this is a button on a graphical user interface.
- Software to generate a pie chart from a set of data points is an example of a Bean that can execute locally.
- Bean that provides real-time price information from a stock or commodities exchange.

Definition of a builder tool:

Builder tools allow a developer to work with JavaBeans in a convenient way. By examining a JavaBean by a process known as Introspection, a builder tool exposes the discovered features of the JavaBean for visual manipulation. A builder tool maintains a list of all JavaBeans available. It allows you to compose the Bean into applets, application, servlets and composite components (e.g. a JFrame), customize its behavior and appearance by modifying its properties and connect other components to the event of the Bean or vice versa.

JavaBeans basic rules

A JavaBean should:

- be public
- implement the Serializable interface
- have a no-arg constructor
- be derived from javax.swing.JComponent or java.awt.Component if it is visual

The classes and interfaces defined in the java.beans package enable you to create JavaBeans.

The Java Bean components can exist in one of the following three phases of development:

- Construction phase
- Build phase
- Execution phase

It supports the standard **component architecture** features of

- Properties
- Events
- Methods
- Persistence.

In addition Java Beans provides support for

- Introspection (Allows Automatic Analysis of a java beans)
- Customization (To make it easy to configure a java beans component)

Elements of a JavaBean:

- **Properties**

Similar to instance variables.

A bean property is a named attribute of a bean that can affect its behavior or appearance. Examples of bean properties include color, label, font, font size, and display size.

- **Methods**

- Same as normal Java methods.
- Every property should have accessor (get) and mutator (set) method.
- All Public methods can be identified by the introspection mechanism.
- There is no specific naming standard for these methods.

- **Events**

- Similar to Swing/AWT event handling.

The JavaBean Component Specification:

Customization: Is the ability of JavaBean to allow its properties to be changed in build and execution phase.

Persistence:- Is the ability of JavaBean to save its state to disk or storage device and restore the saved state when the JavaBean is reloaded.

Communication:- Is the ability of JavaBean to notify change in its properties to other JavaBeans or the container.

Introspection:- Is the ability of a JavaBean to allow an external application to query the properties, methods, and events supported by it.

Services of JavaBean Components

Builder support:- Enables you to create and group multiple JavaBeans in an application.

Layout:- Allows multiple JavaBeans to be arranged in a development environment.

Interface publishing: Enables multiple JavaBeans in an application to communicate with each other.

Event handling:- Refers to firing and handling of events associated with a JavaBean.

Persistence:- Enables you to save the last state of JavaBean.

Features of a JavaBean

- Support for “introspection” so that a builder tool can analyze how a bean works.
- Support for “customization” to allow the customisation of the appearance and behaviour of a bean.
- Support for “events” as a simple communication metaphor than can be used to connect up beans.
- Support for “properties”, both for customization and for programmatic use.
- Support for “persistence”, so that a bean can save and restore its customized state.

Beans Development Kit

Is a development environment to create, configure, and test JavaBeans.

The features of BDK environment are:

- Provides a GUI to create, configure, and test JavaBeans.
- Enables you to modify JavaBean properties and link multiple JavaBeans in an application using BDK.
- Provides a set of sample JavaBeans.
- Enables you to associate pre-defined events with sample JavaBeans.

Identifying BDK Components

- Execute the run.bat file of BDK to start the BDK development environment.
- The components of BDK development environment are:
 - ToolBox
 - BeanBox
 - Properties
 - Method Tracer

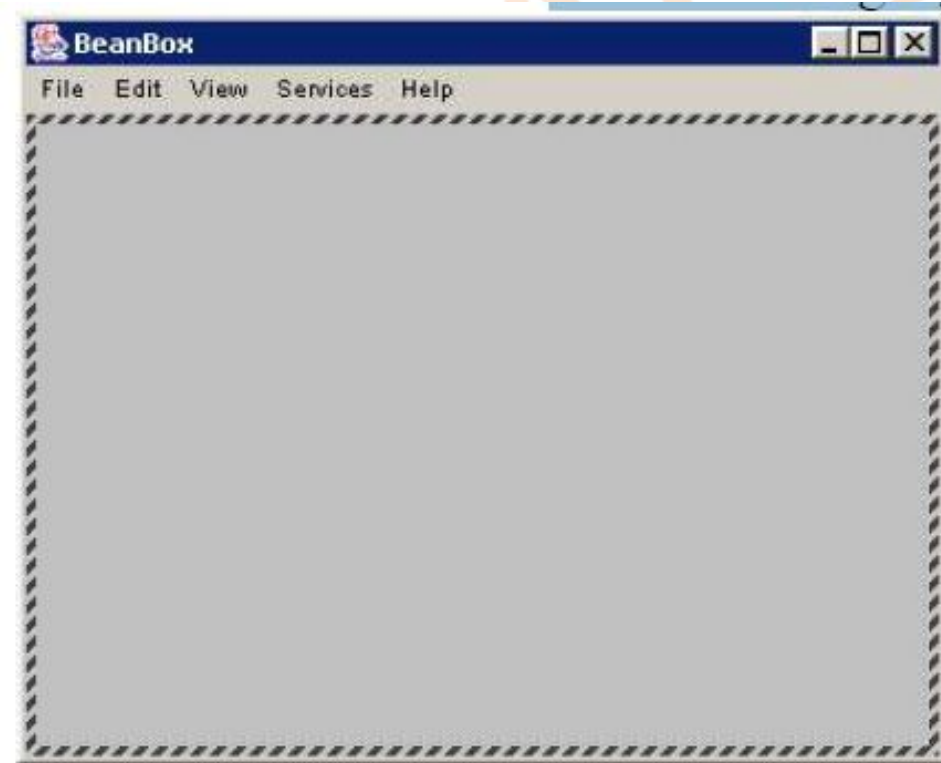
ToolBox window: Lists the sample JavaBeans of BDK.
The following figure shows the ToolBox window:



BeanBox window:

Is a workspace for creating the layout of Java Bean application.

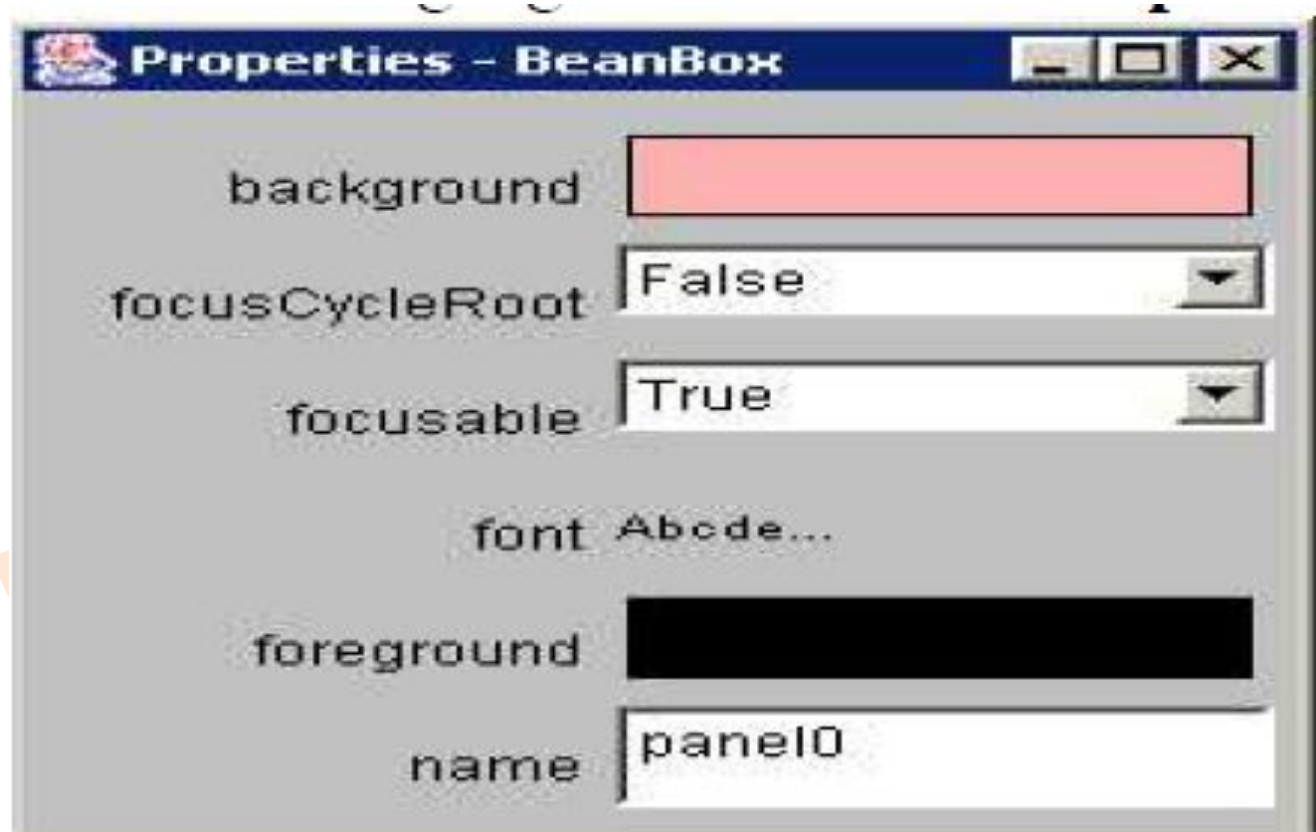
- The following figure shows the BeanBox window:



Properties window:

Displays all the exposed properties of a JavaBean. You can modify JavaBean properties in the properties window.

The following figure shows the **Properties** window:



Method Tracer window:

Displays the debugging messages and method calls for a JavaBean application.
The following figure shows the **Method Tracer** window:



Design patterns for JavaBean Properties:-

A property is a subset of a Bean's state.

A bean property is a named attribute of a bean that can affect its behavior or appearance.

Examples of bean properties include color, label, font, font size, and display size.

- Properties are the private data members of the JavaBean classes.
- Properties are used to accept input from an end user in order to customize a JavaBean.
- Properties can retrieve and specify the values of various attributes, which determine the behavior of a JavaBean.

Types of JavaBeans Properties

- Simple properties
- Boolean properties
- Indexed properties

Simple Properties:

Simple properties refer to the private variables of a JavaBean that can have only a single value.

Simple properties are retrieved and specified using the get and set methods respectively.

- A read/write property has both of these methods to access its values. The get method used to read the value of the property .The set method that sets the value of the property.
- The setXXX() and getXXX() methods are the heart of the java beans properties mechanism. This is also called getters and setters. These accessor methods are used to set the property .

Ex:

```
public double getDepth()  
{  
    return depth;  
}
```

Read only property has only a get method.

Ex:

```
public void setDepth(double d)  
{  
    Depth=d;  
}
```

Write only property has only a set method.

Boolean Properties:

- A Boolean property is a property which is used to represent the values True or False. Have either of the two values, TRUE or FALSE.

Ex:

```
public boolean dotted=false;
public boolean isDotted()
{
    return dotted;
}
public void setDotted(boolean dotted)
{
    this.dotted=dotted;
}
```

Indexed Properties:

Indexed Properties are consists of multiple values. If a simple property can hold an array of value they are no longer called simple but instead indexed properties. The method's signature has to be adapted accordingly. An indexed property may expose set/get methods to read/write one element in the array (so-called 'index getter/setter') and/or so-called 'array getter/setter' which read/write the entire array.

Indexed Properties enable you to set or retrieve the values from an array of property values.

Indexed Properties are retrieved using the following get methods:

Ex:

```
private double data[];  
public double getData(int index)  
{  
    return data[index];  
}
```

Ex:

```
public void setData(int  
index,double value)  
{  
    Data[index]=value;  
}
```

Bound Properties:

A bean that has a bound property generates an event when the property is changed.

Bound Properties are the properties of a JavaBean that inform its listeners about changes in its values.

- Bound Properties are implemented using the `PropertyChangeSupport` class and its methods.
- Bound Properties are always registered with an external event listener.
- The event is of type `PropertyChangeEvent` and is sent to objects that previously registered an interest in receiving such notifications

Constrained Properties:

It generates an event when an attempt is made to change its value .

Constrained Properties are implemented using the `PropertyChangeEvent` class.

The event is sent to objects that previously registered an interest in receiving such notification

Those other objects have the ability to veto the proposed change

This allows a bean to operate differently according to the runtime environment

Design Patterns for Events:

Handling Events in JavaBeans:

Enables Beans to communicate and connect together.

Beans generate events and these events can be sent to other objects.

Event means any activity that interrupts the current ongoing activity.

Example: mouse clicks, pressing key...

User-defined JavaBeans interact with the help of user-defined events, which are also called custom events.

You can use the Java event delegation model to handle these custom events.

The components of the event delegation model are:

- **Event Source:** Generates the event and informs all the event listeners that are registered with it.
- **Event Listener:** Receives the notification, when an event source generates an event.
- **Event Object:** Represents the various types of events that can be generated by the event sources.

Persistence

Persistence means an ability to save properties and events of our beans to non-volatile storage and retrieve later. It has the ability to save a bean to storage and retrieve it at a later time Configuration settings are saved It is implemented by Java serialization.

If a bean inherits directly or indirectly from Component class it is automatically Serializable.

Transient keyword can be used to designate data members of a Bean that should not be serialized.

- Enables developers to customize Beans in an application builder, and then retrieve those Beans, with customized features intact, for future use, perhaps in another environment.
- Java Beans supports two forms of persistence:
 - Automatic persistence
 - External persistence

Automatic Persistence:

Automatic persistence are java's built-in serialization mechanism to save and restore the state of a bean.

External Persistence:

External persistence, on the other hand, gives you the option of supplying your own custom classes to control precisely how a bean state is stored and retrieved.

- Juggle Bean.
- Building an applet
- Your own bean

Customizers:

The Properties window of the BDK allows a developer to modify the several properties of the Bean.

Property sheet may not be the best user interface for a complex component

It can provide step-by-step wizard guide to use component

It can provide a GUI frame with image which visually tells what is changed such as radio button, check box,

.

The Java Beans API:

The Java Beans functionality is provided by a set of classes and interfaces in the `java.beans` package. Table 25-2 lists the interfaces in `java.beans` and provides a brief description of their functionality. Table 25-3 lists the classes in `java.beans`.

Set of classes and interfaces in the `java.beans` package

Package `java.beans`

Contains classes related to developing beans -- components based on the JavaBeans architecture

Interface Summary	
<u>AppletInitializer</u>	This interface is designed to work in collusion with <code>java.beans.Beans.instantiate</code> .
<u>BeanInfo</u>	A bean implementor who wishes to provide explicit information about their bean may provide a <code>BeanInfo</code> class that implements this <code>BeanInfo</code> interface and provides explicit information about the methods, properties, events, etc, of their bean.
<u>Customizer</u>	A customizer class provides a complete custom GUI for customizing a target Java Bean.
<u>DesignMode</u>	This interface is intended to be implemented by, or delegated from, instances of <code>java.beans.beancontext.BeanContext</code> , in order to propagate to its nested hierarchy of <code>java.beans.beancontext.BeanContextChild</code> instances, the current "designTime" property.
<u>ExceptionListener</u>	An <code>ExceptionListener</code> is notified of internal exceptions.
<u>PropertyChangeListener</u>	A "PropertyChange" event gets fired whenever a bean changes a "bound" property.
<u>PropertyEditor</u>	A <code>PropertyEditor</code> class provides support for GUIs that want to allow users to edit a property value of a given type.
<u>VetoableChangeListener</u>	A <code>VetoableChange</code> event gets fired whenever a bean changes a "constrained" property.
<u>Visibility</u>	Under some circumstances a bean may be run on servers where a GUI is not available.

Class Summary	
<u>BeanDescriptor</u>	A BeanDescriptor provides global information about a "bean", including its Java class, its displayName, etc.
<u>Beans</u>	This class provides some general purpose beans control methods.
<u>DefaultPersistenceDelegate</u>	The DefaultPersistenceDelegate is a concrete implementation of the abstract PersistenceDelegate class and is the delegate used by default for classes about which no information is available.
<u>Encoder</u>	An Encoder is a class which can be used to create files or streams that encode the state of a collection of JavaBeans in terms of their public APIs.
<u>EventHandler</u>	The EventHandler class provides support for dynamically generating event listeners whose methods execute a simple statement involving an incoming event object and a target object.
<u>EventSetDescriptor</u>	An EventSetDescriptor describes a group of events that a given Java bean fires.
<u>Expression</u>	An Expression object represents a primitive expression in which a single method is applied to a target and a set of arguments to return a result - as in "a.getFoo()".

FeatureDescriptor	The FeatureDescriptor class is the common baseclass for PropertyDescriptor, EventSetDescriptor, and MethodDescriptor, etc.
IndexedPropertyChangeEvent	An "IndexedPropertyChange" event gets delivered whenever a component that conforms to the JavaBeans specification (a "bean") changes a bound indexed property.
IndexedPropertyDescriptor	An IndexedPropertyDescriptor describes a property that acts like an array and has an indexed read and/or indexed write method to access specific elements of the array.
Introspector	The Introspector class provides a standard way for tools to learn about the properties, events, and methods supported by a target Java Bean.
MethodDescriptor	A MethodDescriptor describes a particular method that a Java Bean supports for external access from other components.
ParameterDescriptor	The ParameterDescriptor class allows bean implementors to provide additional information on each of their parameters, beyond the low level type information provided by the java.lang.reflect.Method class.
PersistenceDelegate	The PersistenceDelegate class takes the responsibility for expressing the state of an instance of a given class in terms of the methods in the class's public API.
PropertyChangeEvent	A "PropertyChange" event gets delivered whenever a bean changes a "bound" or "constrained" property.
PropertyChangeListenerProxy	A class which extends the EventListenerProxy specifically for adding a named PropertyChangeListener.
PropertyChangeSupport	This is a utility class that can be used by beans that support bound properties.
PropertyDescriptor	A PropertyDescriptor describes one property that a Java Bean exports via a pair of accessor methods.
PropertyEditorManager	The PropertyEditorManager can be used to locate a property editor for any given type name.
PropertyEditorSupport	This is a support class to help build property editors.
SimpleBeanInfo	This is a support class to make it easier for people to provide BeanInfo classes.
Statement	A Statement object represents a primitive statement in which a single method is applied to a target and a set of arguments - as in "a.setFoo(b)".
VetoableChangeListenerProxy	A class which extends the EventListenerProxy specifically for associating a VetoableChangeListener with a "constrained" property.
VetoableChangeSupport	This is a utility class that can be used by beans that support constrained properties.



The BeanInfo Interface :

By default an Introspector uses the Reflection API to determine the features of a JavaBean. However, a JavaBean can provide its own BeanInfo which will be used instead by the Introspector to determine the discussed information. This allows a developer hiding specific properties, events and methods from a builder tool or from any other tool which uses the Introspector class. Moreover it allows supplying further details about events/properties/methods as you are in charge of creating the descriptor objects.

The following table lists some of the methods of BeanInfo interface:

Method	Description
<code>MethodDescriptor[] getMethodDescriptors()</code>	Returns an array of the method descriptor objects of a <code>JavaBean</code> . The method descriptor objects are used to determine information about the various methods defined in a <code>JavaBean</code> .
<code>EventDescriptor[] getEventDescriptors()</code>	Returns an array of the event descriptor objects of a <code>JavaBean</code> . The event descriptor objects determine the information about the events associated with a <code>JavaBean</code> .
Method	Description
<code>PropertyDescriptor[] getPropertyDescriptors()</code>	Returns an array of the property descriptor objects of a <code>JavaBean</code> . The property descriptor objects are used to determine information about the various custom properties of a <code>JavaBean</code> .
<code>Image getIcon(int icon_type)</code>	Returns a corresponding <code>Image</code> object for one of the fields of the <code>BeanInfo</code> interface passed as a parameter to this method. The <code>BeanInfo</code> interface defines <code>int</code> fields, such as <code>ICON_COLOR_32x32</code> and <code>ICON_MONO_32x32</code> to represent icons.

The BeanBox Menus

This section explains each item in the BeanBox File, Edit, and View menus.

File Menu

Save

Saves the Beans in the BeanBox, including each Bean's size, position, and internal state. The saved file can be loaded via File | Load.

SerializeComponent...

Saves the Beans in the BeanBox to a serialized (.ser) file. This file must be put in a .jar file to be useable.

MakeApplet...

Generates an applet from the BeanBox contents.

Load...

Loads Saved files into the BeanBox. Will not load .ser files.

LoadJar...

Loads a Jar file's contents into the ToolBox.

Print

Prints the BeanBox contents.

Clear

Removes the BeanBox contents.

Exit

Quits the BeanBox without offering to save.

Edit Menu

Cut

Removes the Bean selected in the BeanBox. The cut Bean is serialized, and can then be pasted.

Copy

Copies the Bean selected in the BeanBox. The copied Bean is serialized, and can then be pasted.

Paste

Drops the last cut or copied Bean into the BeanBox.

Report...

Generates an introspection report on the selected Bean.

Events

Lists the event-firing methods, grouped by interface.

Bind property...

Lists all the bound property methods of the selected Bean.

View Menu

Disable Design Mode

Removes the ToolBox and the Properties sheet from the screen. Eliminates all 14 beanBox design and test behavior (selected Bean, etc.), and makes the BeanBox behave like an application.

Hide Invisible Beans

Hides invisible Beans

A JavaBean is best described as a:

- (a) Java applet
- (b) Reusable software component
- (c) Java servlet
- (d) GUI library

A JavaBean is best described as a:

- (a) Java applet
- (b) Reusable software component
- (c) Java servlet
- (d) GUI library

Answer: (b)

Explanation: A JavaBean is a reusable, self-contained software component written in Java that can be manipulated visually in development tools. It encapsulates data and behavior in a single class.

2. Which of the following is true about JavaBeans?

- (a) They must have a public constructor with arguments
- (b) They must be abstract
- (c) They must have a no-argument constructor
- (d) They must extend Applet

2. Which of the following is true about JavaBeans?

- (a) They must have a public constructor with arguments
- (b) They must be abstract
- (c) They must have a no-argument constructor
- (d) They must extend Applet

Answer: (c)

Explanation: Every JavaBean must have a public no-argument constructor to allow easy instantiation within visual development environments.

3. A JavaBean class must implement which interface to allow object persistence?

- (a) Runnable
- (b) Serializable
- (c) Cloneable
- (d) Remote

3. A JavaBean class must implement which interface to allow object persistence?

- (a) Runnable
- (b) Serializable
- (c) Cloneable
- (d) Remote

Answer: (b)

Explanation: To enable storage and retrieval of bean state, the class must implement `java.io.Serializable`.

4. Which of the following methods define JavaBean properties?

- (a) set and get methods
- (b) main method
- (c) init and start methods
- (d) run and stop methods

4. Which of the following methods define JavaBean properties?

- (a) set and get methods
- (b) main method
- (c) init and start methods
- (d) run and stop methods

Answer: (a)

Explanation: Properties in JavaBeans are accessed using getter and setter methods that follow specific naming conventions.

In a JavaBean, properties represent:

- (a) Variables that hold state
- (b) Methods only
- (c) Event listeners
- (d) Package names

In a JavaBean, properties represent:

- (a) Variables that hold state
- (b) Methods only
- (c) Event listeners
- (d) Package names

Answer: (a)

Explanation: Properties represent the data fields (state) of the bean, accessed through corresponding getter and setter methods.

What is the purpose of introspection in JavaBeans?

- (a) To analyze bean methods and properties at runtime
- (b) To encrypt data inside beans
- (c) To initialize constructors
- (d) To handle exceptions

What is the purpose of introspection in JavaBeans?

- (a) To analyze bean methods and properties at runtime
- (b) To encrypt data inside beans
- (c) To initialize constructors
- (d) To handle exceptions

Answer: (a)

Explanation: Introspection allows development tools to discover a bean's properties, events, and methods by examining its class structure.

Which of the following tools use JavaBeans for component-based development?

- (a) NetBeans
- (b) Eclipse
- (c) JBuilder
- (d) All of these

Which of the following tools use JavaBeans for component-based development?

- (a) NetBeans
- (b) Eclipse
- (c) JBuilder
- (d) All of these

Answer: (d)

Explanation: All major Java IDEs use JavaBeans architecture to represent reusable components that can be dragged and dropped into applications.

Which package provides the JavaBeans classes?

- (a) java.applet
- (b) java.beans
- (c) java.io
- (d) java.util

Which package provides the JavaBeans classes?

- (a) java.applet
- (b) java.beans
- (c) java.io
- (d) java.util

Answer: (b)

Explanation: The `java.beans` package contains classes that support the JavaBeans component model, such as `PropertyDescriptor` and `Introspector`.

JavaBeans help achieve which of the following software principles?

- (a) Reusability and Modularity
- (b) Inheritance and Polymorphism
- (c) Garbage collection
- (d) Code obfuscation

JavaBeans help achieve which of the following software principles?

- (a) Reusability and Modularity
- (b) Inheritance and Polymorphism
- (c) Garbage collection
- (d) Code obfuscation

Answer: (a)

Explanation: The main goal of JavaBeans is to encourage reusable, modular design through encapsulated software components.

Which of the following is not a valid property type in a JavaBean?

- (a) Simple property
- (b) Indexed property
- (c) Bound property
- (d) Private property

Which of the following is not a valid property type in a JavaBean?

- (a) Simple property
- (b) Indexed property
- (c) Bound property
- (d) Private property

Answer: (d)

Explanation: "Private property" is not a recognized JavaBeans term. Valid property types are simple, indexed, bound, and constrained properties.

A bound property in JavaBeans means:

- (a) Property value never changes
- (b) Change in property value triggers event notification
- (c) It is read-only
- (d) It is transient

A bound property in JavaBeans means:

- (a) Property value never changes
- (b) Change in property value triggers event notification
- (c) It is read-only
- (d) It is transient

Answer: (b)

Explanation: A bound property fires a `PropertyChangeEvent` whenever its value changes, allowing other objects to respond to these updates.

Which class provides event-handling support in JavaBeans?

- (a) EventObject
- (b) PropertyChangeEvent
- (c) PropertyChangeListener
- (d) All of these

Which class provides event-handling support in JavaBeans?

- (a) EventObject
- (b) PropertyChangeEvent
- (c) PropertyChangeListener
- (d) All of these

Answer: (d)

Explanation: JavaBeans use the `java.beans` event classes (`EventObject`, `PropertyChangeEvent`, and `PropertyChangeListener`) to handle property change events.

JavaBeans are used in which type of applications?

- (a) Component-based software development
- (b) Operating systems
- (c) Machine learning frameworks
- (d) Database drivers only

JavaBeans are used in which type of applications?

- (a) Component-based software development
- (b) Operating systems
- (c) Machine learning frameworks
- (d) Database drivers only

Answer: (a)

Explanation: JavaBeans were designed for component-based application development, promoting reuse of GUI and business components.

A bean that notifies other objects when its properties change is called a:

- (a) Bound bean
- (b) Constrained bean
- (c) Stateless bean
- (d) Synchronized bean

A bean that notifies other objects when its properties change is called a:

- (a) Bound bean
- (b) Constrained bean
- (c) Stateless bean
- (d) Synchronized bean

Answer: (a)

Explanation: A bound bean uses property change listeners to notify other objects when one of its properties has changed.

What is the main advantage of using JavaBeans?

- (a) They reduce compilation time
- (b) They support hardware-level programming
- (c) They allow reusable, portable, and visual programming components
- (d) They make Java code platform-dependent

What is the main advantage of using JavaBeans?

- (a) They reduce compilation time
- (b) They support hardware-level programming
- (c) They allow reusable, portable, and visual programming components
- (d) They make Java code platform-dependent

Answer: (c)

Explanation: JavaBeans are designed for portability and reusability across different development environments, allowing visual programming and code reuse.

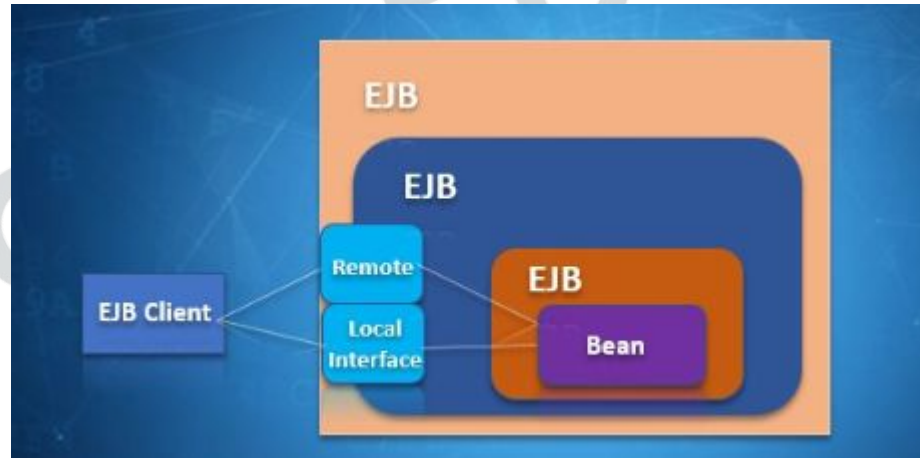
Chapter - 6

Enterprise Java beans (EJB)

What is EJB?

Enterprise Java Beans (EJB) is one of the several Java APIs for standard manufacture of enterprise software. EJB is a server-side software element that summarizes business logic of an application. Enterprise Java Beans web repository yields a runtime domain for web related software elements including computer reliability, Java Servlet Lifecycle (JSL) management, transaction procedure and other web services. The EJB enumeration is a subset of the Java EE enumeration.

The EJB enumeration was originally developed by IBM in 1997 and later adopted by Sun Microsystems in 1999 and enhanced under the Java Community Process.



EJB stands for:

- (a) Enterprise Java Binary
- (b) Enterprise JavaBean
- (c) Embedded Java Bean
- (d) Extended Java Base

EJB stands for:

- (a) Enterprise Java Binary
- (b) Enterprise JavaBean
- (c) Embedded Java Bean
- (d) Extended Java Base

Answer: (b)

Explanation: EJB stands for **Enterprise JavaBeans**, a server-side component architecture for enterprise-level applications in Java.

EJBs are mainly used for developing:

- (a) Standalone applications
- (b) GUI applications
- (c) Web-based and distributed enterprise applications
- (d) System software

EJBs are mainly used for developing:

- (a) Standalone applications
- (b) GUI applications
- (c) Web-based and distributed enterprise applications
- (d) System software

Answer: (c)

Explanation: EJBs simplify the development of **distributed, transactional, and secure server-side applications.**

The EJB component model is part of which Java technology?

- (a) J2SE
- (b) J2ME
- (c) J2EE
- (d) JavaFX

The EJB component model is part of which Java technology?

- (a) J2SE
- (b) J2ME
- (c) J2EE
- (d) JavaFX

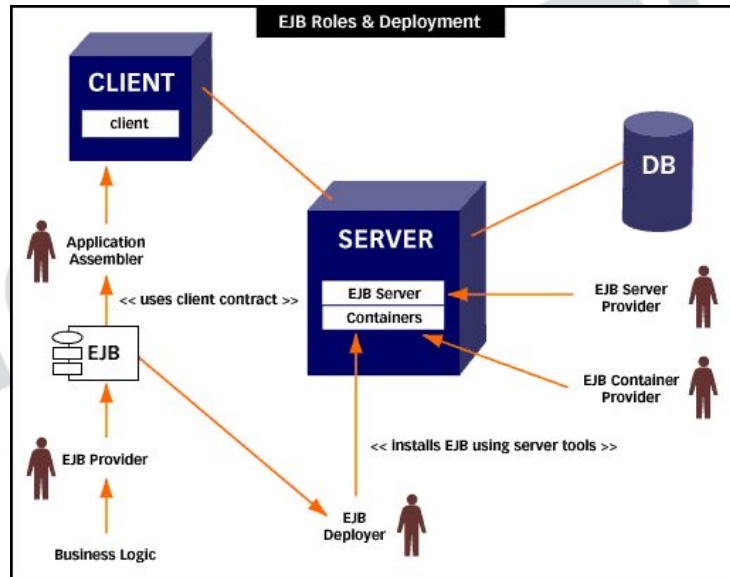
Answer: (c)

Explanation: EJB is a core component of Java 2 Enterprise Edition (J2EE), now known as Jakarta EE.

GOALS OF JAVA ENTERPRISE BEANS

The EJB Specifications try to meet several goals which are described as following:

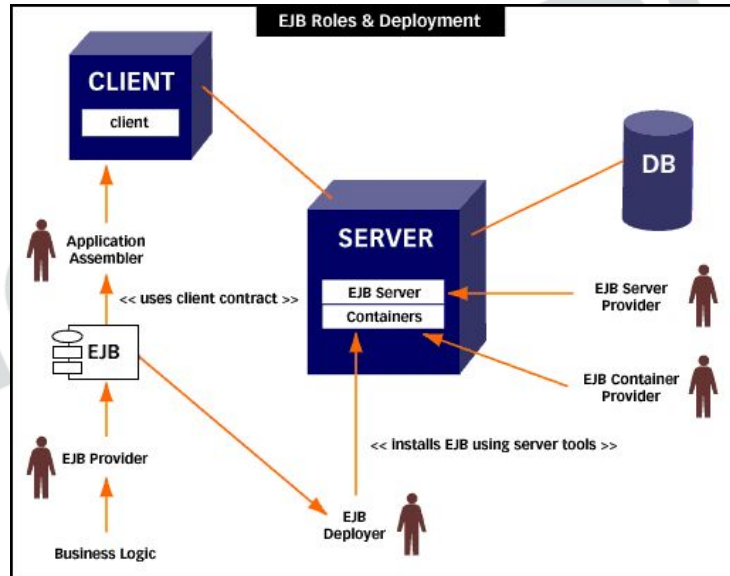
- EJB is designed to make it easy for developers to create applications, freeing them from low-level system details of managing transactions, threads, load balancing, and so on. Application developers can concentrate on business logic and leave the details of managing the data processing to the framework. For specialised applications, though, it's to customise these lower-level services.



GOALS OF JAVA ENTERPRISE BEANS

EJB aims to be the standard way for client/server applications to be built in the Java language. Just as the original JavaBeans (or Delphi component etc.) from different vendors can be combined to produce a custom client, EJB server components from different vendors can be combined to produce a custom server. EJB components, being Java classes, will of course run in any EJB-compliant server without recompilation. This is a benefit that platform-specific solutions cannot offer.

- Finally, the EJB is compatible with and uses other Java APIs, can interoperate with non-Java applications, and is compatible with CORBA.



The primary goal of Enterprise JavaBeans (EJB) specification is to:

- (a) Make developers manage low-level system services manually
- (b) Simplify development by handling low-level system details automatically
- (c) Restrict application portability
- (d) Eliminate the need for Java Virtual Machine

The primary goal of Enterprise JavaBeans (EJB) specification is to:

- (a) Make developers manage low-level system services manually
- (b) Simplify development by handling low-level system details automatically
- (c) Restrict application portability
- (d) Eliminate the need for Java Virtual Machine

Answer: (b)

Explanation:

EJB aims to **simplify enterprise application development** by managing complex system services (transactions, threading, load balancing) automatically, allowing developers to focus on **business logic**.

EJB helps developers by:

- (a) Allowing them to focus on business logic instead of system-level programming
- (b) Replacing Java with a new programming language
- (c) Removing the need for object-oriented programming
- (d) Providing GUI design tools

EJB helps developers by:

- (a) Allowing them to focus on business logic instead of system-level programming
- (b) Replacing Java with a new programming language
- (c) Removing the need for object-oriented programming
- (d) Providing GUI design tools

Answer: (a)

Explanation:

EJB frees programmers from writing system-level code like transaction and thread management so they can **concentrate on business functionality**.

According to EJB specifications, EJB components:

- (a) Are platform-specific and require recompilation
- (b) Can run on any EJB-compliant server without recompilation
- (c) Are limited to a specific vendor's application server
- (d) Cannot interoperate with other Java APIs

According to EJB specifications, EJB components:

- (a) Are platform-specific and require recompilation
- (b) Can run on any EJB-compliant server without recompilation
- (c) Are limited to a specific vendor's application server
- (d) Cannot interoperate with other Java APIs

Answer: (b)

Explanation:

EJB components are **platform-independent**; they can run on **any EJB-compliant application server** without recompilation, ensuring portability and reusability.

EJB components can interoperate with:

- (a) Only Java applications
- (b) CORBA and non-Java applications
- (c) Only local databases
- (d) Only applets

.

EJB components can interoperate with:

- (a) Only Java applications
- (b) CORBA and non-Java applications
- (c) Only local databases
- (d) Only applets

Answer: (b)

Explanation:

EJB is **compatible with CORBA** and can **interoperate with non-Java systems**, making it suitable for heterogeneous enterprise environments.

EJB components can interoperate with:

- (a) Only Java applications
- (b) CORBA and non-Java applications
- (c) Only local databases
- (d) Only applets

Answer: (b)

Explanation:

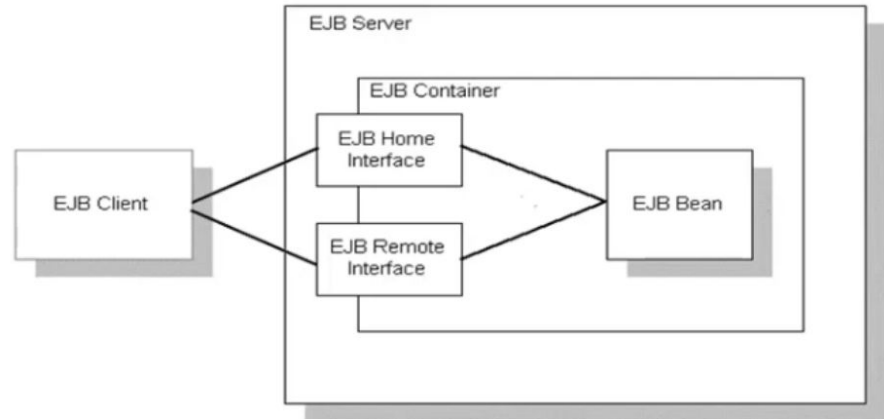
EJB is **compatible with CORBA** and can **interoperate with non-Java systems**, making it suitable for heterogeneous enterprise environments.

ARCHITECTURE OF AN EJB/ CLIENT SYSTEM

The architecture of Enterprise JavaBeans (EJB) is designed to facilitate the development and deployment of distributed, transactional, and secure enterprise applications using Java. It operates on a container-based model, where the EJB container provides crucial system-level services to the EJB components, allowing developers to focus on business logic.

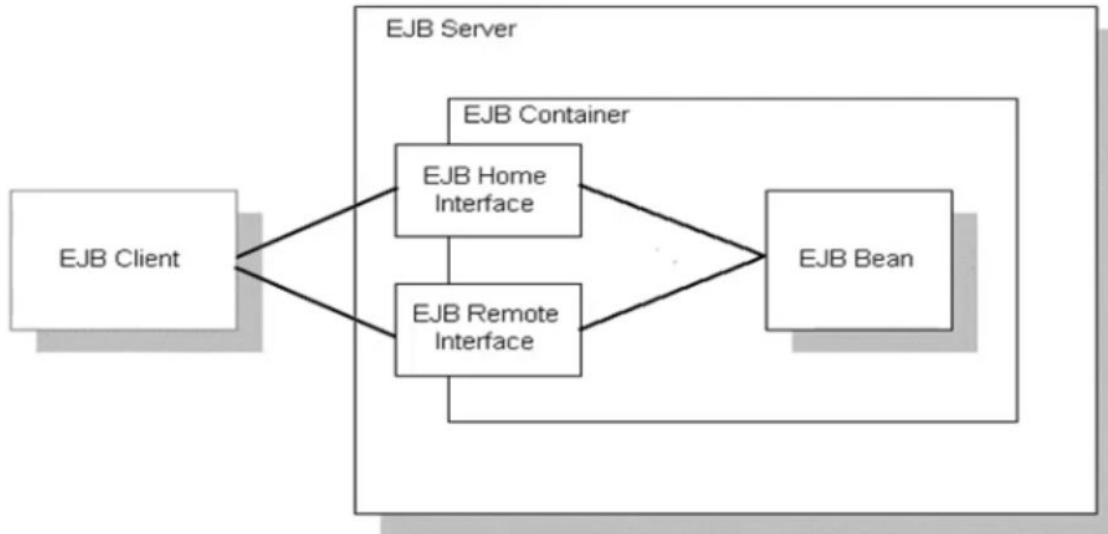
The main components of the EJB architecture are:

EJB Server: This is the high-level process or application that provides the runtime environment for EJB applications. It manages the EJB container and offers services such as process and thread management, system resource management, and potentially database connection pooling and caching.



ARCHITECTURE OF AN EJB/ CLIENT SYSTEM

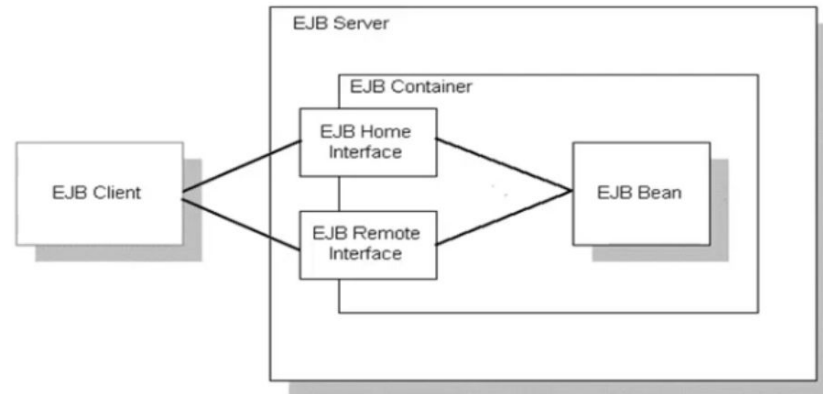
EJB Container: Residing within the EJB server, the container is a runtime environment that manages the lifecycle and execution of Enterprise Beans. It provides essential services like transaction management, security, concurrency control, resource pooling, and naming services (typically through JNDI).



ARCHITECTURE OF AN EJB/ CLIENT SYSTEM

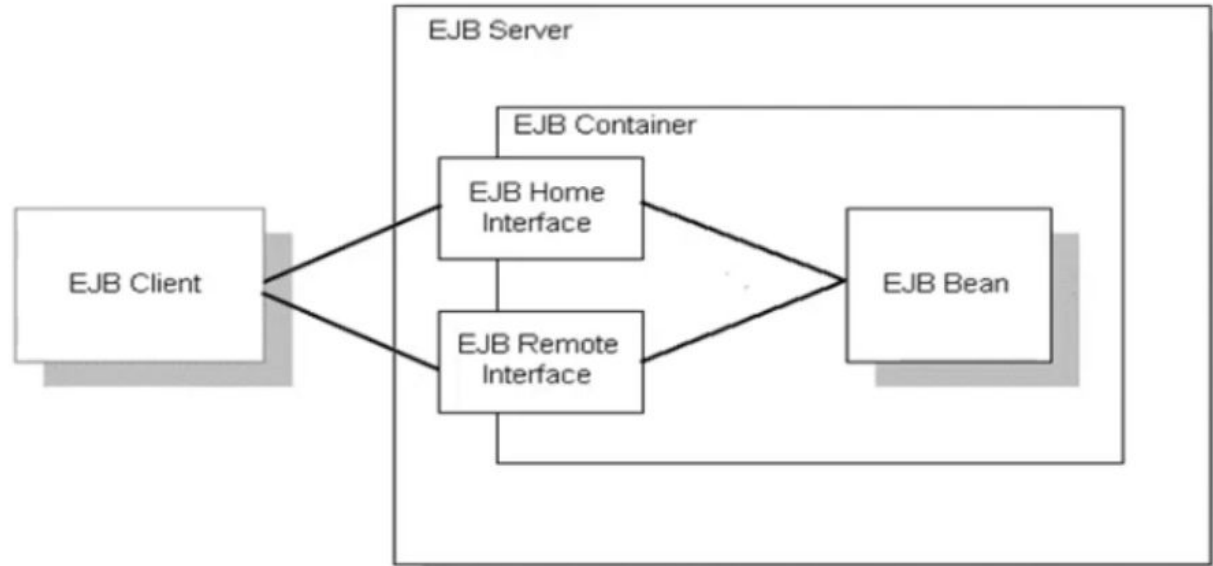
Enterprise Beans (EJBs): These are the server-side components encapsulating the business logic of the application. There are three main types:

- **Session Beans:** Represent a single client's interaction with the EJB container. They can be either stateful (maintaining conversational state with a specific client) or stateless (not maintaining client-specific state between method invocations).
- **Message-Driven Beans (MDBs):** Enable asynchronous processing of messages, typically from a Java Message Service (JMS) queue or topic. They are stateless and handle messages independently.
- **Entity Beans:** (Deprecated in EJB 3.x and largely replaced by the Java Persistence API - JPA). These represented persistent data stored in a database and provided object-relational mapping capabilities.



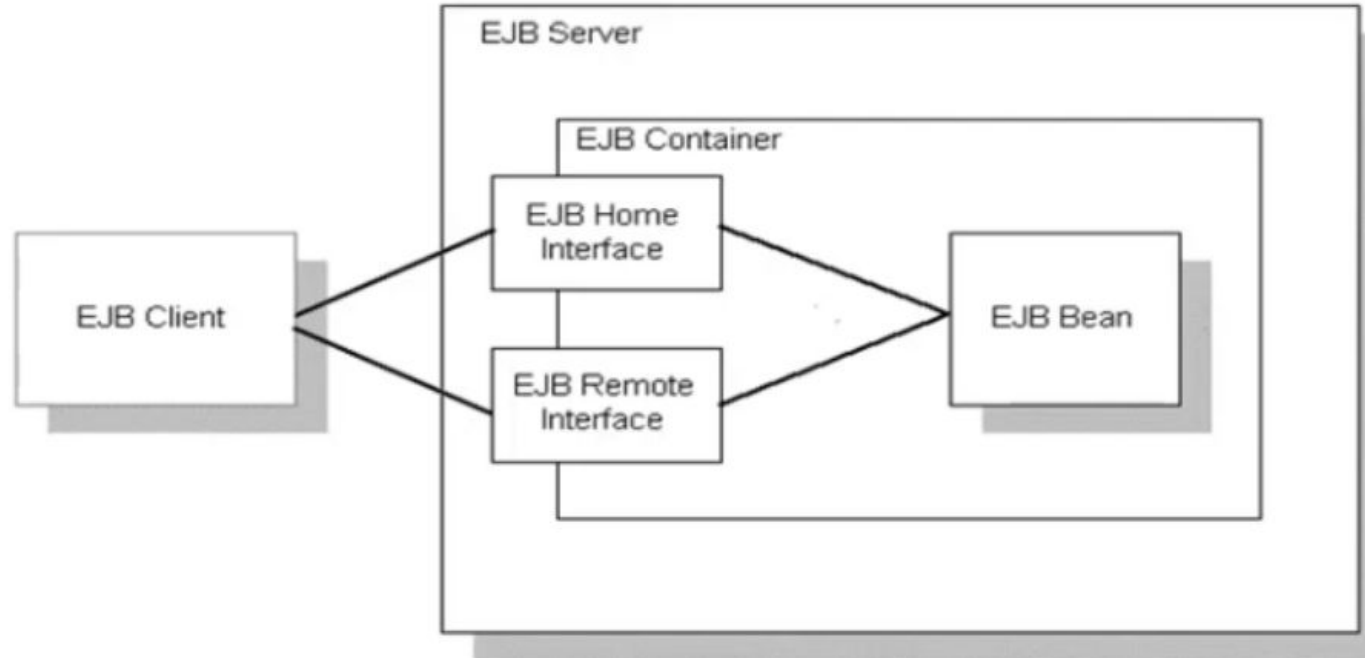
ARCHITECTURE OF AN EJB/ CLIENT SYSTEM

- **EJB Client:** This is any application or component that interacts with Enterprise Beans. It locates beans, typically using JNDI, and invokes their business methods through their remote or local interfaces.
- **Deployment Descriptor:** An XML file packaged with the Enterprise Beans (in an EJB JAR file or an EAR file), containing metadata about the beans, their configuration, transaction attributes, and security roles, which the EJB container uses during deployment and runtime.



ARCHITECTURE OF AN EJB/ CLIENT SYSTEM

- **Remote/Local Interfaces:** Define the business methods that clients can invoke on the Enterprise Beans. Remote interfaces allow clients in different JVMs or on different machines to access the bean, while local interfaces are for clients within the same JVM as the bean.



The EJB architecture is designed to:

- (a) Be vendor-specific and tightly coupled
- (b) Serve as a standard method for building Java-based client/server applications
- (c) Replace CORBA completely
- (d) Focus only on GUI development

The EJB architecture is designed to:

- (a) Be vendor-specific and tightly coupled
- (b) Serve as a standard method for building Java-based client/server applications
- (c) Replace CORBA completely
- (d) Focus only on GUI development

Answer: (b)

Explanation:

EJB provides a **standard, component-based framework** for building **Java-based distributed client/server enterprise applications**, allowing interoperability among components from different vendors.

The EJB container is responsible for:

- (a) Managing bean lifecycle, security, and transactions
- (b) Handling GUI rendering
- (c) Managing client-side resources only
- (d) Generating bytecode for JavaBeans

The EJB container is responsible for:

- (a) Managing bean lifecycle, security, and transactions
- (b) Handling GUI rendering
- (c) Managing client-side resources only
- (d) Generating bytecode for JavaBeans

Answer: (a)

Explanation: The container provides **system-level services** like transactions, security, concurrency, and lifecycle management.

The Remote Interface in EJB is used to:

- (a) Declare lifecycle methods
- (b) Define methods accessible to clients
- (c) Configure deployment settings
- (d) Handle XML data

The Remote Interface in EJB is used to:

- (a) Declare lifecycle methods
- (b) Define methods accessible to clients
- (c) Configure deployment settings
- (d) Handle XML data

Answer: (b)

Explanation: The **Remote Interface** declares business methods that can be called by clients remotely.

The Home Interface in EJB is used to:

- (a) Handle bean persistence
- (b) Provide lifecycle methods like create and remove
- (c) Define business logic
- (d) Manage events

The Home Interface in EJB is used to:

- (a) Handle bean persistence
- (b) Provide lifecycle methods like create and remove
- (c) Define business logic
- (d) Manage events

Answer: (b)

Explanation: The **Home Interface** defines lifecycle methods for creating, finding, and removing EJB objects.

The Deployment Descriptor in EJB is written in:

- (a) HTML
- (b) XML
- (c) JSON
- (d) YAML

The Deployment Descriptor in EJB is written in:

- (a) HTML
- (b) XML
- (c) JSON
- (d) YAML

Answer: (b)

Explanation: Deployment descriptors are **XML files** (`ejb-jar.xml`) that specify configuration details such as transactions and security.

The container in EJB is responsible for:

- (a) Only compiling Java code
- (b) Providing runtime environment and services for beans
- (c) Performing manual transaction handling
- (d) Generating JAR files

The container in EJB is responsible for:

- (a) Only compiling Java code
- (b) Providing runtime environment and services for beans
- (c) Performing manual transaction handling
- (d) Generating JAR files

Answer: (b)

Explanation: The EJB container provides runtime services and handles system-level concerns automatically.

Which of the following is not managed by the EJB container?

- (a) Transaction management
- (b) Memory allocation
- (c) Security
- (d) Object pooling

Which of the following is not managed by the EJB container?

- (a) Transaction management
- (b) Memory allocation
- (c) Security
- (d) Object pooling

Answer: (b)

Explanation: Memory management is handled by the **Java Virtual Machine (JVM)**, not the EJB container.

Types of Enterprise Java Beans

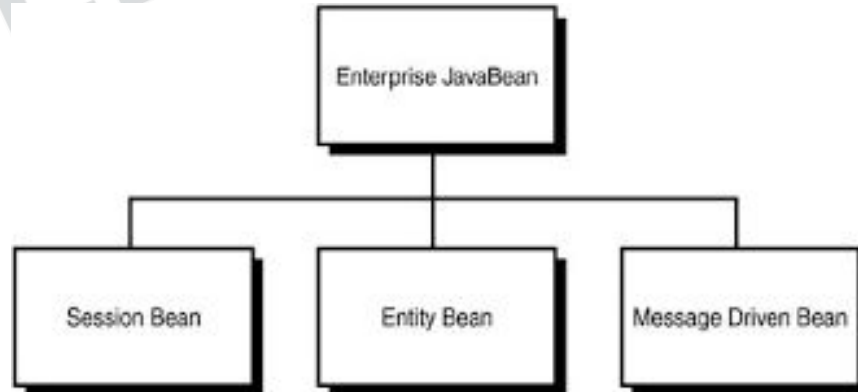
There are **three** types of EJB:

1. Session Bean: Session bean contains business logic that can be invoked by local, remote or web service client.

There are two types of session beans: (i) Stateful session bean and (ii) Stateless session bean.

(i) Stateful Session bean :

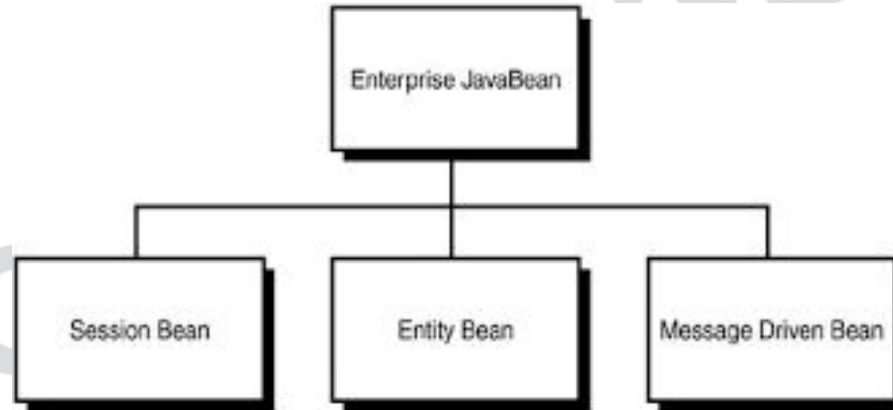
Stateful session bean performs business task with the help of a state. Stateful session bean can be used to access various method calls by storing the information in an instance variable. Some of the applications require information to be stored across separate method calls. In a shopping site, the items chosen by a customer must be stored as data is an example of stateful session bean.



Types of Enterprise Java Beans

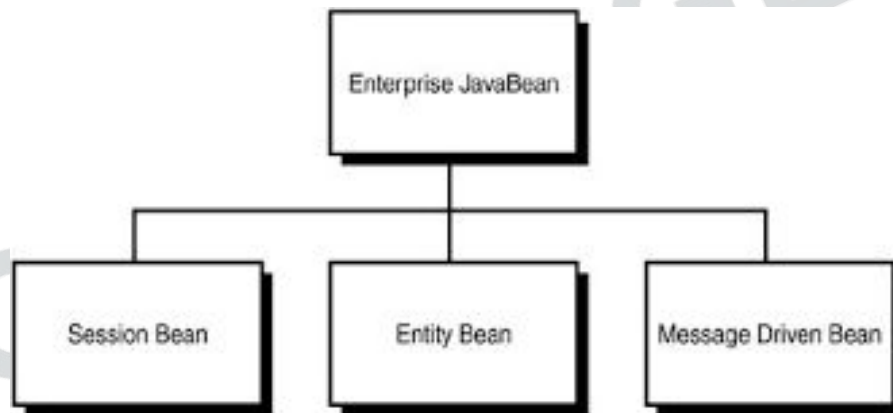
(ii) Stateless Session bean :

Stateless session bean implement business logic without having a persistent storage mechanism, such as a state or database and can used shared data. Stateless session bean can be used in situations where information is not required to used across call methods.



Types of Enterprise Java Beans

2. **Message Driven Bean:** Like Session Bean, it contains the business logic but it is invoked by passing message.



Types of Enterprise Java Beans

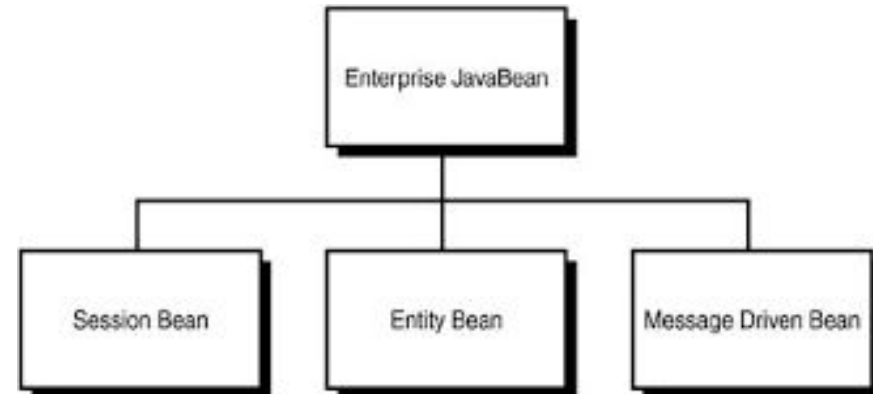
3. Entity Bean: It summarizes the state that can be remained in the database. It is deprecated. Now, it is replaced with JPA (Java Persistent API). There are two types of entity bean:

- **(i) Bean Managed Persistence :**

In a bean managed persistence type of entity bean, the programmer has to write the code for database calls. It persists across multiple sessions and multiple clients.

- **(ii) Container Managed Persistence :**

Container managed persistence are enterprise bean that persists across database. In container managed persistence the container take care of database calls.



Which of the following is *not* a valid type of Enterprise JavaBean?

- (a) Session Bean
- (b) Entity Bean
- (c) Message-Driven Bean
- (d) Servlet Bean

Which of the following is *not* a valid type of Enterprise JavaBean?

- (a) Session Bean
- (b) Entity Bean
- (c) Message-Driven Bean
- (d) Servlet Bean

Answer: (d)

Explanation:

The three standard types of EJBs are **Session**, **Entity**, and **Message-Driven Beans**.

There is **no “Servlet Bean”**; servlets belong to web components, not enterprise beans.

A Session Bean in EJB is mainly used to:

- (a) Represent persistent data stored in a database
- (b) Handle client-specific business logic and temporary operations
- (c) Receive asynchronous messages from JMS
- (d) Provide static configuration information

A Session Bean in EJB is mainly used to:

- (a) Represent persistent data stored in a database
- (b) Handle client-specific business logic and temporary operations
- (c) Receive asynchronous messages from JMS
- (d) Provide static configuration information

Answer: (b)

Explanation:

Session Beans contain **business logic** that serves a client during a session.

They are not persistent; their data usually lasts only for the client's session.

A Session Bean in EJB is mainly used to:

- (a) Represent persistent data stored in a database
- (b) Handle client-specific business logic and temporary operations
- (c) Receive asynchronous messages from JMS
- (d) Provide static configuration information

A Session Bean in EJB is mainly used to:

- (a) Represent persistent data stored in a database
- (b) Handle client-specific business logic and temporary operations
- (c) Receive asynchronous messages from JMS
- (d) Provide static configuration information

Answer: (b)

Explanation:

Session Beans contain **business logic** that serves a client during a session.

They are not persistent; their data usually lasts only for the client's session.

Entity Beans are designed to:

- (a) Represent and manage persistent data stored in a database
- (b) Handle temporary user sessions
- (c) Deliver asynchronous messages to clients
- (d) Manage XML deployment descriptors

Entity Beans are designed to:

- (a) Represent and manage persistent data stored in a database
- (b) Handle temporary user sessions
- (c) Deliver asynchronous messages to clients
- (d) Manage XML deployment descriptors

Answer: (a)

Explanation:

Entity Beans map Java objects to **database tables or records**.

They allow persistence so that data can be automatically saved and retrieved by the EJB container.

Message-Driven Beans (MDBs) are mainly used for:

- (a) Handling synchronous database queries
- (b) Receiving and processing asynchronous messages (JMS)
- (c) Managing GUI threads
- (d) Controlling home interface creation

Message-Driven Beans (MDBs) are mainly used for:

- (a) Handling synchronous database queries
- (b) Receiving and processing asynchronous messages (JMS)
- (c) Managing GUI threads
- (d) Controlling home interface creation

Answer: (b)

Explanation:

Message-Driven Beans are used for **asynchronous communication** — typically through **Java Message Service (JMS)**.

They process messages sent by other components or systems.

ADVANTAGES OF EJB ARCHITECTURE

Benefits to Application Developers:

The EJB component architecture is the backbone of the J2EE platform. The EJB architecture provides the following benefits to the application developer:

- **Simplicity:** It is easier to develop an enterprise application with the EJB architecture than without it. Since, EJB architecture helps the application developer access and use enterprise services with minimal effort and time, writing an enterprise bean is almost as simple as writing a Java class. The application developer does not have to be concerned with system-level issues, such as security, transactions, multithreading, security protocols, distributed programming, connection resource pooling, and so forth. As a result, the application developer can concentrate on the business logic for domain-specific applications.
- **Application Portability:** An EJB application can be deployed on any J2EEcompliant server. This means that the application developer can sell the application to any customers who use a J2EE-compliant server. This also means that enterprises are not locked into a particular application server vendor. Instead, they can choose the “best-of-breed” application server that meets their requirements.

ADVANTAGES OF EJB ARCHITECTURE

Benefits to Application Developers:

- **Component Reusability:** An EJB application consists of enterprise bean components. Each enterprise bean is a reusable building block. There are two essential ways to reuse an enterprise bean:

- 1) An enterprise bean not yet deployed can be reused at application development time by being included in several applications. The bean can be customised for each application without requiring changes, or even access, to its source code.

- 2) Other applications can reuse an enterprise bean that is already deployed in a customer's operational environment, by making calls to its client-view interfaces. Multiple applications can make calls to the deployed bean.

In addition, the business logic of enterprise beans can be reused through Java subclassing of the enterprise bean class.

- **Ability to Build Complex Applications:** The EJB architecture simplifies building complex enterprise applications. These EJB applications are built by a team of developers and evolve over time. The component-based EJB architecture is well suited to the development and maintenance of complex enterprise applications. With its clear definition of roles and well-defined interfaces, the EJB architecture promotes and supports team-based development and lessens the demands on individual developers.

In addition, the business logic of enterprise beans can be reused through Java subclassing of the enterprise bean class.

- **Separation of Business Logic from Presentation Logic:** An enterprise bean typically encapsulates a business process or a business entity (an object representing enterprise business data), making it independent of the presentation logic. The business programmer need not worry about formatting the output; the Web page designer developing the Web page need be concerned only with the output data that will be passed to the Web page. In addition, this separation makes it possible to develop multiple presentation logic for the same business process or to change the presentation logic of a business process without needing to modify the code that implements the business process.

- **Easy Development of Web Services:** The Web services features of the EJB architecture provide an easy way for Java developers to develop and access Web services. Java developers do not need to bother about the complex details of Web services description formats and XML-based wire protocols but instead can program at the familiar level of enterprise bean components, Java interfaces and data types. The tools provided by the container manage the mapping to the Web services standards.

• **Deployment in many Operating Environments**— The goal of any software development company is to sell an application to many customers. Since, each customer has a unique operational environment, the application typically needs to be customised at deployment time to each operational environment, including different database schemas.

- 1) The EJB architecture allows the bean developer to separate the common application business logic from the customisation logic performed at deployment.
- 2) The EJB architecture allows an entity bean to be bound to different database schemas. This persistence binding is done at deployment time. The application developer can write a code that is not limited to a single type of database management system (DBMS) or database schema.
- 3) The EJB architecture facilitates the deployment of an application by establishing deployment standards, such as those for data source lookup, other application dependencies, security configuration, and so forth. The standards enable the use of deployment tools. The standards and tools remove much of the possibility of miscommunication between the developer and the deployer.

- **Distributed Deployment:** The EJB architecture makes it possible for applications to be deployed in a distributed manner across multiple servers on a network. The bean developer does not have to be aware of the deployment topology when developing enterprise beans but rather writes the same code whether the client of an enterprise bean is on the same machine or a different one.
- **Application Interoperability:** The EJB architecture makes it easier to integrate applications from different vendors. The enterprise bean's client-view interface serves as a well-defined integration point between applications.

- **Integration with non-Java Systems:** The related J2EE APIs, such as the J2EE Connector specification and the Java Message Service (JMS) specification, and J2EE Web services technologies, such as the Java API for XML-based RPC (JAXRPC), make it possible to integrate enterprise bean applications with various non-Java applications, such as ERP systems or mainframe applications, in a standard way.
- **Educational Resources and Development Tools:** Since, the EJB architecture is an industry wide standard, the EJB application developer benefits from a growing body of educational resources on how to build EJB applications. More importantly, powerful application development tools available from leading tool vendors simplify the development and maintenance of EJB applications.

Benefits to Customers

A customer's perspective on EJB architecture is different from that of the application developer. The EJB architecture provides the following benefits to the customer:

- **Choice of Server:** Because the EJB architecture is an industry wide standard and is part of the J2EE platform, customer organisations have a wide choice of J2EEcompliant servers. Customers can select a product that meets their needs in terms of scalability, integration capabilities with other systems, security protocols, price, andso forth. Customers are not locked in to a specific vendor's product. Should their needs change, customers can easily redeploy an EJB application in a server from a different vendor.

Benefits to Customers

- **Facilitation of Application Management:** Because the EJB architecture provides a standardised environment, server vendors have the required motivation to develop application management tools to enhance their products. As a result, sophisticated application management tools provided with the EJB container allow the customer's IT department to perform such functions as starting and stopping the application, allocating system resources to the application, and monitoring security violations, among others.

Benefits to Customers

- **Integration with a Customer's Existing Applications and Data:** The EJB architecture and the other related J2EE APIs simplify and standardise the integration of EJB applications with any non-Java applications and systems at the customer operational environment. For example, a customer does not have to change an existing database schema to fit an application. Instead, an EJB application can be made to fit the existing database schema when it is deployed.

Benefits to Customers

- **Integration with Enterprise Applications of Customers, Partners, and Suppliers:** The interoperability and Web services features of the EJB architecture allows EJB-based applications to be exposed as Web services to other enterprises. This means that enterprises have a single, consistent technology for developing intranet, Internet, and e-business applications.

- **Application Security:** The EJB architecture shifts most of the responsibility for an application's security from the application developer to the server vendor, system administrator, and the deployer. The people performing these roles are more qualified than the application developer to secure the application. This leads to better security of operational applications.

SERVICES OFFERED BY EJB

As we have learnt earlier, in J2EE all the components run inside their own containers. JSP, Servlets and JavaBeans and EJBs have their own web container. The container of EJB provides certain built-in services to EJBs, which is used by EJBs to perform different functions. The services that EJB container provides are:

- Component Pooling
- Resource Management
- Transaction Management
- Security
- Persistence
- Handling of multiple clients

SERVICES OFFERED BY EJB

i) Component Pooling

The EJB container handles the pooling of EJB components. If, there are no requests for a particular EJB then the container will probably contain zero or one instance of that component in the memory. If, the need arises then it will increase component instances to satisfy all incoming requests. Then again, if, the number of requests decrease, the container will decrease the component instances in the pool. The most important thing is that the client is absolutely unaware of this component pooling which the container handles.

ii) Resource Management

The container is also responsible for maintaining database connection pools. It provides us a standard way of obtaining and returning database connections. The container also manages EJB environment references and references to other EJBs. The container manages the following types of resources and makes them available to EJBs:

- JDBC 2.0 Data Sources
- JavaMail Sessions
- JMS Queues and Topics
- URL Resources
- Legacy Enterprise Systems via J2EE Connector Architecture

iii) Transaction Management

This is the most important functionality served by the container. A transaction is a single unit of work, composed of one or more steps. If, all the steps are successfully executed, then, the transaction is committed otherwise it is rolled back. There are different types of transactions and it is absolutely unimaginable not to use transactions in today's business Environments.

iv) Security

The EJB container provides it's own authentication and authorisation control, allowing only specific clients to interact with the business process. Programmers are not required to create a security architecture of their own, they are provided with a built-in system, all they have to do is to use it.

v) Persistence

Persistence is defined as saving the data or state of the EJB on non J-C volatile media. The container, if, desired can also maintain persistent data of the application. The container is then responsible for retrieving and saving the data for programmers, while taking care of concurrent access from multiple clients and not corrupting the data.

vi) Handling of Multiple Clients

The EJB container handles multiple clients of different types. A JSP based thin client can interact with EJBs with same ease as that of GUI based thick client. The container is smart enough to allow even non-Java clients like COM based applications to interact with the EJB system.

Table 1: Entity Beans and Stateful Session Beans: Life-Cycle Differences

Functional Area	Stateful Session Bean	Entity Bean
Object state	Maintained by the container in the main memory across transactions. Swapped to secondary storage when deactivated.	Maintained in the database or other resource manager. Typically cached in the memory in a transaction.
Object sharing	A session object can be used by only one client.	An entity object can be shared by multiple clients. A client may pass an object reference to another client.
State externalisation	The container internally maintains the session object's state. The state is inaccessible to other programs.	The entity object's state is typically stored in a database. Other programs, such as an SQL query, can access the state in the database.
Transactions	The state of a session object can be synchronised with a transaction but is not recoverable.	The state of an entity object is typically changed transactionally and is recoverable.
Failure recovery	A session object is not guaranteed to survive failure and restart of its container. The references to session objects held by a client becomes invalid after the failure.	An entity object survives the failure and the restart of its container. A client can continue using the references to the entity objects after the container restarts.

Q. Every EJB class file has (UPLT - 2018)

- A. one accompanying interface and one XML file**
- B. one accompanying interface and two XML files**
- C. two accompanying interfaces and one XML file**
- D. two accompanying interfaces and two XML files**

Q. Every EJB class file has (UPLT - 2018)

- A. one accompanying interface and one XML file**
- B. one accompanying interface and two XML files**
- C. two accompanying interfaces and one XML file**
- D. two accompanying interfaces and two XML files**

Answer: C

In **Enterprise JavaBeans**, each **EJB class** (the actual bean implementation) is associated with:

Two Interfaces

- **Remote Interface** → defines methods accessible to remote clients
- **Home Interface** → defines lifecycle methods (like create, remove)

2. One XML Deployment Descriptor

- Describes bean configuration, transaction type, security, and mappings.

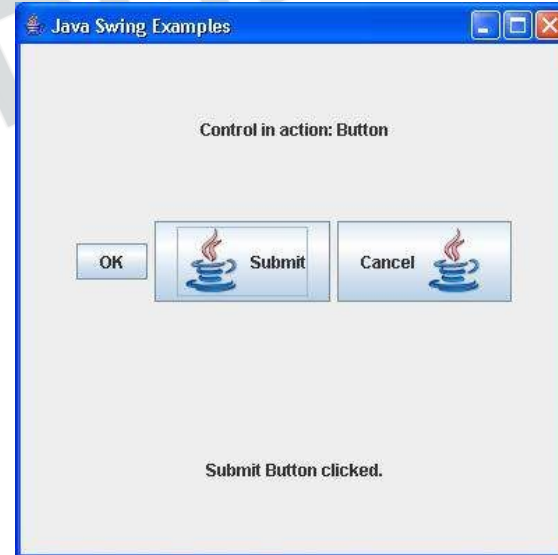
Chapter - 7

Java Swing

Swing — Overview

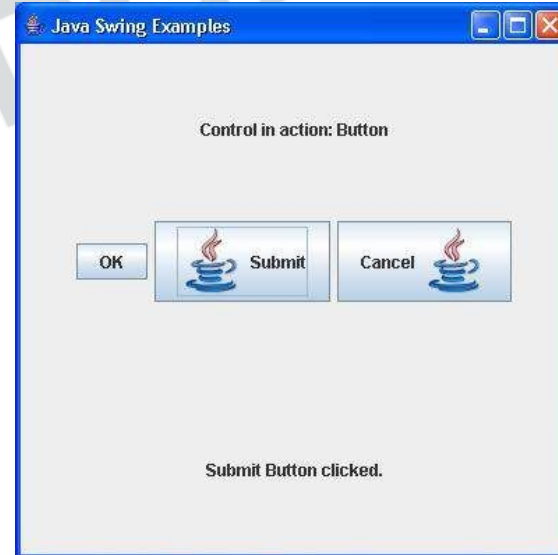
Swing API is a set of extensible GUI Components to ease the developer's life to create JAVA based Front End/GUI Applications. It is build on top of AWT API and acts as a replacement of AWT API, since it has almost every control corresponding to AWT controls. Swing component follows a Model-View-Controller architecture to fulfill the following criterias.

- A single API is to be sufficient to support multiple look and feel.
- API is to be model driven so that the highest level API is not required to have data.
- API is to use the Java Bean model so that Builder Tools and IDE can provide better services to the developers for use.



Swing — Overview

- Swing is a Set of API (API- Set of Classes and Interfaces)
- Swing is Provided to Design Graphical User Interfaces
- Swing is an Extension library to the AWT (Abstract Window Toolkit)
- Includes New and improved Components that have been enhancing the looks and Functionality of GUIs'
- Swing can be used to build (Develop) The Standalone swing GUI Apps as Servlets and Applets
- Swing is Entirely written in Java.

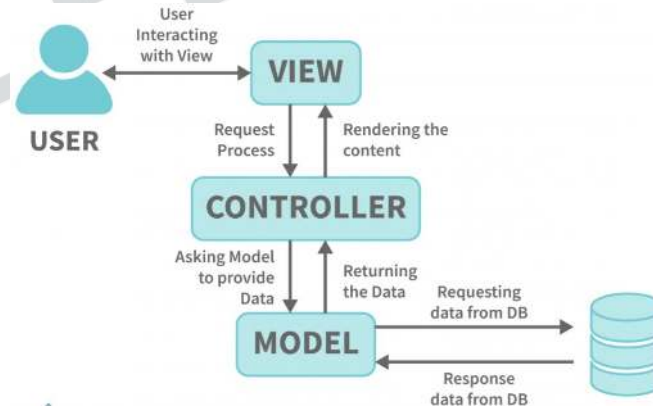


-

MVC Architecture

Swing API architecture follows loosely based MVC architecture in the following manner.

- Model represents component's data.
- View represents visual representation of the component's data.
- Controller takes the input from the user on the view and reflects the changes in Component's data.
- Swing component has Model as a separate element, while the View and Controller part are clubbed in the User Interface elements. Because of which, Swing has a pluggable look-and-feel architecture.



Java AWT	Java Swing
Java AWT is an API to develop GUI applications in Java.	Swing is a part of Java Foundation Classes and is used to create various applications.
Components of AWT are heavy weighted.	The components of Java Swing are lightweight.
Components are platform dependent.	Components are platform independent.
Execution Time is more than Swing.	Execution Time is less than AWT.
AWT components require java.awt package.	Swing components requires javax.swing package.

What is JFC?

JFC stands for Java Foundation Classes. JFC is the set of GUI components that simplify desktop Applications. Many programmers think that JFC and Swing are one and the same thing, but that is not so. JFC contains Swing [A UI component package] and quite a number of other items:

- Cut and paste: Clipboard support.
- Accessibility features: Aimed at developing GUIs for users with disabilities.
- The Desktop Colors Features were first introduced in Java 1.1
- Java 2D: it has Improved colors, images, and text support.

Swing Features

- **Light Weight** - Swing components are independent of native Operating System's API as Swing API controls are rendered mostly using pure JAVA code instead of underlying operating system calls.
- **Rich Controls** - Swing provides a rich set of advanced controls like Tree, TabbedPane, slider, colorpicker, and table controls.
- **Highly Customizable** - Swing controls can be customized in a very easy way as visual appearance is independent of internal representation.
- **Pluggable look-and-feel** - SWING based GUI Application look and feel can be changed at run-time, based on available values.

Which of the following statements about Java Swing is TRUE?

- (a) Swing components are heavyweight because they rely on native OS peers.
- (b) Swing components are lightweight and platform-dependent.
- (c) Swing components are lightweight and platform-independent.
- (d) Swing components use native peer components for rendering.

Which of the following statements about Java Swing is TRUE?

- (a) Swing components are heavyweight because they rely on native OS peers.
- (b) Swing components are lightweight and platform-dependent.
- (c) Swing components are lightweight and platform-independent.
- (d) Swing components use native peer components for rendering.

Answer: (c)

Explanation:

Swing components are **lightweight** (rendered using Java code, not OS peers) and hence **platform-independent** — they look and behave the same across operating systems.

The Java Swing library is a part of which broader framework?

- (a) Abstract Window Toolkit (AWT)
- (b) Java Foundation Classes (JFC)
- (c) Java Development Kit (JDK) Core API
- (d) Java Virtual Machine (JVM)

The Java Swing library is a part of which broader framework?

- (a) Abstract Window Toolkit (AWT)
- (b) Java Foundation Classes (JFC)
- (c) Java Development Kit (JDK) Core API
- (d) Java Virtual Machine (JVM)

Answer: (b)

Explanation:

Swing is part of **Java Foundation Classes (JFC)**, which includes Swing, AWT, 2D graphics, accessibility, and pluggable look-and-feel APIs.

Which of the following statements correctly distinguishes AWT and Swing?

(NIELIT Scientist-B 2018)

- (a) AWT components are lightweight whereas Swing components are heavyweight.
- (b) AWT components are platform-independent whereas Swing components are platform-dependent.
- (c) AWT components are heavyweight whereas Swing components are lightweight.
- (d) Both AWT and Swing components are heavyweight.

Which of the following statements correctly distinguishes AWT and Swing?

(NIELIT Scientist-B 2018)

- (a) AWT components are lightweight whereas Swing components are heavyweight.
- (b) AWT components are platform-independent whereas Swing components are platform-dependent.
- (c) AWT components are heavyweight whereas Swing components are lightweight.
- (d) Both AWT and Swing components are heavyweight.

Answer: (c)

Explanation:

AWT uses native OS widgets (hence **heavyweight**), while Swing components are drawn in Java (hence **lightweight**).

The term “pluggable look and feel” in Swing refers to:

(UPPSC Lecturer–Computer Science)

- (a) Ability to change the window title bar at runtime.
- (b) Ability to change the operating system theme.
- (c) Ability to change the appearance and behavior of components dynamically.
- (d) Ability to switch between AWT and Swing interfaces.

The term “pluggable look and feel” in Swing refers to:

(UPPSC Lecturer–Computer Science)

- (a) Ability to change the window title bar at runtime.
- (b) Ability to change the operating system theme.
- (c) Ability to change the appearance and behavior of components dynamically.
- (d) Ability to switch between AWT and Swing interfaces.

Answer: (c)

Explanation:

Swing allows **pluggable look and feel (PLAF)** — the appearance and behavior of GUI components can be changed **dynamically** at runtime (e.g., Metal, Nimbus, Windows look & feel).

Popular Java Editors

To write your Java programs, you will need a text editor. There are even more sophisticated IDE available in the market. But for now, you can consider one of the following:

Notepad: On Windows machine, you can use any simple text editor like Notepad (Recommended for this tutorial), TextPad.

Netbeans: Netbeans is a Java IDE that is open source and free, which can be downloaded from <http://www.netbeans.org/index.html>.

Eclipse: Eclipse is also a Java IDE developed by the Eclipse open source community and can be downloaded from <http://www.eclipse.org/>

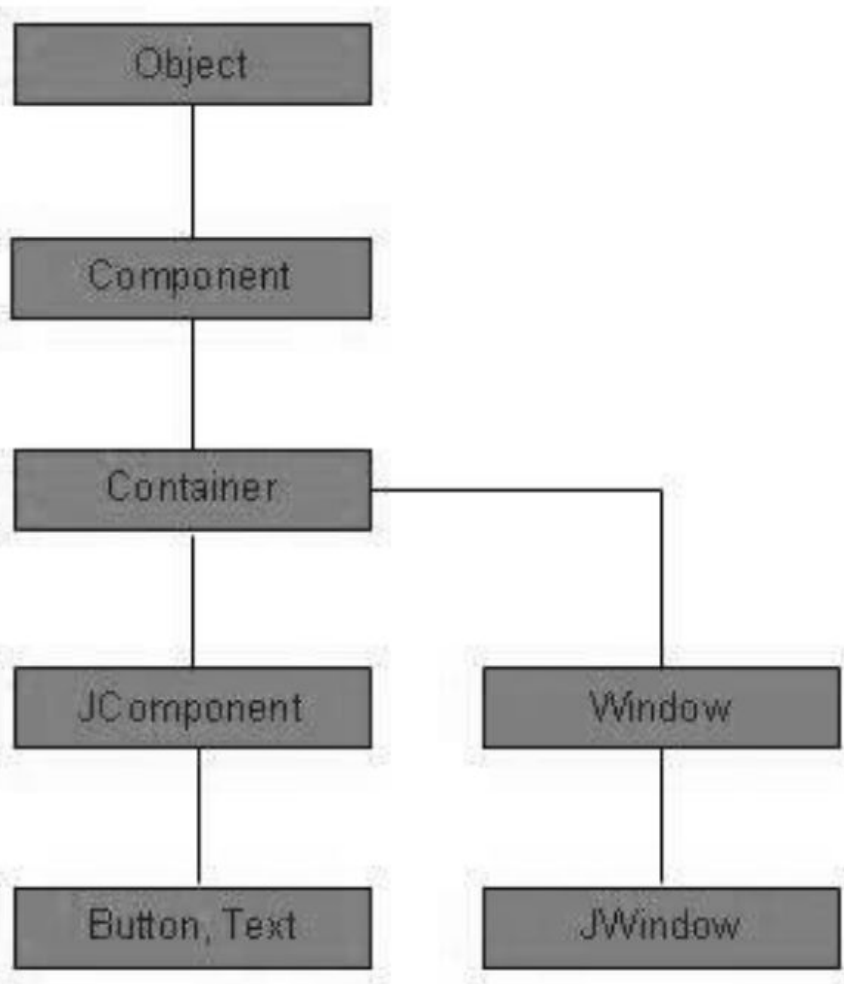
3. Swing — Controls

Every user interface considers the following three main aspects:

UI Elements: These are the core visual elements the user eventually sees and interacts with. GWT provides a huge list of widely used and common elements varying from basic to complex, which we will cover in this tutorial.

Layouts: They define how UI elements should be organized on the screen and provide a final look and feel to the GUI (Graphical User Interface). This part will be covered in the Layout chapter.

Behavior: These are the events which occur when the user interacts with UI elements. This part will be covered in the Event Handling chapter.



Swing Component Hierarchy

`java.lang.Object`

↳ `java.awt.Component`

↳ `java.awt.Container`

↳

`javax.swing.JComponent`

↳ `JFrame`,

`JPanel`, `JButton`, `JLabel`,

`JTextField`, `JTable`, etc.

Every SWING controls inherits properties from the following Component class hierarchy.

Sr. No.	Class & Description
1	<u>Component</u> A Component is the abstract base class for the non menu user-interface controls of SWING. Component represents an object with graphical representation
2	<u>Container</u> A Container is a component that can contain other SWING components
3	<u>JComponent</u> A JComponent is a base class for all SWING UI components. In order to use a SWING component that inherits from JComponent, the component must be in a containment hierarchy whose root is a top-level SWING container

Class	Description
Component	A Component is the Abstract base class for about the non-menu user-interface controls of Java SWING. Components are representing an object with a graphical representation.
Container	A Container is a component that can container Java SWING Components
JComponent	A JComponent is a base class for all swing UI Components In order to use a swing component that inherits from JComponent, the component must be in a containment hierarchy whose root is a top-level Java Swing container.
JLabel	A JLabel is an object component for placing text in a container.
JButton	This class creates a labeled button.

JColorChooser	A JColorChooser provides a pane of controls designed to allow the user to manipulate and select a color.
JCheckBox	A JCheckBox is a graphical (GUI) component that can be in either an on-(true) or off-(false) state.
JRadioButton	The JRadioButton class is a graphical (GUI) component that can be in either an on-(true) or off-(false) state. in the group
JList	A JList component represents the user with the scrolling list of text items.
JComboBox	A JComboBox component is Presents the User with a show up Menu of choices.
JTextField	A JTextField object is a text component that will allow for the editing of a single line of text.
JPasswordField	A JPasswordField object it is a text component specialized for password entry.

JTextArea	A JTextArea object is a text component that allows for the editing of multiple lines of text.
ImageIcon	A ImageIcon control is an implementation of the Icon interface that paints Icons from Images
JScrollbar	A JScrollbar control represents a scroll bar component in order to enable users to Select from range values.
JOptionPane	JOptionPane provides set of standard dialog boxes that prompt users for a value or Something.
JFileChooser	A JFileChooser it Controls represents a dialog window from which the user can select a file.
JProgressBar	As the task progresses towards completion, the progress bar displays the tasks percentage on its completion.

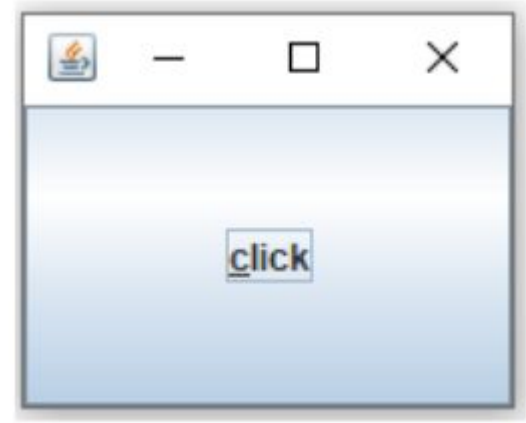
JSlider	A JSlider this class is letting the user graphically (GUI) select by using a value by sliding a knob within a bounded interval.
JSpinner	A JSpinner this class is a single line input where the field that lets the user select by using a number or an object value from an ordered sequence.

KNOWLEDGE


```
import javax.swing.*;
import java.awt.event.*;

public class SwingDemo1
{
    public static void main (String[]args)
    {
        JFrame jf = new JFrame ();
        jf.setSize (100, 100);
        JButton jb = new JButton ("click");
        jb.setMnemonic ('C');
        MyListener ob = new MyListener ();
        jb.addActionListener (ob);
        jf.add (jb);
        jf.setVisible (true);
    }
}

class MyListener implements ActionListener
{
    public void actionPerformed (ActionEvent ae)
    {
        System.out.println ("Button Clicked");
    }
}
```



Which of the following is the immediate superclass of all Swing components (except top-level containers)?

(NIELIT Scientist-B 2017)

- (a) `java.awt.Component`
- (b) `java.awt.Container`
- (c) `javax.swing.JComponent`
- (d) `java.awt.Window`

Which of the following is the immediate superclass of all Swing components (except top-level containers)?

(NIELIT Scientist-B 2017)

- (a) `java.awt.Component`
- (b) `java.awt.Container`
- (c) `javax.swing.JComponent`
- (d) `java.awt.Window`

Answer: (c)

Explanation:

Most Swing components (like `JButton`, `JLabel`, `TextField`) directly inherit from **`javax.swing.JComponent`**, which provides painting, borders, and tooltip support.

In Swing, which class provides the base functionality like borders, double buffering and tooltips?

- (a) `javax.swing.JComponent`
- (b) `java.awt.Container`
- (c) `javax.swing.JFrame`
- (d) `java.awt.Panel`

In Swing, which class provides the base functionality like borders, double buffering and tooltips?

- (a) `javax.swing.JComponent`
- (b) `java.awt.Container`
- (c) `javax.swing.JFrame`
- (d) `java.awt.Panel`

Answer: (a)

Explanation:

JComponent provides essential Swing-specific features — like **borders, double buffering, and lightweight rendering** — to all Swing components.

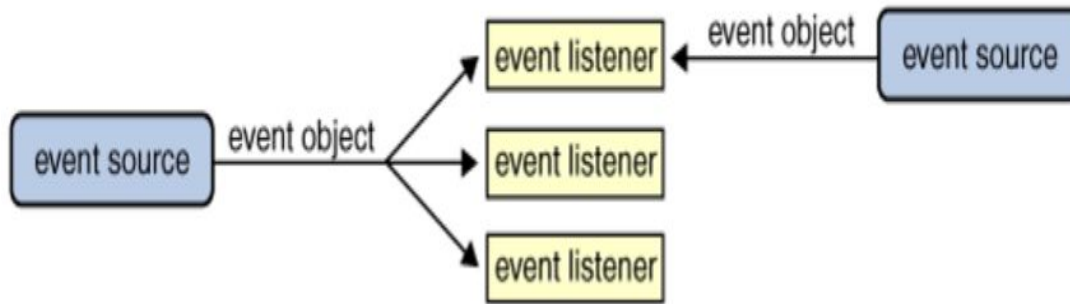
Additional event types that are specific to swing GUI components are declared in package _____

- (a) Javax. awt. Font
- (b) Javax. awt. event
- (c) Javax. swing. event
- (d) Javax. swing. Font

TRB Poly. Lect. 2017

What is an Event?

Change in the state of an object is known as **Event**, i.e., event describes the change in the state of the source. Events are generated as a result of user interaction with the graphical user interface components. For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from the list, and scrolling the page are the activities that causes an event to occur.



Java.awt.event

Provides the **core event classes and listener interfaces** used for event handling in **AWT** and also in **Swing** (for common user actions).

Implements the **Event Delegation Model** (source → listener → event).

Characteristics

- Handles **basic user input** and **windowing events**.
- Used for **low-level** GUI interactions (keyboard, mouse, action, focus, window, etc.).
- Imported automatically by both AWT and Swing programs.

Sr.No.	Class & Description
1	ActionListener This interface is used for receiving the action events.
2	ComponentListener This interface is used for receiving the component events.
3	ItemListener This interface is used for receiving the item events.
4	KeyListener This interface is used for receiving the key events.
5	MouseListener This interface is used for receiving the mouse events.
6	WindowListener This interface is used for receiving the window events.

7

AdjustmentListener

This interface is used for receiving the adjustment events.

8

ContainerListener

This interface is used for receiving the container events.

9

MouseMotionListener

This interface is used for receiving the mouse motion events.

10

FocusListener

This interface is used for receiving the focus events.

javax.swing.event Package

Purpose

- Provides **additional, higher-level event types** and listener interfaces that are **specific to Swing GUI components** (not present in AWT).

Characteristics

- Used for **data-driven** or **model-based** Swing components (like **JList**, **JTable**, **JTree**).
- Supports **fine-grained control** over user selections, document edits, and UI state changes.
- Part of **Java Foundation Classes (JFC)**.

Listener

Method(s)

Trigger

ChangeListener

stateChanged()

Tab/Slider change

ListSelectionListe
ner

valueChanged()

List or table selection

DocumentListener

insertUpdate(), removeUpdate(),
changedUpdate()

Text model changes

TreeSelectionListe
ner

valueChanged()

Tree node selection

MenuListener

menuSelected(), menuCanceled()

Menu actions



Additional event types that are specific to swing GUI components are declared in package _____

- (a) Javax. awt. Font
- (b) Javax. awt. event
- (c) Javax. swing. event
- (d) Javax. swing. Font

TRB Poly. Lect. 2017

Additional event types that are specific to swing GUI components are declared in package _____

- (a) Javax. awt. Font
- (b) Javax. awt. event
- (c) Javax. swing. event
- (d) Javax. swing. Font

TRB Poly. Lect. 2017

Ans. (c)

Which method is invoked when a user clicks a JButton that has an ActionListener registered with it?

- (a) buttonPressed()
- (b) actionPerformed(ActionEvent e)
- (c) performAction()
- (d) onClick()

Which method is invoked when a user clicks a JButton that has an ActionListener registered with it?

- (a) buttonPressed()
- (b) actionPerformed(ActionEvent e)
- (c) performAction()
- (d) onClick()

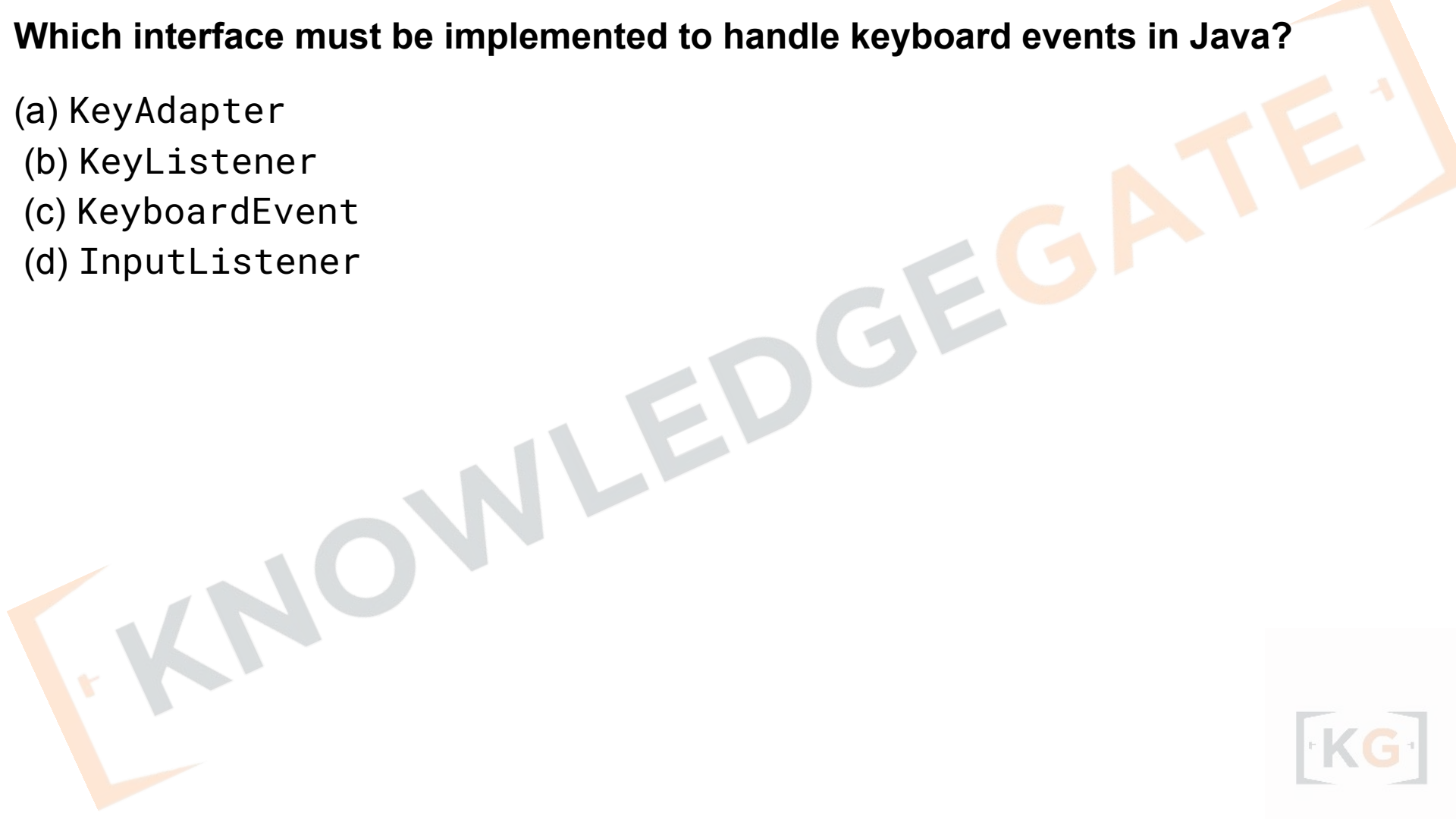
Answer: (b) actionPerformed(ActionEvent e)

Explanation:

When an ActionEvent occurs (like clicking a JButton), the **actionPerformed()** method of the registered ActionListener is called automatically.

Which interface must be implemented to handle keyboard events in Java?

- (a) KeyAdapter
- (b) KeyListener
- (c) KeyEvent
- (d) InputListener



Which interface must be implemented to handle keyboard events in Java?

- (a) KeyAdapter
- (b) KeyListener
- (c) KeyEvent
- (d) InputListener

Answer: (b) KeyListener

Explanation:

Keyboard events (key press, release, type) are handled by implementing the **KeyListener** interface, which defines:

- keyPressed(KeyEvent e)
- keyReleased(KeyEvent e)
- keyTyped(KeyEvent e)

Chapter - 8

Java as internet programming language

Java's Lineage

- Java is related to C++, which is a direct descendant of C.
- Much of the character of Java is inherited from these two languages.
- From C, Java derives its syntax.
- Many of Java's object-oriented features were influenced by C++.

- **The Birth of Modern Programming: C**
- The C language shook the computer world.
- When a computer language is designed, trade-offs are often made, such as the following:
 - Ease-of-use versus power
 - Safety versus efficiency
 - Rigidity versus extensibility
- Invented and first implemented by Dennis Ritchie on a DEC PDP-11 running the UNIX operating system,
- C was the result of a development process that started with an older language called BCPL, developed by Martin Richards.
- BCPL influenced a language called B, invented by Ken Thompson, which led to the development of C in the 1970s.
- C was formally standardized in December 1989, when the American National Standards Institute (ANSI) standard for C was adopted.
- C is a language designed by and for programmers.

C++: The Next Step

- C is a successful and useful language, you might ask *why a need for something else* existed. The answer is *complexity*.
- To solve this problem, a new way to program was invented, called *object-oriented programming (OOP)*.
- C++ was invented by Bjarne Stroustrup in 1979, while he was working at Bell Laboratories in Murray Hill, New Jersey.
- Stroustrup initially called the new language “C with Classes.”
- However, in 1983, the name was changed to C++. C++ extends C by adding object-oriented features.

The Stage Is Set for Java

- By the end of the 1980s and the early 1990s, object-oriented programming using C++ took hold.
- Within a few years, the World Wide Web and the Internet would reach critical mass.
- This event would precipitate another revolution in programming.

The Creation of Java

- The second force was, of course, the World Wide Web.
- **The emergence of the World Wide Web, Java was propelled to the forefront of computer language design, because the Web, too, demanded portable programs.**
- The Internet ultimately led to Java's large-scale success.
- **Java is simply the "Internet version of C++."**
- Java was to Internet programming what C was to system programming: a revolutionary force that changed the world.

How Java Changed the Internet

- Java innovated a new type of networked program called the applet that changed the way the online world thought about content.
- Java also addressed some of the thorniest issues associated with the Internet: portability and security.

Feature

Explanation

1. Platform Independence

Java programs compile to **bytecode**, executed by the **JVM** on any OS (Windows, Mac, Linux, etc.) — ensuring cross-platform compatibility.

2. Security

Java provides **bytecode verification**, and **security managers** that prevent unauthorized access — essential for running untrusted code from the Internet.

3. Portability

Java code and libraries are standardized, so applications can be moved between platforms without modification.

4. Robustness

Exception handling and strong type checking make Java suitable for reliable networked environments.

5. Dynamic Linking

Classes are loaded dynamically (on demand), which supports distributed web applications where components can be downloaded from remote servers.

6. Multithreading

Built-in thread support helps manage multiple tasks like file downloads, streaming, and user interaction in Internet apps.

7. Networking Support (java.net)

Java provides **native classes for TCP/IP networking** (Socket, URL, Datagram) — no external libraries required.

8. Distributed Computing (RMI, CORBA)

Remote Method Invocation (RMI) and CORBA allow programs to run across multiple networked systems as if they were local.

Java is often referred to as an Internet language because it:

- (a) Supports pointer arithmetic for faster networking
- (b) Enables platform-dependent applets
- (c) Provides security and portability for web applications
- (d) Uses binary machine code for execution

Java is often referred to as an Internet language because it:

- (a) Supports pointer arithmetic for faster networking
- (b) Enables platform-dependent applets
- (c) Provides security and portability for web applications
- (d) Uses binary machine code for execution

Answer: (c)

Explanation:

Java's **security, platform independence, and networking support (java.net)** make it ideal for Internet-based distributed applications.

Which of the following Java features ensures the same program runs on different platforms?

(UGC NET – 2017)

- (a) Compiled to native code
- (b) Bytecode execution through JVM
- (c) Platform-specific libraries
- (d) Hardware abstraction

Which of the following Java features ensures the same program runs on different platforms?

(UGC NET – 2017)

- (a) Compiled to native code
- (b) Bytecode execution through JVM
- (c) Platform-specific libraries
- (d) Hardware abstraction

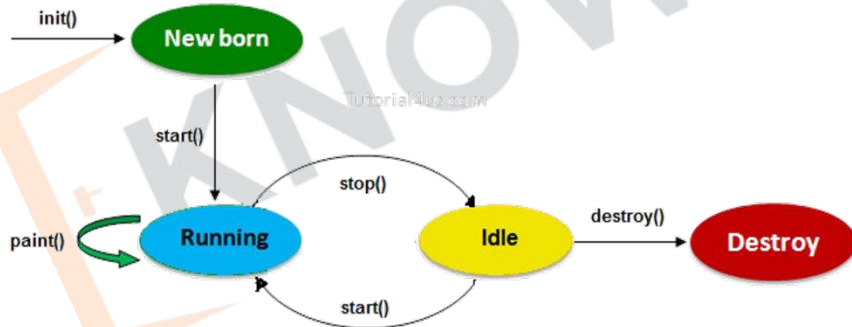
Answer: (b)

Explanation:

Java compiles code into **bytecode**, which is executed by the **Java Virtual Machine (JVM)** — ensuring platform independence (“Write Once, Run Anywhere”).

Java Applets

- An *applet* is a special kind of Java program that is designed to be transmitted over the Internet and automatically executed by a Java-compatible web browser.
- Applets are intended to be small programs.
- They are typically used to display data provided by the server, handle user input, or provide simple functions, such as a loan calculator, that execute locally, rather than on the server.
- In essence, the applet allows some functionality to be moved from the server to the client.
- The creation of the applet changed Internet programming because it expanded the universe of objects that can move about freely in cyberspace.



Java Applets

- Two very broad categories of objects that are transmitted between the server and the client:
 - passive information and
 - e-mail, you are viewing passive data.
 - dynamic, active programs.
 - the applet is a dynamic, self-executing program.
- As desirable as dynamic, networked programs are, they also present serious problems in the areas of security and portability.
Obviously, a program that downloads and executes automatically on the client computer must be prevented from doing harm.
- It must also be able to run in a variety of different environments and under different operating systems.
- **As you will see, Java solved these problems in an effective and elegant way.**

Applets in Java are used for:

(UGC NET – 2016)

- (a) Creating standalone desktop applications
- (b) Creating web-based interactive programs running in browsers
- (c) Compiling Java code into native OS format
- (d) Executing server-side logic

Applets in Java are used for:

(UGC NET – 2016)

- (a) Creating standalone desktop applications
- (b) Creating web-based interactive programs running in browsers
- (c) Compiling Java code into native OS format
- (d) Executing server-side logic

Answer: (b)

Explanation:

Applets are **client-side Java programs** that run inside browsers (sandboxed environment) to make web pages dynamic and interactive.

Security

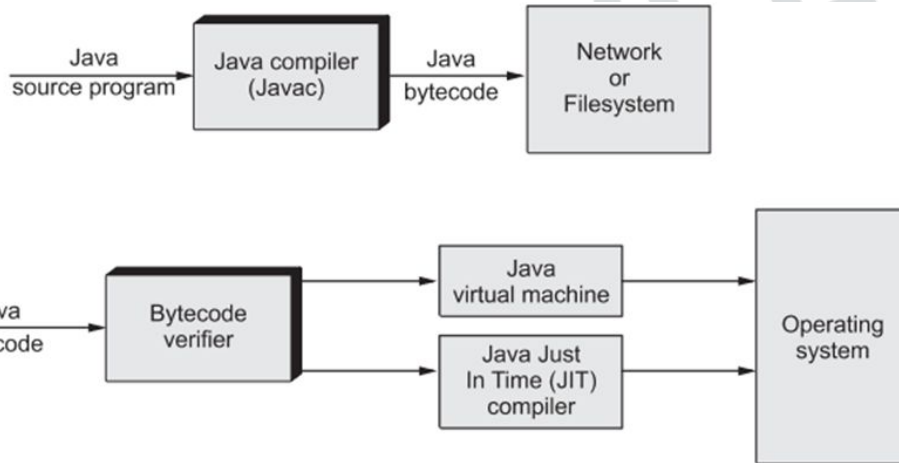
- Every time you download a “normal” program, a risk, because the code you are downloading might contain a virus, Trojan horse, or other harmful code.
- For example, a virus program might gather private information, such as credit card numbers, bank account balances, and passwords, by searching the contents of your computer’s local file system.
- Java enabled applets to be downloaded and executed on the client computer safely.
- Java achieved this protection by confining an applet to the Java execution environment and not allowing it access to other parts of the computer.
- The ability to download applets with confidence that no harm will be done and that no security will be breached is considered by many to be the single most innovative aspect of Java.

Portability

- Portability is a major aspect of the Internet because there are many different types of computers and operating systems connected to it.
- If a Java program were to be run on virtually any computer connected to the Internet, there needed to be some way to enable that program to execute on different systems.
- For example, applet must be able to be downloaded and executed by the wide variety of CPUs, operating systems, and browsers connected to the Internet.
- It is not practical to have different versions of the applet for different computers.
- The *same code must work on all* computers. Therefore, some means of generating portable executable code was needed.

Java's Magic: The Bytecode

- The key that allows Java to solve both the security and the portability, the output of a Java compiler is not executable code. Rather, it is bytecode.
- *Bytecode* is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the *Java Virtual Machine (JVM)*.
- *In essence, the original JVM was designed as an interpreter for bytecode.*



Java's Magic: The Bytecode

- Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
- Once the run-time package exists for a given system, any Java program can run on it.
- Remember, although the details of the JVM will differ from platform to platform, all understand the same Java bytecode.
- The execution of bytecode by the JVM is the easiest way to create truly portable programs.
- The Java program is executed by the JVM helps to make it secure. Because the JVM is in control, it can contain the program and prevent it from generating side effects outside of the system.

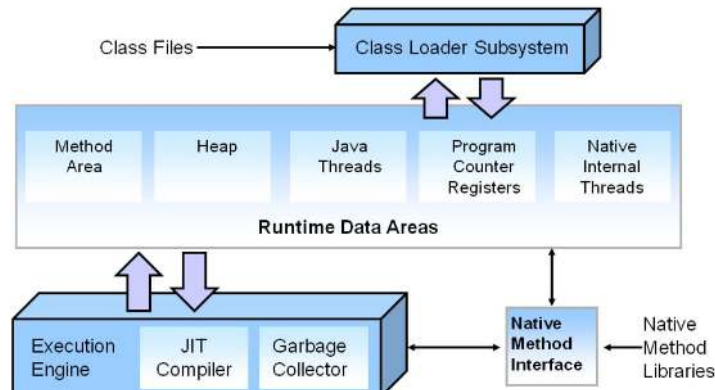
Java's Magic: The Bytecode

- In general, when a program is compiled to an intermediate form and then interpreted by a virtual machine, it runs slower than it would run if compiled to executable code.
- However, with Java, the differential between the two is not so great.
- Because bytecode has been highly optimized, the use of bytecode enables the JVM to execute programs much faster than you might expect.
- Java was designed as an interpreted language, there is nothing about Java that prevents on-the-fly compilation of bytecode into native code in order to boost performance.
 - Sun began supplying its HotSpot technology.

HotSpot

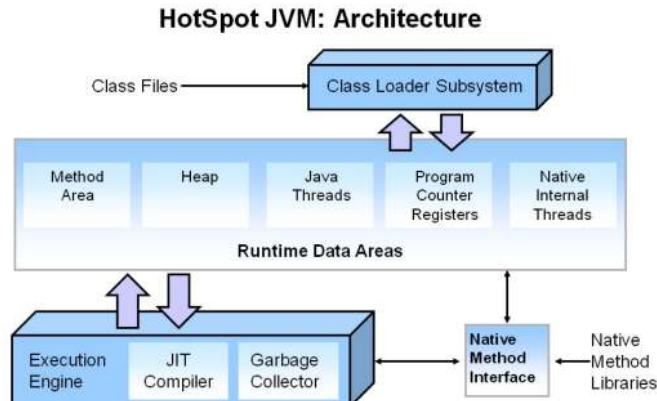
- HotSpot provides a Just-In-Time (JIT) compiler for bytecode.
- When a JIT compiler is part of the JVM, selected portions of bytecode are compiled into executable code in real time, on a piece-by-piece, demand basis.
- It is important to understand that it is not practical to compile an entire Java program into executable code all at once, because Java performs various run-time checks that can be done only at run time. Instead, a JIT compiler compiles code as it is needed, during execution.

HotSpot JVM: Architecture



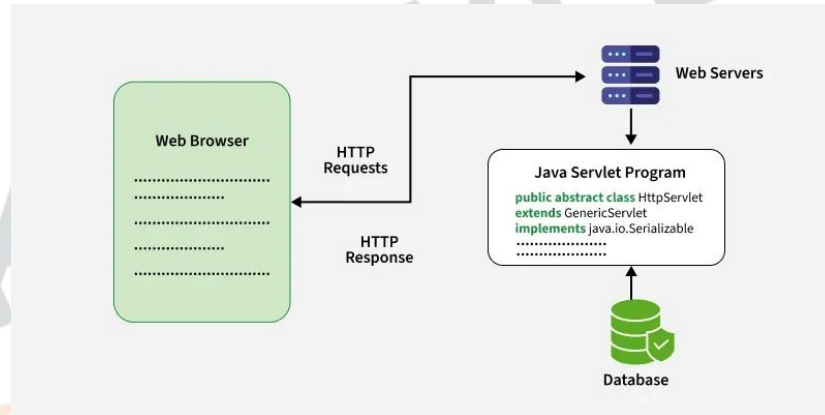
HotSpot

- Furthermore, not all sequences of bytecode are compiled—only those that will benefit from compilation. The remaining code is simply interpreted.
- However, the just-in-time approach still yields a significant performance boost.
- Even when dynamic compilation is applied to bytecode, the portability and safety features still apply, because the JVM is still in charge of the execution environment.



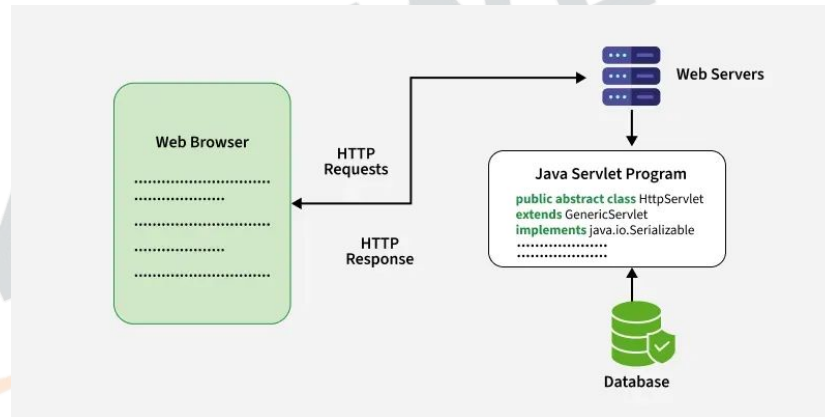
Servlets: Java on the Server Side

- As useful as applets can be, they are just one half of the client/server equation.
- At Serverside, *servlet*.
- A servlet is a small program that executes on the server. Just as applets dynamically extend the functionality of a web browser, servlets dynamically extend the functionality of a web server.
- Thus, with the advent of the servlet, Java spanned both sides of the client/server connection.



Servlets: Java on the Server Side

- Servlets are used to create dynamically generated content that is then served to the client. For example, an online store might use a servlet to look up the price for an item in a database.
- Dynamically generated content is available through mechanisms such as CGI (Common Gateway Interface).
- Because servlets (like all Java programs) are compiled into bytecode and executed by the JVM, they are highly portable. Thus, the same servlet can be used in a variety of different server environments. The only requirements are that the server support the JVM and a servlet container.



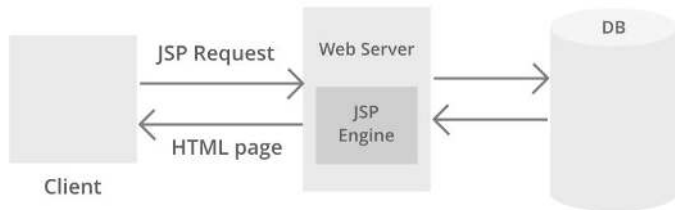
JSP (Java Server Pages)

A JSP page consists of HTML tags and JSP tags. The jsp pages are easier to maintain than servlet because we can separate designing and development.

Advantage of JSP over Servlet

There are many advantages of JSP over servlet. They are as follows:

- 1) Extension to Servlet**
- 2) Easy to maintain**
- 3) Fast Development: No need to recompile and redeploy**
- 4) Less code than Servlet**



Remote Method Invocation (RMI)

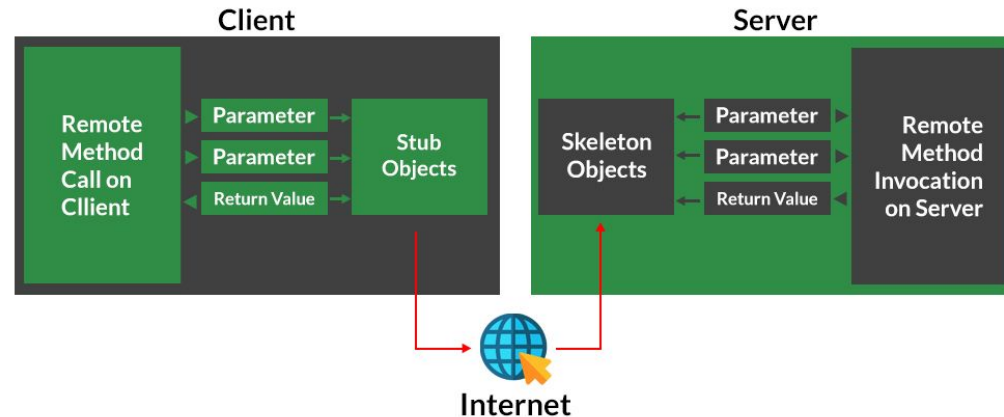
Remote Method Invocation (RMI) is an API that allows an object to invoke a method on an object that exists in another address space, which could be on the same machine or on a remote machine. Through RMI, an object running in a JVM present on a computer (Client-side) can invoke methods on an object present in another JVM (Server-side). RMI creates a public remote server object that enables client and server-side communications through simple method calls on the server object.

Stub Object: The stub object on the client machine builds an information block and sends this information to the server.

The block consists of

- An identifier of the remote object to be used
- Method name which is to be invoked
- Parameters to the remote JVM

Working of RMI

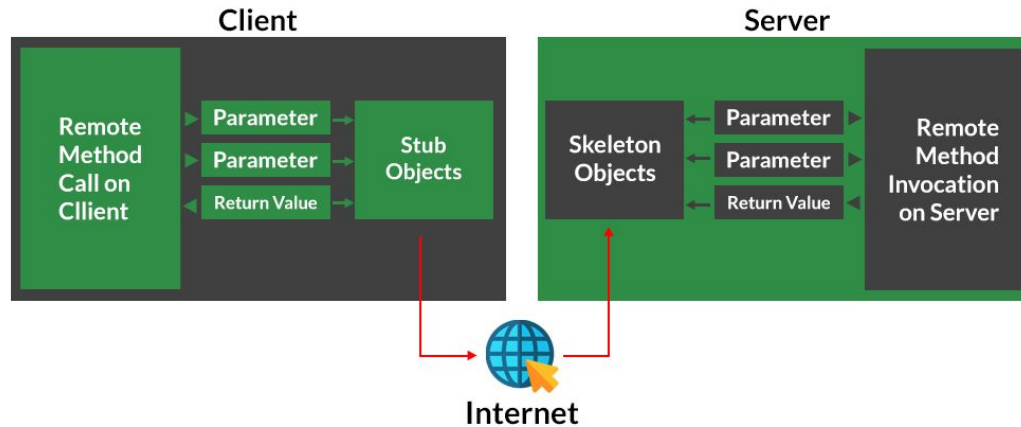


Remote Method Invocation (RMI)

Skeleton Object: The skeleton object passes the request from the stub object to the remote object. It performs the following tasks

- It calls the desired method on the real object present on the server.
- It forwards the parameters received from the stub object to the method.

Working of RMI



JavaBeans and Enterprise JavaBeans (EJB)

JavaBeans: Reusable software components used in web forms or applications.

EJB: Server-side components for large-scale enterprise Internet apps.

Which of the following is NOT a server-side Java Internet technology?

- (a) Servlet
- (b) JSP (Java Server Pages)
- (c) Applet
- (d) EJB (Enterprise JavaBeans)

Which of the following is NOT a server-side Java Internet technology?

- (a) Servlet
- (b) JSP (Java Server Pages)
- (c) Applet
- (d) EJB (Enterprise JavaBeans)

Answer: (c)

Explanation:

Applets are **client-side** Java programs, while **Servlets**, **JSP**, and **EJB** run on the **server-side** to handle web requests.

Which of the following packages provides networking capabilities in Java?

(TRB Polytechnic 2017)

- (a) `java.io`
- (b) `java.net`
- (c) `java.applet`
- (d) `java.rmi`

Which of the following packages provides networking capabilities in Java?

(TRB Polytechnic 2017)

- (a) `java.io`
- (b) `java.net`
- (c) `java.applet`
- (d) `java.rmi`

Answer: (b)

Explanation:

The **`java.net`** package provides classes like `Socket`, `ServerSocket`, `URL`, and `DatagramSocket` for TCP/IP and UDP-based network communication.

Remember key packages:

- `java.net` → Networking
- `java.rmi` → Distributed
- `java.applet` → Client-side web
- `javax.servlet` / `javax.ejb` → Server-side web

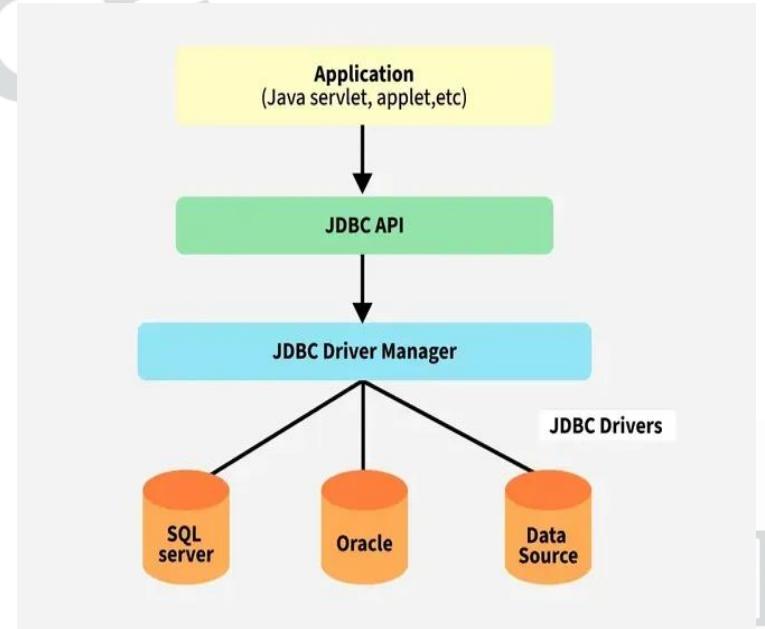
Chapter - 9

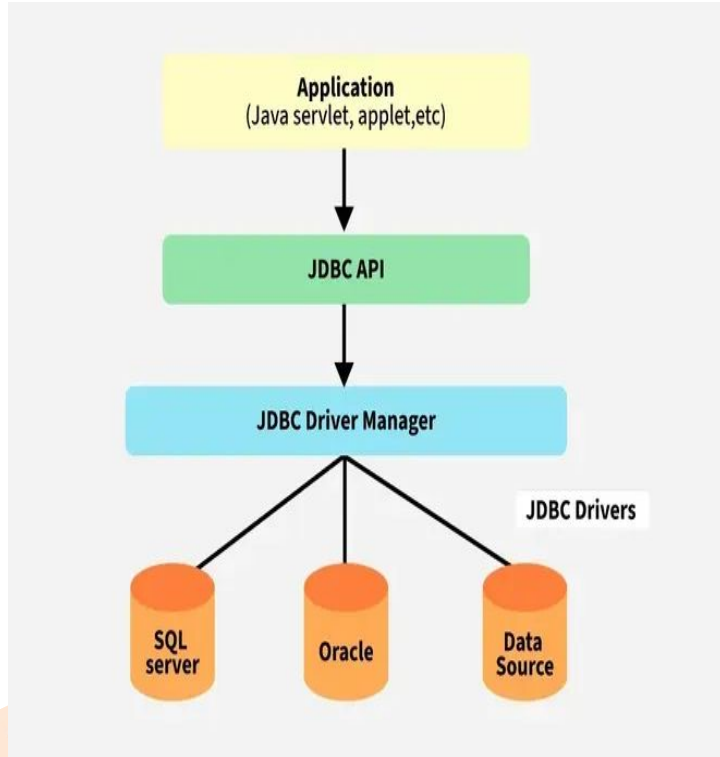
The connectivity model, JDBC/ODBC, Bridge



JDBC Overview

- JDBC stands for Java Database Connectivity, which is a standard Java API for database independent connectivity between the Java programming language and a wide range of databases.
- The JDBC library includes APIs for each of the tasks mentioned below that are commonly associated with database usage.
 - Making a connection to a database.
 - Creating SQL or MySQL statements.
 - Executing SQL or MySQL queries in the database.
 - Viewing & Modifying the resulting records.





Applications of JDBC

- Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database. Java can be used to write different types of executables, such as,
 - Java Applications
 - Java Applets
 - Java Servlets
 - Java ServerPages (JSPs)
 - Enterprise JavaBeans (EJBs).
- All of these different executables are able to use a JDBC driver to access a database, and take advantage of the stored data.
- JDBC provides the same capabilities as ODBC, allowing Java programs to contain database independent code.

JDBC stands for:

- (a) Java Database Communication
- (b) Java Database Connectivity
- (c) Java Data Control
- (d) Java Driver Connection

JDBC stands for:

- (a) Java Database Communication
- (b) Java Database Connectivity
- (c) Java Data Control
- (d) Java Driver Connection

Answer: (b) Java Database Connectivity

Explanation:

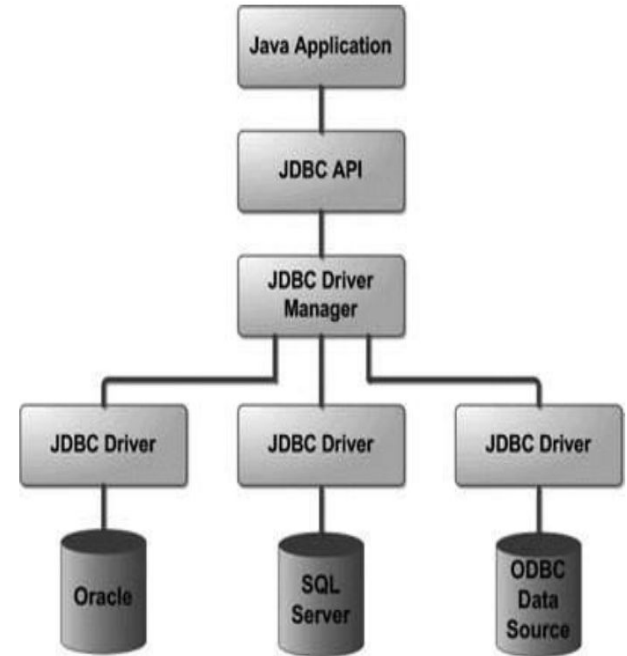
JDBC is an API (Application Programming Interface) that enables Java applications to connect and interact with relational databases using SQL commands.

JDBC Architecture

- The JDBC API supports both two-tier and three-tier processing models for database access but in general, JDBC Architecture consists of two layers –

- **JDBC API:** This provides the application-to-JDBC Manager connection.

- **JDBC Driver API:** This supports the JDBC Manager-to-Driver Connection.
- The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.
- The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.



The JDBC API is defined in which package?

- (a) java.net
- (b) java.sql
- (c) java.io
- (d) javax.jdbc

The JDBC API is defined in which package?

- (a) java.net
- (b) java.sql
- (c) java.io
- (d) javax.jdbc

Answer: (b) java.sql

Explanation:

All JDBC classes and interfaces such as `Connection`, `Statement`, `PreparedStatement`, and `ResultSet` are part of the `java.sql` package.

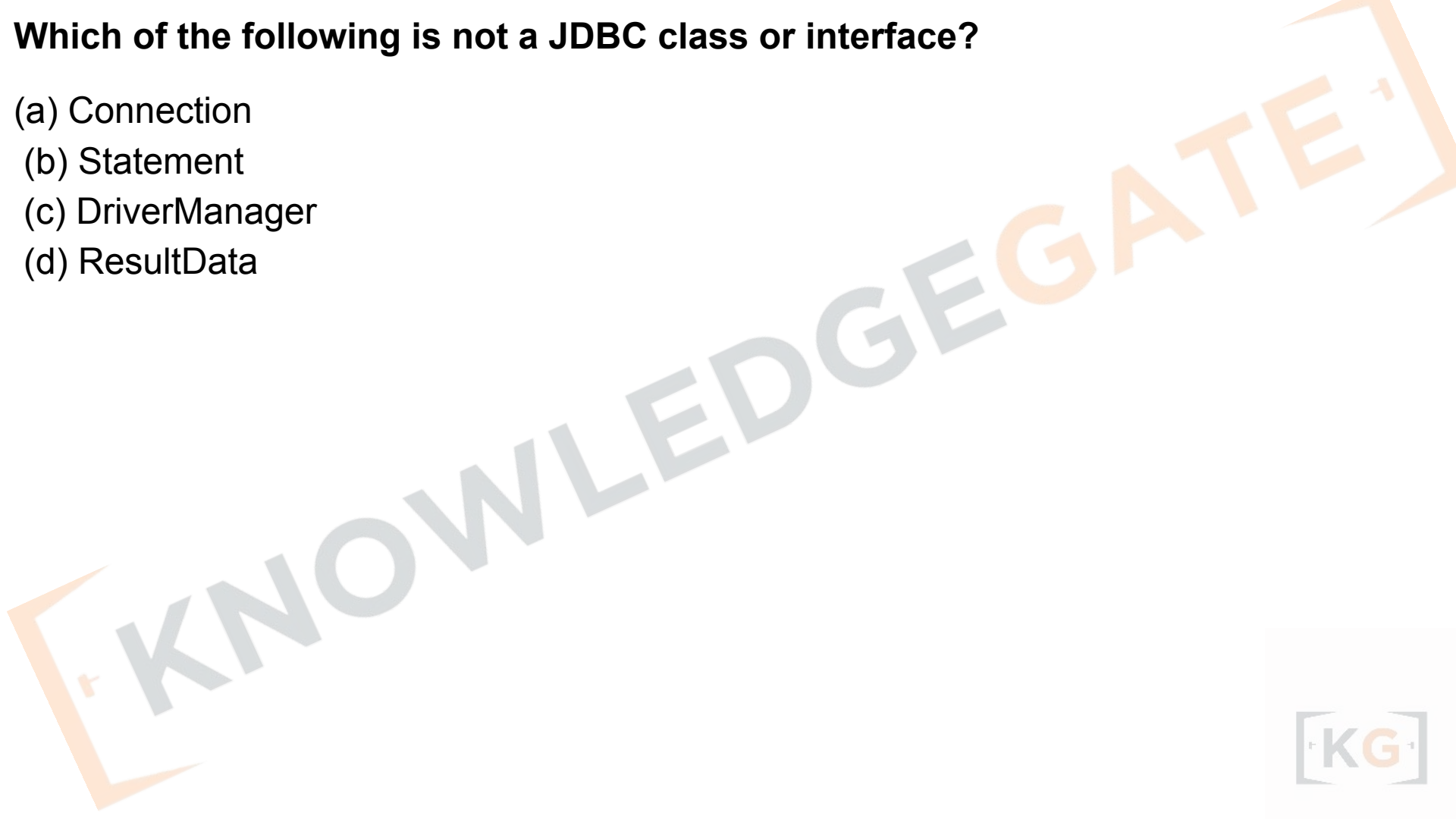
JDBC Components

The JDBC API provides the following interfaces and classes –

- **DriverManager:** This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.
- **Driver:** This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.
- **Connection:** This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement:** You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet:** These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
- **SQLException:** This class handles any errors that occur in a database application.

Which of the following is not a JDBC class or interface?

- (a) Connection
- (b) Statement
- (c) DriverManager
- (d) ResultData



Which of the following is not a JDBC class or interface?

- (a) Connection
- (b) Statement
- (c) DriverManager
- (d) ResultData

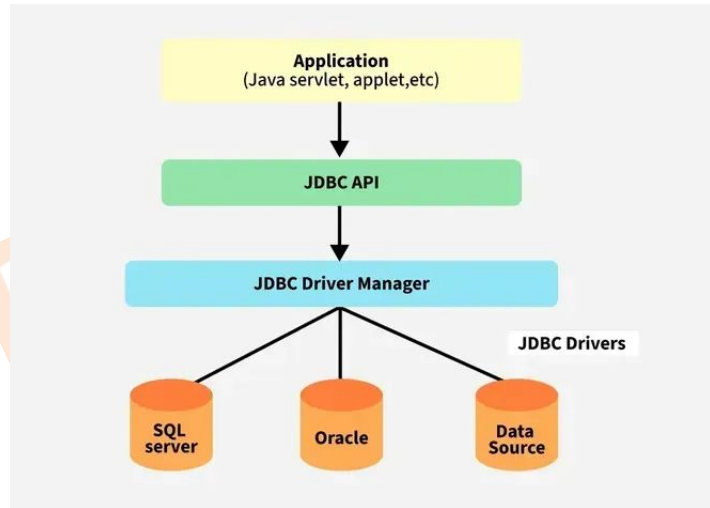
Answer: (d) ResultData

Explanation:

ResultData does not exist in JDBC. The correct interface for holding query results is **ResultSet**.

JDBC Driver

- JDBC drivers implement the defined interfaces in the JDBC API, for interacting with your database server.
- For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.
- The **Java.sql** package that ships with JDK, contains various classes with their behaviours defined and their actual implementations are done in third-party drivers. Third party vendors implement the java.sql.Driver interface in their database driver.



JDBC Drivers Types

- JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates.

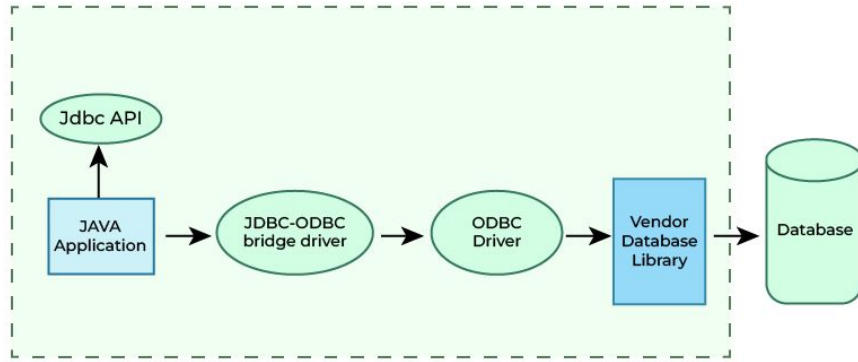
JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand.

There are 4 types of JDBC drivers:

1. Type-1 driver or JDBC-ODBC bridge driver
2. Type-2 driver or Native-API driver (partially java driver)
3. Type-3 driver or Network Protocol driver (fully java driver)
4. Type-4 driver or Thin driver (fully java driver) - It is a widely used driver. The older drivers like (JDBC-ODBC) bridge driver have been deprecated and no longer supported in modern versions of Java.

Type 1: JDBC-ODBC Bridge Driver

Type-1 driver or JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. Type-1 driver is also called Universal driver because it can be used to connect to any of the databases.



Type 1: JDBC-ODBC Bridge Driver

Advantages

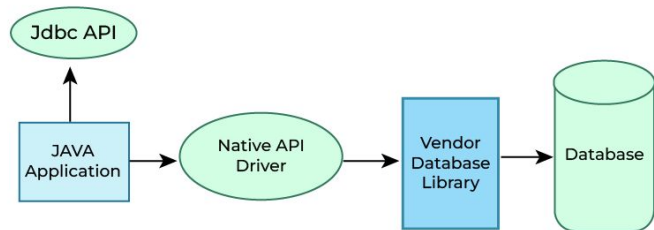
- This driver software is built-in with JDK so no need to install separately.
- It is a database independent driver.

Disadvantages

- As a common driver is used in order to interact with different databases, the data transferred through this driver is not so secured.
- The ODBC bridge driver is needed to be installed in individual client machines.
- Type-1 driver isn't written in java, that's why it isn't a portable driver.

Type 2: JDBC-Native API

- In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls, which are unique to the database. These drivers are typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge. The vendor-specific driver must be installed on each client machine.
- If we change the Database, we have to change the native API. The Native API driver uses the client-side libraries of the database. In order to interact with different database, this driver needs their local API, that's why data transfer is much more secure as compared to type-1 driver. This driver is not fully written in Java that is why it is also called Partially Java driver.
- The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

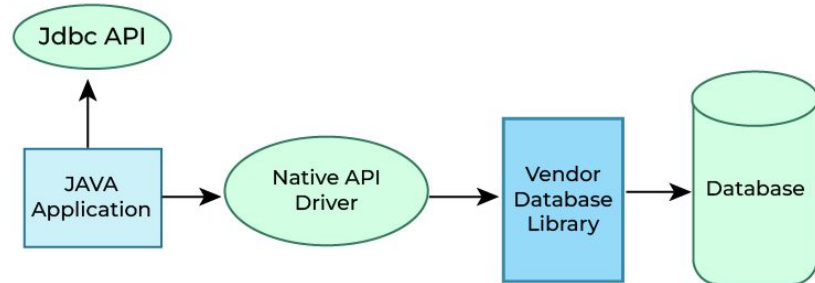


Advantage

- Native-API driver gives better performance than JDBC-ODBC bridge driver.
- More secure compared to the type-1 driver.

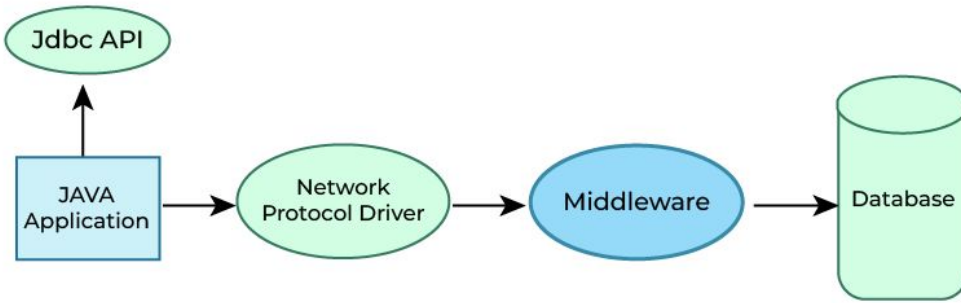
Disadvantages

- Driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- Type-2 driver isn't written in java, that's why it isn't a portable driver
- It is a database dependent driver.



Type 3: JDBC-Net pure Java

- In a Type 3 driver, a three-tier approach is used to access databases. The JDBC clients use standard network sockets to communicate with a middleware application server.
 - The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server.
- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. Here all the database connectivity drivers are present in a single server, hence no need of individual client-side installation.



Type 3: JDBC-Net pure Java

Advantages

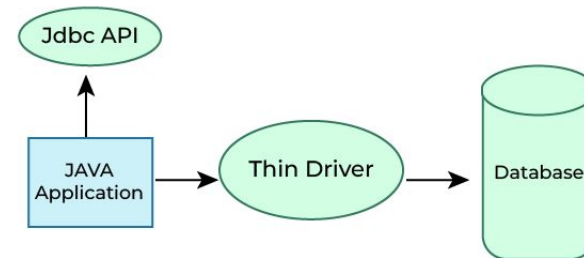
- Type-3 drivers are fully written in Java, hence they are portable drivers.
- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- Easy to switch databases

Disadvantages

- Network support is required on client machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

Type 4: 100% Pure Java

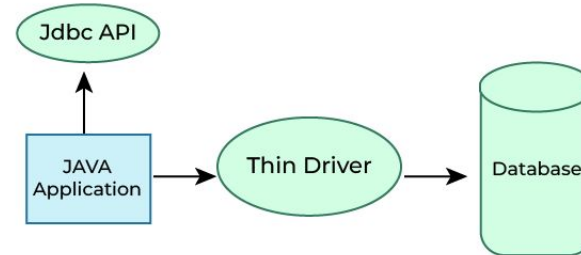
- Type-4 driver is also called native protocol driver. This driver interact directly with database. It does not require any native database library, that is why it is also known as Thin Driver. In a Type 4 driver, a pure Java-based driver communicates directly with the vendor's database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself.
- This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.
- MySQL's Connector/J driver is a Type 4 driver.



Type 4: 100% Pure Java

Advantages

- Does not require any native library and Middleware server, so no client-side or server-side installation.
- It is fully written in Java language, hence they are portable drivers.
- If the database changes, a new driver may be needed.



When to Use Which Driver?

- If you are accessing one type of database, such as Oracle, Sybase or IBM, the preferred driver type is type-4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations, where a type 3 or type 4 driver is not available yet for your database.
- The type 1 driver is not considered a deployment-level driver and is typically used for development and testing purposes only.

Which of the following statements about the JDBC-ODBC Bridge is correct?

- (a) It is a Type 4 driver written in pure Java.
- (b) It is a Type 1 driver that uses ODBC driver internally.
- (c) It is a Type 2 driver using native API.
- (d) It is used for servlet connections only.

Which of the following statements about the JDBC-ODBC Bridge is correct?

- (a) It is a Type 4 driver written in pure Java.
- (b) It is a Type 1 driver that uses ODBC driver internally.
- (c) It is a Type 2 driver using native API.
- (d) It is used for servlet connections only.

Answer: (b)

Explanation:

The **JDBC-ODBC Bridge** (Type 1 driver) translates JDBC calls into ODBC calls and relies on the ODBC driver for database communication. It is platform-dependent and deprecated in Java 8 and later.

Type IV JDBC driver is a driver

- (a) Which is written in C++
- (b) Which requires an intermediate layer
- (c) Which communicates through Java sockets
- (d) Which translates JDBC function calls into API and native to DBMS

ISRO Scientist/Engineer 17.12.2017

Type IV JDBC driver is a driver

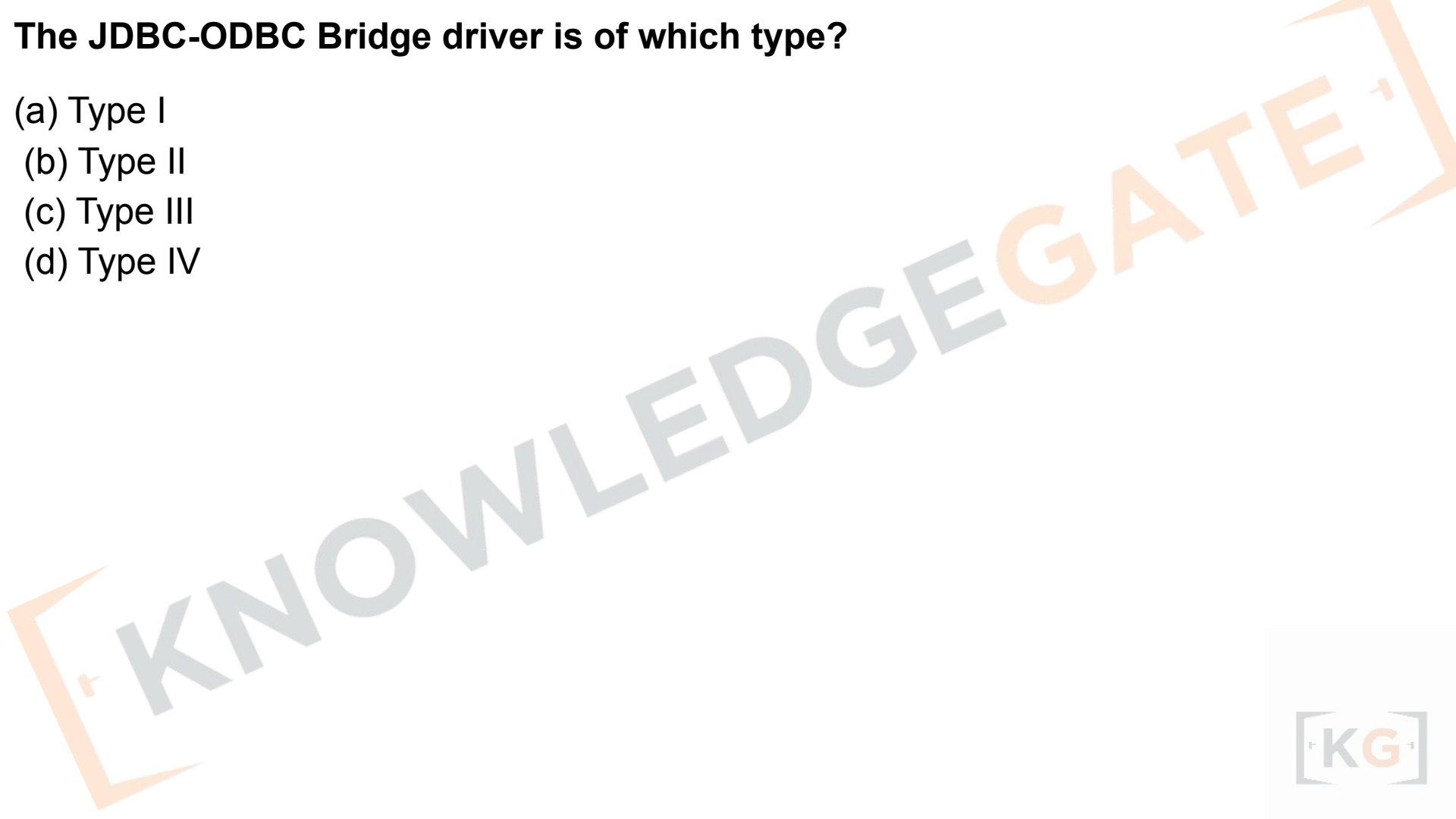
- (a) Which is written in C++
- (b) Which requires an intermediate layer
- (c) Which communicates through Java sockets
- (d) Which translates JDBC function calls into API and native to DBMS

ISRO Scientist/Engineer 17.12.2017

Ans. (c) :

The JDBC-ODBC Bridge driver is of which type?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV



The JDBC-ODBC Bridge driver is of which type?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV

Answer: (a)

Explanation:

The **JDBC-ODBC Bridge** is a **Type 1** driver that converts JDBC calls into ODBC calls using the ODBC driver.

It is **platform-dependent**, slow, and **deprecated from Java 8 onwards**.

Which JDBC driver type is called a "Pure Java" or "Thin Driver"?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV

Which JDBC driver type is called a "Pure Java" or "Thin Driver"?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV

Answer: (d)

Explanation:

The **Type 4 driver** is a **pure Java driver** that converts JDBC calls directly into the database's native protocol.

It is **platform-independent**, **fast**, and **widely used** in modern applications (e.g., MySQL and Oracle drivers).

Which of the following driver types uses middleware (application server) to communicate with the database?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV

Which of the following driver types uses middleware (application server) to communicate with the database?

- (a) Type I
- (b) Type II
- (c) Type III
- (d) Type IV

Answer: (c)

Explanation:

Type 3 driver (Network Protocol driver) uses a **middleware server** that translates JDBC calls into database-specific calls.

It is portable and useful in large distributed environments.

Summary Table – JDBC Driver Types

Type	Name	Description	Dependency	Example
Type 1	JDBC-ODBC Bridge	Converts JDBC to ODBC calls	Platform-dependent	Deprecated
Type 2	Native-API Driver	Uses native DB API libraries	Platform-dependent	Oracle OCI
Type 3	Network Protocol Driver	Uses middleware server	Platform-independent	IBM WebSphere
Type 4	Thin Driver (Pure Java)	Direct DB communication	Platform-independent	MySQL, Oracle

Creating JDBC Application

There are following six steps involved in building a JDBC application:

- Import the packages: Requires that you include the packages containing the JDBC classes needed for database programming. Most often, using import **java.sql.*** will suffice.
- Register the JDBC driver: Requires that you initialize a driver so you can open a communication channel with the database.

[1] IMPORT PACKAGES

```
import java.sql.*;
```



[2] REGISTER DRIVER (load JDBC driver)

```
Class.forName("com.mysql.cj.jdbc.Driver"); // Type 4 ex.  
// (Many modern drivers auto-register)
```



Creating JDBC Application

- Open a connection: Requires using the `DriverManager.getConnection()` method to create a `Connection` object, which represents a physical connection with the database.
- Execute a query: Requires using an object of type `Statement` for building and submitting an SQL statement to the database.

[3] OPEN CONNECTION (get a DB session)

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://host:3306/db", "user", "pass");
```

|



[4] EXECUTE QUERY (create stmt + run SQL)

```
Statement st = con.createStatement();  
ResultSet rs = st.executeQuery("SELECT * FROM Student");  
// for DML: int n = st.executeUpdate("UPDATE ...");
```

|



Creating JDBC Application

- Extract data from result set: Requires that you use the appropriate `ResultSet.getXXX()` method to retrieve the data from the result set.
- Clean up the environment: Requires explicitly closing all database resources versus relying on the JVM's garbage collection.

[5] EXTRACT DATA (read from ResultSet cursor)

```
while (rs.next()) {  
    int id = rs.getInt(1); // or rs.getInt("id")  
    String name = rs.getString(2); // getXXX(...)  
}
```

|



[6] CLEAN UP (ALWAYS close resources)

```
rs.close();  
st.close();  
con.close();  
// Tip: use try-with-resources to auto-close
```

Which method is used to establish a database connection in JDBC?

- (a) connect()
- (b) getConnection()
- (c) makeConnection()
- (d) establishConnection()

Which method is used to establish a database connection in JDBC?

- (a) connect()
- (b) getConnection()
- (c) makeConnection()
- (d) establishConnection()

Answer: (b) getConnection()

Explanation:

`DriverManager.getConnection()` establishes a connection between the Java program and the database using a connection URL, username, and password.

What is the role of the DriverManager class in JDBC?

- (a) It defines SQL syntax for databases.
- (b) It manages and loads database drivers and establishes connections.
- (c) It executes queries and stores results.
- (d) It represents a database table.

What is the role of the DriverManager class in JDBC?

- (a) It defines SQL syntax for databases.
- (b) It manages and loads database drivers and establishes connections.
- (c) It executes queries and stores results.
- (d) It represents a database table.

Answer: (b)

Explanation:

DriverManager is responsible for **loading and managing JDBC drivers** and for establishing connections using the registered driver.

Which JDBC interface is used to execute SQL statements?

- (a) Driver
- (b) Connection
- (c) Statement
- (d) ResultSet

Which JDBC interface is used to execute SQL statements?

- (a) Driver
- (b) Connection
- (c) Statement
- (d) ResultSet

Answer: (c)

Explanation:

Statement is used to send SQL queries to the database. It has methods like **executeQuery()** for SELECT and **executeUpdate()** for INSERT/UPDATE/DELETE statements.

Which JDBC interface stores the result returned by a SELECT query?

- (a) DriverManager
- (b) ResultSet
- (c) PreparedStatement
- (d) Connection

Which JDBC interface stores the result returned by a SELECT query?

- (a) DriverManager
- (b) ResultSet
- (c) PreparedStatement
- (d) Connection

Answer: (b)

Explanation:

A **ResultSet** object holds the data retrieved from the database. It provides methods like **next()**, **getInt()**, and **getString()** to iterate and fetch column values.

Chapter - 10

Introduction to servlets.

What is a Servlet?

A Servlet is basically a Java program that runs on the server side and helps create dynamic web applications.

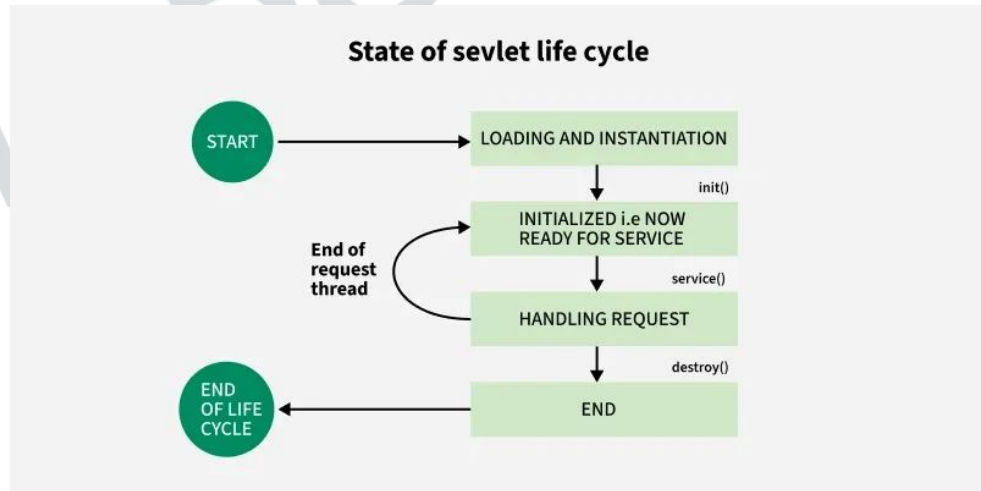
Here's what the slide is saying in simple terms:

1. **Servlet is a technology** → Used to create web applications.
2. **Servlet is an API** → Provides predefined interfaces and classes (like a toolkit) to build Servlets.
3. **Servlet is an interface** → Must be implemented when creating a Servlet.
4. **Servlet is a class** → Extends the server's capabilities by handling requests (like HTTP requests) and sending back **responses**.
5. **Servlet is a web component** → Runs on a web server (like Tomcat) and generates dynamic web pages (unlike static HTML pages).

States of the Servlet Life Cycle

The Servlet life cycle consists of four main stages:

- Loading a Servlet
- Initializing a Servlet
- Handling Client Requests
- Destroying a Servlet

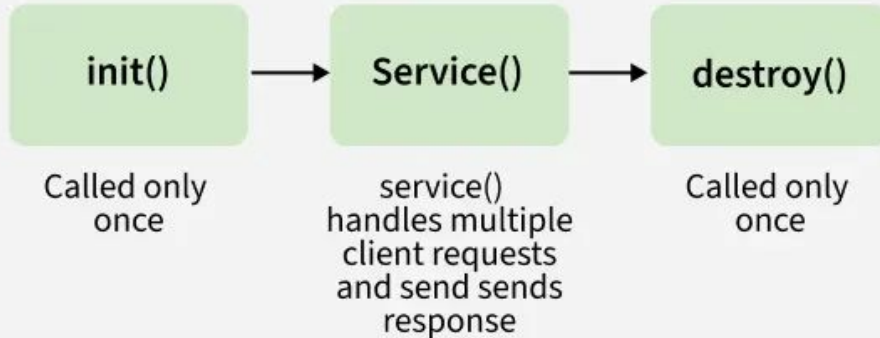


Servlet Life Cycle Methods

There are three life cycle methods of a Servlet:

- `init()`
- `service()`
- `destroy()`

Life Cycle of Servlet



Servlet Life Cycle Methods

init() Method

- Invoked once when the Servlet is instantiated.
- Used to initialize resources (e.g., database connections).
- Prefer the non-parameterized init() method for fewer redundant calls.

- **Example:**

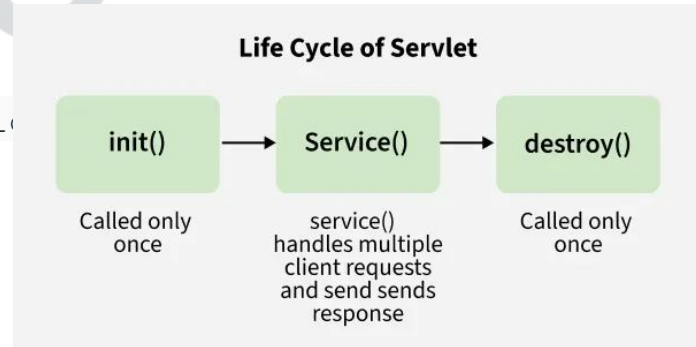
a. `@Override`

b. `public void init() throws ServletException`

c. `// Initialization code`

d. `}`

- Exception handling is possible in init().
- Constructor is not recommended for initialization because it cannot throw ServletException



Servlet Life Cycle Methods

service() Method

- Handles client requests and responses.
- Determines HTTP request type (GET, POST, PUT, DELETE) and delegates to `doGet()`, `doPost()`, etc.

Example:

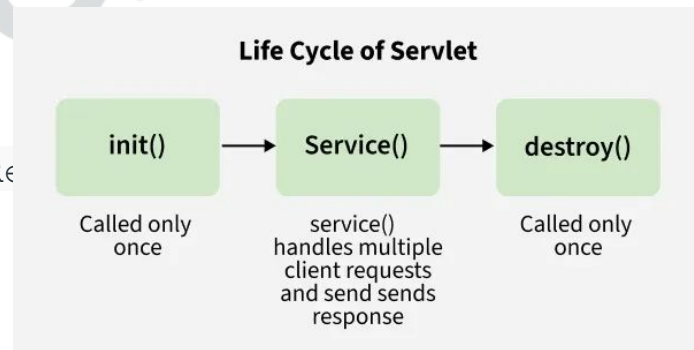
```
@Override
```

```
public void service(ServletRequest req, ServletResponse res)
```

```
throws ServletException, IOException {
```

```
    // Request handling code
```

```
}
```



Servlet Life Cycle Methods

destroy() Method

- Called once at the end of the Servlet's life cycle.
- Cleans up resources (e.g., closes DB connections, releases memory).

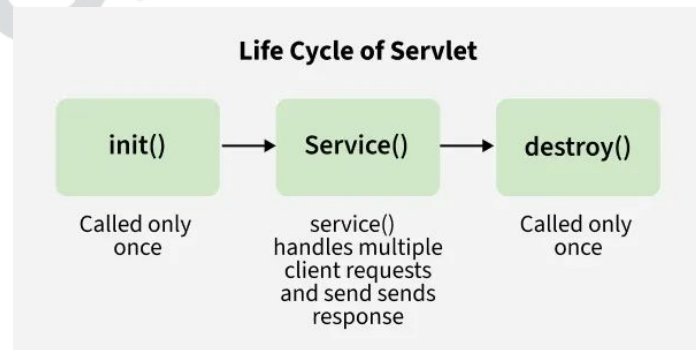
Example:

```
@Override
```

```
public void destroy() {
```

```
    // Cleanup code
```

```
}
```



The lifecycle of a Servlet is managed by:

- (a) Web Client
- (b) Servlet Engine (Container)
- (c) Database Server
- (d) Java Compiler

The lifecycle of a Servlet is managed by:

- (a) Web Client
- (b) Servlet Engine (Container)
- (c) Database Server
- (d) Java Compiler

Answer: (b)

Explanation:

The **Servlet Container (e.g., Tomcat)** manages servlet life cycle — loading, initialization, servicing requests, and destruction.

Which method is called only once in the lifecycle of a servlet when it is first loaded into memory?

- (a) start()
- (b) init()
- (c) service()
- (d) destroy()

Which method is called only once in the lifecycle of a servlet when it is first loaded into memory?

- (a) start()
- (b) init()
- (c) service()
- (d) destroy()

Answer: (b)

Explanation:

The **init()** method initializes the servlet and is invoked only once when the servlet is first loaded into the container.

Q. Which method is called only once in the lifecycle of a servlet when it is first loaded into memory?

- A.** start()
- B.** init()
- C.** service()
- D.** destroy()

Q. Which method is called only once in the lifecycle of a servlet when it is first loaded into memory?

- A.** start()
- B.** init()
- C.** service()
- D.** destroy()

Answer: (b)

Explanation:

The **init()** method initializes the servlet and is invoked only once when the servlet is first loaded into the container.

Q. The `destroy()` method of a servlet is called:

- A. When a servlet is first initialized
- B. When the servlet is idle
- C. Once, just before servlet is removed from memory
- D. Every time the servlet completes a request

Q. The `destroy()` method of a servlet is called:

- A. When a servlet is first initialized
- B. When the servlet is idle
- C. Once, just before servlet is removed from memory
- D. Every time the servlet completes a request

Answer: (c)

Explanation:

`destroy()` is called **only once**, when the servlet is being **unloaded from memory**, to release resources.

Q. A Servlet is:

- A.** A Java program that runs on a client machine
- B.** A Java program that runs on a web server
- C.** A Java applet running in a browser
- D.** A Java compiler extension

Q. A Servlet is:

- A.** A Java program that runs on a client machine
- B.** A Java program that runs on a web server
- C.** A Java applet running in a browser
- D.** A Java compiler extension

Answer: (b)

Explanation:

A **Servlet** is a **server-side Java program** that handles client requests and generates responses, typically over HTTP.

Q. Servlets are part of which Java technology?

- A.** Java Foundation Classes (JFC)
- B.** Java Enterprise Edition (JEE)
- C.** Java Standard Edition (JSE)
- D.** Java Micro Edition (JME)

Q. Servlets are part of which Java technology?

- A.** Java Foundation Classes (JFC)
- B.** Java Enterprise Edition (JEE)
- C.** Java Standard Edition (JSE)
- D.** Java Micro Edition (JME)

Answer: (b)

Explanation:

Servlets belong to the **Java EE (Jakarta EE)** platform — the enterprise framework for building server-side web applications.

HTTP (Hyper Text Transfer Protocol)

- Http is the protocol that allows web servers and browsers to exchange data over the web.
- It is a request response protocol.
- Http uses reliable TCP connections by default on TCP port 80.
- *It is stateless means each request is considered as the new request. In other words, server doesn't recognize the user by default*

Http Request Methods

Every request has a header that tells the status of the client. There are many request methods. Get and Post requests are mostly used.

HTTP Request	Description
GET	Asks to get the resource at the requested URL.
POST	Asks the server to accept the body info attached. It is like GET request with extra info sent with the request.
HEAD	Asks for only the header part of whatever a GET would return. Just like GET but with no body.
TRACE	Asks for the loopback of the request message, for testing or troubleshooting.
PUT	Says to put the enclosed info (the body) at the requested URL.
DELETE	Says to delete the resource at the requested URL.
OPTIONS	Asks for a list of the HTTP methods to which the thing at the request URL can respond.

Content Type

Content Type is also known as MIME (Multipurpose internet Mail Extension) Type.

It is a HTTP header that provides the description about what are you sending to the browser.

There are many content types:

- text/html
- text/plain
- application/msword
- application/vnd.ms-excel
- application/jar
- application/pdf
- application/octet-stream
- application/x-zip
- Images/jpeg
- video/quicktime etc.



Servlet API

- The `javax.servlet` and `javax.servlet.http` packages represent interfaces and classes for servlet api.
- The **`javax.servlet`** package contains many interfaces and classes that are used by the servlet or web container.
- The **`javax.servlet.http`** package contains interfaces and classes that are responsible for http requests only.

Interfaces in javax.servlet package

- ServletRequest
- ServletResponse
- RequestDispatcher
- ServletConfig
- ServletContext
- SingleThreadModel
- Filter
- FilterConfig
- FilterChain
- ServletRequestListener
- ServletRequestAttributeListener
- ServletContextListener
- ServletContextAttributeListener

Classes in javax.servlet package

There are many classes in javax.servlet package. They are as follows:

- GenericServlet
- ServletInputStream
- ServletOutputStream
- ServletRequestWrapper
- ServletResponseWrapper
- ServletRequestEvent
- ServletContextEvent
- ServletRequestAttributeEvent
- ServletContextAttributeEvent
- ServletException
- UnavailableException

Interfaces in javax.servlet.http package

There are many interfaces in javax.servlet.http package. They are as follows:

- HttpServletRequest
- HttpServletResponse
- HttpSession
- HttpSessionListener
- HttpSessionAttributeListener
- HttpSessionBindingListener
- HttpSessionActivationListener
- HttpSessionContext (deprecated now)

Classes in javax.servlet.http package

There are many classes in javax.servlet.http package. They are as follows:

- HttpServlet
- Cookie
- HttpServletRequestWrapper
- HttpServletResponseWrapper
- HttpSessionEvent
- HttpSessionBindingEvent
- HttpUtils (deprecated now)

Servlet Interface

- Servlet interface provides common behaviour to all the servlets. Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).

Method	Description
public void init(ServletConfig config)	Initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.
public void service(ServletRequest request, ServletResponse response)	Provides response for the incoming request. It is invoked at each request by the web container.
public void destroy()	Is invoked only once and indicates that servlet is being destroyed.
public ServletConfig getServletConfig()	Returns the object of ServletConfig.
public String getServletInfo()	Returns information about servlet such as writer, copyright, version etc.

Q. Which package contains the core classes and interfaces for Servlets?

- A. `java.servlet.*`
- B. `javax.servlet.*`
- C. [java.net.*](#)
- D. `javax.net.*`

Q. Which package contains the core classes and interfaces for Servlets?

- A. `java.servlet.*`
- B. `javax.servlet.*`
- C. [java.net.*](#)
- D. `javax.net.*`

Answer: (b)

Explanation:

All servlet classes and interfaces are in the `javax.servlet` and `javax.servlet.http` packages.

Q. Which interface must be implemented by all Servlets?

- A.** ServletContext
- B.** ServletRequest
- C.** Servlet
- D.** HttpServlet

Q. Which interface must be implemented by all Servlets?

- A. ServletContext
- B. ServletRequest
- C. Servlet
- D. HttpServlet

Answer: (c)

Explanation:

All Servlets must implement the **javax.servlet.Servlet** interface either directly or indirectly through **GenericServlet** or **HttpServlet**.

Q. By which technology do we separate our business logic from the presentation logic?
(NIELIT Scientists-B 04.12.2016 (CS))

- A.** Servlet
- B.** JSP
- C.** Both (A) & (B)
- D.** None of the above

Ans. (b)

Q. By which technology do we separate our business logic from the presentation logic?
(NIELIT Scientists-B 04.12.2016 (CS))

- A.** Servlet
- B.** JSP
- C.** Both (A) & (B)
- D.** None of the above

Ans. (b)

MVC Architecture (Model–View–Controller)

In the **MVC model** used for Java web applications:

- **Model** → Represents data & business logic (Java classes, beans, EJBs)
- **View** → JSP pages (handles presentation)
- **Controller** → Servlets (handles control logic, request/response flow)

Q. Which class provides a generic, protocol-independent implementation of the Servlet interface?

- A.** HttpServlet
- B.** GenericServlet
- C.** ServletConfig
- D.** ServletContext

Q. Which class provides a generic, protocol-independent implementation of the Servlet interface?

- A.** HttpServlet
- B.** GenericServlet
- C.** ServletConfig
- D.** ServletContext

Answer: (b)

Explanation:

GenericServlet is an abstract class that implements the **Servlet** and **ServletConfig** interfaces and provides a framework for any protocol (not just HTTP).

Q. Which class should be extended to create an HTTP-specific servlet?

- A.** `HttpServletRequest`
- B.** `GenericServlet`
- C.** `HttpServlet`
- D.** `ServletRequest`

Q. Which class should be extended to create an HTTP-specific servlet?

- A.** `HttpServletRequest`
- B.** `GenericServlet`
- C.** `HttpServlet`
- D.** `ServletRequest`

Answer: (c)

Explanation:

Most web applications extend **`HttpServlet`**, which provides methods like **`doGet()`** and **`doPost()`** for handling HTTP requests.

Q. Which of the following methods in HttpServlet is used to handle HTTP GET requests?

- A.** service()
- B.** doGet()
- C.** doPost()
- D.** getRequest()

Q. Which of the following methods in HttpServlet is used to handle HTTP GET requests?

- A.** service()
- B.** doGet()
- C.** doPost()
- D.** getRequest()

Answer: (b)

Explanation:

doGet() handles HTTP **GET** requests sent by the client browser. Similarly, **doPost()** handles **POST** requests.

Q. What is the purpose of the `ServletConfig` object?

- A.** To store initialization parameters for a servlet
- B.** To handle client requests
- C.** To store user session information
- D.** To define servlet context attributes

Q. What is the purpose of the `ServletConfig` object?

- A. To store initialization parameters for a servlet
- B. To handle client requests
- C. To store user session information
- D. To define servlet context attributes

Answer: (a)

Explanation:

`ServletConfig` provides initialization parameters and configuration information specific to a servlet instance.

Q. The `service()` method of `HttpServlet` internally calls which methods based on the HTTP request type?

- A.** `init()` and `destroy()`
- B.** `doGet()` and `doPost()`
- C.** `init()` and `service()`
- D.** `service()` and `doInit()`

Q. The `service()` method of `HttpServlet` internally calls which methods based on the HTTP request type?

- A.** `init()` and `destroy()`
- B.** `doGet()` and `doPost()`
- C.** `init()` and `service()`
- D.** `service()` and `doInit()`

Answer: (b)

Explanation:

In `HttpServlet`, the `service()` method is already implemented to dispatch requests to the correct method (`doGet()`, `doPost()`, etc.) based on the HTTP request type.

Q. Which of the following objects provides information about the servlet's environment and context?

- A.** ServletRequest
- B.** ServletContext
- C.** ServletConfig
- D.** HttpSession

Q. Which of the following objects provides information about the servlet's environment and context?

- A.** ServletRequest
- B.** ServletContext
- C.** ServletConfig
- D.** HttpSession

Answer: (b)

Explanation:

ServletContext provides access to web application–wide information such as initialization parameters and environment setup shared by all servlets.

Q. Match the following interfaces of Java Servlet package: (UGC NET June 2014)

LIST-I

- a. Servlet Config
- b. Servlet Context
- c. Servlet Request
- d. Servlet Response

LIST-II

- i. Enables Servlets to log events
- ii. Read data from a client
- iii. Write data to a client
- iv. To get initialization parameters

A. a-iii; b-iv; c-ii; d-i

B. a-iii; b-ii; c-iv; d-i

C. a-ii; b-iii; c-iv; d-i

D. a-iv; b-i; c-ii; d-iii